2026 届 本 科生毕业论文　　存档编号_____

湖北文理学院

毕 业 论 文

论文题目　基于 Apollo 的多障碍物场景下路径规划算法实现

学　　院　计算机工程学院

专　　业　计算机科学与技术

班　　级　计科 2211

学　　号　2022117117

学生姓名　廖嘉旺

指导教师　孙成娇

2026 年 5 月

材 料 清 单

一、毕业论文

二、毕业设计任务书

三、毕业设计开题申请表

四、毕业设计开题报告正文

五、外文文献原文及译文

声 明

本人廖嘉旺，学号 2022117117，系湖北文理学院计算机工程学院计算机科学与技术专业计科 2211 班学生，指导教师孙成娇。所做论文内容主体均为原创，无任何抄袭、剽窃他人劳动成果的行为。如有发现此类行为，本人愿意为此承担一切道义及法律责任。

特此声明。

学生签名：_____

2025 年 5 月 28 日

基于 Apollo 的多障碍物场景下路径规划算法实现

摘要：自动驾驶技术的快速发展对路径规划算法提出了更高要求，尤其是在多障碍物密集场景下，现有规划算法面临路径阻塞、决策保守和通行效率低下等挑战。本文以百度 Apollo 开源自动驾驶平台为研究基础，深入分析了其 Planning 模块的架构设计与核心算法实现，从安全距离、决策粒度和时空协调三个维度系统识别了 PathBoundsDecider 在多障碍物场景下的规划缺陷，即安全裕度一刀切压缩通行空间、逐障碍物独立决策导致矛盾避让方向、动态障碍物被完全排除在路径决策之外导致路径-速度协调缺失。

针对上述三个问题，本文建立了时变最优通行带的数学模型，将多障碍物避让方向选择建模为组合优化问题，并提出了系统性的改进方案。在 SL 层，提出多因子威胁度评估模型实现差异化安全距离分配，提出基于扫描线的时变障碍物分组算法通过纵向重叠检测将相互关联的障碍物聚为连通分量并在组级执行全局最优决策，证明了最优避让方向分割在横向排序后具有连续性并进而证明了最大间隙策略的最优性，将组合搜索复杂度从 $O(2^k)$ 降至 $O(k \log k)$。在 ST 层，引入到达时间函数和预测轨迹将动态障碍物纳入路径边界决策，构建时空可通行走廊并将其边界约束与 SpeedBoundsDecider 的 ST 边界在速度 QP 层面融合，实现路径-速度协调规划。

仿真实验结果表明，与 Apollo 原始算法相比，改进算法在多障碍物场景下将平均规划成功率从 79.8% 提升至 95.6%，平均通行速度提升 31.7%，平均停车次数减少 81.5%，算法总复杂度 $O(n \log n)$，满足实时性要求。

关键词：自动驾驶；路径规划；Apollo；多障碍物避让；组合优化

Implementation of Path Planning Algorithm in Multi-obstacle Scenarios Based on Apollo

Abstract: The rapid development of autonomous driving technology has raised higher requirements for path planning algorithms, especially in dense multi-obstacle scenarios where existing algorithms suffer from path blockage, conservative decisions, and low traffic efficiency. Based on Baidu's Apollo open-source autonomous driving platform, this thesis conducts a source-code-level analysis of its Planning module and systematically identifies three planning deficiencies in PathBoundsDecider from the dimensions of safety margin, decision granularity, and spatiotemporal coordination: a uniform safety buffer that unnecessarily compresses the passage band, independent per-obstacle nudge decisions that produce contradictory avoidance directions, and complete exclusion of dynamic obstacles from path decisions leading to path-speed coordination failure.

To address these three problems, this thesis establishes a time-varying optimal passage band mathematical model, formulating the multi-obstacle nudge direction selection as a combinatorial optimization problem, and proposes a systematic improvement scheme. At the SL layer, a multi-factor threat assessment model is proposed for differentiated safety margin allocation; a sweep-line-based time-varying obstacle grouping algorithm clusters longitudinally overlapping obstacles into connected components and makes globally optimal decisions at the group level; the contiguity of the optimal partition in lateral order (Lemma) and the optimality of the maximum-gap strategy (Theorem) are proved, reducing the search space from $O(2^k)$ to $O(k \log k)$. At the ST layer, arrival time functions and predicted trajectories are incorporated to include dynamic obstacles in path boundary decisions; a spatiotemporal passable corridor is constructed and its boundary constraints are fused with SpeedBoundsDecider's ST boundaries at the speed QP level, achieving path-speed coordinated planning.

Simulation results demonstrate that compared with Apollo's original algorithm, the proposed algorithm improves average planning success rate from 79.8% to 95.6%, increases average traffic speed by 31.7%, and reduces average stop count by 81.5%, with $O(n \log n)$ computational complexity satisfying real-time requirements.

Keywords: Autonomous Driving; Path Planning; Apollo; Multi-obstacle; Optimal Passage Band; Obstacle Grouping; Combinatorial Optimization

目　　录

摘要 ..... **II**
Abstract ..... **III**
1　绪论 **1**
1.1　研究背景与意义 1
1.2　国内外研究现状 3
1.3　研究内容与技术路线 9
1.4　论文组织结构 11
2　相关技术与理论基础 **12**
2.1　Frenet 坐标系 12
2.2　凸优化与二次规划 14
2.3　SL 图与路径边界 17
2.4　ST 图与速度规划基础 18
2.5　Apollo 平台架构概述 20
2.6　组合优化与区间图基础 22
2.7　本章小结 23
3　Apollo Planning 模块深度分析 **25**
3.1　Planning 模块总体架构 25
3.2　ReferenceLine 平滑 27
3.3　路径-速度解耦两阶段优化 30
3.4　PathDecider 决策分析 33
3.5　SpeedBoundsDecider 概述 36
3.6　本章小结 37
4　多障碍物场景下的问题分析 **38**
4.1　问题概述 38
4.2　安全裕度一刀切压缩通行空间 39
4.3　SL 层路径边界独立决策缺陷 42
4.4　ST 层路径-速度协调缺失 47
4.5　问题总结与改进方向 50
4.6　本章小结 51
5　改进算法设计与实现 **52**
5.1　时变最优通行带问题建模 52
5.2　Apollo Baseline 的次优性分析 55
5.3　多因子威胁度评估与动态安全距离 59
5.4　SL 层基于重叠分组的最优通行带 65
5.5　ST 层时空可通行走廊 72
5.6　与 Apollo 系统的集成 85
5.7　本章小结 88

6 实验与结果分析 90
6.1 实验环境与配置 90
6.2 测试场景设计 91
6.3 评价指标 93
6.4 实验结果与分析 93
6.5 本章小结 99
7 总结与展望 100
7.1 研究工作总结 100
7.2 主要创新点 101
7.3 研究局限性 103
7.4 未来研究方向 103
参考文献 105
致谢 110

1 绪论

1.1 研究背景与意义

1.1.1 研究背景

随着人工智能、传感器技术和高性能计算的飞速发展，自动驾驶技术已从实验室概念验证阶段逐步迈入大规模道路测试和商业化落地阶段。根据国际自动机工程师学会（Society of Automotive Engineers, SAE）的分级标准，自动驾驶系统分为 L0 至 L5 共六个等级[1]，其中 L3 及以上等级要求系统能够在特定或全部场景下独立完成驾驶任务。路径规划模块作为自动驾驶系统感知—决策—控制链路中的核心环节，负责根据环境信息生成安全、高效、舒适的行驶轨迹，其性能直接决定了自动驾驶系统的安全水平和用户体验。

在实际城市道路环境中，自动驾驶车辆面临的一个核心挑战是多障碍物密集场景的处理。城市交叉路口、繁忙路段、施工区域等典型场景中，往往同时存在大量的动态障碍物，如机动车、行人、非机动车，以及静态障碍物，如施工围挡、违停车辆、临时路障等。这些障碍物数量多、分布复杂、运动状态多变，对路径规划算法提出了极高的要求[2]。具体而言，多障碍物场景下的路径规划需要解决以下几个关键问题。

第一，可行域压缩问题。当多个障碍物同时出现在自车前方时，每个障碍物都会在规划空间中占据一定区域，这些禁行区域的叠加可能导致剩余可行域被严重压缩，甚至收缩为零，使规划算法无法找到可行解。

第二，决策耦合问题。对于每个障碍物，规划系统需要做出跟随、超越、避让等决策，而这些决策之间往往存在耦合关系。例如，对障碍物 A 选择左绕可能导致与障碍物 B 发生冲突，需要全局考虑所有障碍物的决策组合才能找到最优方案。

第三，实时性问题。多障碍物场景下的规划问题复杂度随障碍物数量急剧增加，而自动驾驶系统通常要求规划模块在 100 毫秒内完成一个规划周期[3]，这对算法的计算效率提出了严格的约束。

百度 Apollo 作为全球领先的开源自动驾驶平台，提供了从感知、预测到规划、控制的完整技术栈[4]。其 Planning 模块采用了基于凸优化的路径规划方法，

通过 Scenario-Stage-Task 三级管理架构实现了场景化的规划策略，具有较强的工程实践价值和学术研究价值。然而，通过对 Apollo Planning 模块源码的深入分析，本文发现其在多障碍物密集场景下仍存在一系列亟待解决的问题，包括 PathBoundsDecider 逐障碍物独立决策导致的矛盾避让方向、动态障碍物被路径决策完全排除、以及安全裕度对所有障碍物采用统一固定值而缺乏基于威胁度的分级机制等。这些问题直接影响了系统在复杂交通环境中的通行效率和规划质量。

1.1.2 研究意义

本课题立足于百度 Apollo 这一国内事实标准的开源自动驾驶平台，针对多障碍物密集场景下的路径规划失败问题展开研究，其意义体现在工程落地、学术推进与方法论迁移三个层面。

工程意义 百度 Apollo 自 2017 年开源以来，已发展为全球装机量最大的自动驾驶开源栈之一，其 Planning 模块几乎成为国内 L4 级自动驾驶研发的事实参考实现。蘑菇车联、元戎启行、轻舟智航等量产公司的规划栈或直接 fork 自 Apollo，或在 Apollo 的 Scenario-Stage-Task 架构基础上二次开发，百度自家的萝卜快跑 Robotaxi 亦已在武汉、北京等多座城市开展无人化商业运营。这意味着针对 PathBoundsDecider 缺陷的任何切实改进，都具备直接下沉到现网车队与量产产品的可能，而不是停留在仿真实验的工具链演示。本文所有改进均以 patch 形式嵌入 Apollo 原有数据流，不引入外部求解器与额外通信开销，单个规划周期计算增量控制在毫秒级，从工程可移植性上满足下游企业的低成本集成需求。

学术意义 路径-速度解耦架构自 DARPA Urban Challenge 以来长期作为工业级规划系统的主流范式，其核心缺陷在于路径阶段的静态化假设与速度阶段的时序信息之间存在结构性断层。本文识别的矛盾避让方向、动态障碍物路径失声、统一安全裕度三类问题，本质上都是同一断层在不同算子上的投影。通过引入到达时间函数 $\tau(s)$ 将速度维度的时序预测反哺到路径边界构建阶段，本文为路径-速度解耦框架在动态场景下的信息共享给出了一个可操作的改进范式，既保留了解耦架构在求解效率与凸性上的优势，又弥补了其在多动态障碍物场景下的决策盲区，可为后续相关架构的改进提供参考思路。

方法论意义 本文构建的多因子威胁度评估、基于组合优化的全局避让方向决策、时空走廊融合三者耦合的技术路线，其核心思想不局限于 Apollo 这一具体平台。多因子威胁度将距离、相对速度、碰撞时间等异质风险信号在可解释权

重下聚合为统一标量，可直接替换其他规划系统中基于固定阈值的粗粒度危险等级；基于重叠分组的避让方向枚举将原本的 NP 形式决策压缩到线性规模，其最优性证明对任意以横向间隙为代价的路径规划器均成立；走廊融合所采用的时变边界表达可平移到基于优化的局部规划、基于采样的 Frenet 格点规划以及混合整数规划等多种技术栈。上述组合思路因此具备跨平台、跨算法范式的迁移价值，为多障碍物场景下的规划问题提供了一种超越具体实现的通用处理框架。

1.2 国内外研究现状

1.2.1 路径规划方法分类

自动驾驶路径规划技术经过数十年发展，已形成了多种成熟的算法体系，可大致分为以下四类[1-2,5-8]。四类方法的层次关系与代表算法如图1.1所示。图中以粉色高亮的优化方法分支即本文所属技术路线，Apollo 的 PiecewiseJerkPathOptimizer（PJPO）与本文改进算法均属于该分支。

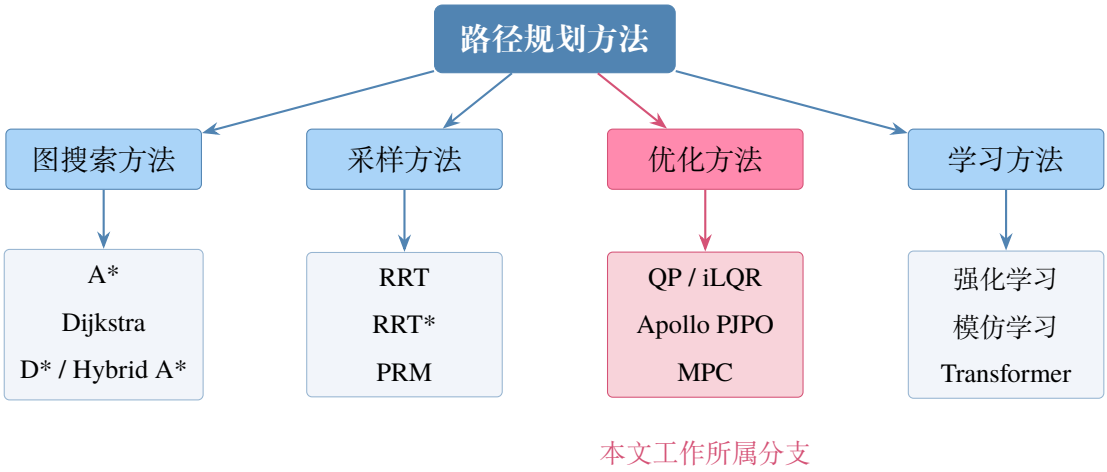


图 1.1 路径规划方法分类

如图所示，四类方法各有其适用边界。图搜索方法依赖状态空间离散化，适合低维、低速场景，其完备性保证使其成为泊车与 OpenSpace 规划的首选。采样方法通过随机采样规避维度灾难，在复杂约束下具有较强的可行性发现能力，但最优性依赖采样密度，在多障碍物密集场景下存在组合爆炸风险。优化方法将规划问题显式表达为带约束的数学规划问题，能在凸性假设下保证全局最优与实时性，是 Apollo、Waymo 等工业级 L4 平台的主流选择，但对问题的非凸性处理能力有限。学习方法近年受到广泛关注，但由于决策过程不可解释与形式化安全验证困难，目前主要作为传统方法的补充。

本文的技术栈定位明确选择优化方法分支。一方面，优化方法在 Apollo 平台已具备成熟工程基础，改进成果可直接复用求解器与数据结构；另一方面，本文识别的 PathBoundsDecider 缺陷本质上是凸化过程中丢失全局信息所致，属于优化范式内部可解决的问题，无需引入额外的采样或学习开销。

（1）**图搜索方法**

图搜索方法将连续的规划空间离散化为图结构，通过搜索算法寻找最优路径。经典的 A* 算法[9]通过引入启发式函数指导搜索方向，在保证最优性的同时显著提升了搜索效率。Dolgov et al.[10-11]提出的 Hybrid A* 算法将连续状态空间的车辆运动学约束与离散图搜索相结合，能够生成满足非完整约束的可行路径，被广泛应用于自动驾驶的泊车和低速场景。该算法在 Apollo 的 OpenSpace Planner 中得到了实际应用。图搜索方法的优点是理论基础扎实、完备性好，但在高维状态空间中面临计算复杂度快速增长的问题。

（2）**采样规划方法**

采样规划方法通过在状态空间中随机采样来构建搜索树或路线图，避免了对状态空间的显式离散化。快速探索随机树（Rapidly-exploring Random Tree, RRT）[12]及其改进版本 RRT*[13]与双向扩展的 RRT-Connect[14]是该类方法的代表，能够高效处理高维规划问题；Kuwata et al.[15]将其拓展到城市道路实时规划，在 2007 年 DARPA Urban Challenge 的 MIT 车队与 CMU 的 Boss 车队[16]中得到实战验证。Werling et al.[17]提出了基于 Frenet 坐标系的轨迹规划方法，通过在参考线的横向和纵向空间中独立采样终端状态，使用多项式曲线连接起始状态和终端状态，生成候选轨迹集合并根据代价函数进行评估筛选；Q. Li, X. Li[18]进一步从运输系统视角论证了 Frenet 框架在结构化道路下的普适性；McNaughton et al.[19]则给出了面向高速公路的时空状态格（Spatiotemporal State Lattice）扩展。这一方法论被 Apollo 的 Lattice Planner 所采用。采样方法具有灵活性强、可处理复杂约束的优点，但在多障碍物密集场景下面临采样效率低和组合爆炸的问题。

（3）**优化规划方法**

基于优化的方法将路径规划建模为数学优化问题，通过求解带约束的目标函数获得最优轨迹。二次规划（QP）方法将轨迹表示为分段多项式，以平滑度、偏离参考线程度等为目标函数，以障碍物约束、动力学约束等为约束条件，利用 OSQP[20]等高效求解器求解，非线性场景则可借助 IPOPT[21]等内点法求解器。该路线的早期代表是 Borrelli et al.[22]提出的基于模型预测控制（Model Predictive Control, MPC）的主动转向方法、Xu et al.[23]提出的面向自动驾驶的实时轨迹优

化器，以及 Ziegler et al.[24]在 Bertha Benz 自动驾驶车上首次将连续局部优化部署到公开城市道路的工作。Fan et al.[3]详细描述了 Apollo 期望最大化（Expectation-Maximization, EM）Motion Planner 的设计，采用路径 QP 和速度 QP 的两阶段优化方法，Zhou et al.[25-26]进一步给出了 Apollo 路径 QP 的分段 Jerk 形式化表达与双环迭代锚点平滑（DL-IAPS）和分段 Jerk 速度优化（PJSO）算法。Chen et al.[27]提出了迭代 EM 规划框架，进一步增强了优化方法的灵活性；Sun et al.[28]则专门针对大曲率道路给出了鲁棒路径优化器。受无人机轨迹生成中最小 Snap 方法[29]与安全飞行走廊（Safe Flight Corridor)[30]的启发，地面自动驾驶也逐步引入了类似的走廊凸分解思想。优化方法的优点是能够生成高质量的平滑轨迹，并在凸优化框架下保证全局最优性，但对问题的凸性要求较高，多障碍物场景下的非凸约束处理是主要挑战。

（4）**学习型方法**

近年来，基于深度学习和强化学习的规划方法受到了广泛关注[31]。端到端学习方法直接从传感器输入映射到控制输出，具有较强的场景泛化能力；Alahi et al.[32]提出的 Social LSTM 和基于 Transformer 的预测性运动规划[33]也为规划层提供了更可靠的先验；李升波 等[34]系统梳理了多模态协同与端到端训练的技术演进，进一步展示了该路线的发展趋势。然而，由于决策过程的不可解释性和安全性验证困难，学习型方法在工业级自动驾驶系统中的应用仍然有限，通常作为传统方法的辅助或补充[35]；Shalev-Shwartz et al.[36]提出的责任敏感安全（Responsibility-Sensitive Safety, RSS）模型从形式化角度为学习型决策提供了安全外壳。

表 1.1 四类路径规划方法对比

维度	图搜索	采样	优化	学习
完备性	分辨率完备	概率完备	局部最优	无形式保证
最优性	离散最优	渐近最优	凸下全局最优	数据依赖
实时性	随维度恶化	高	高（10–100 ms）	推理快、训练贵
非凸处理	强	强	弱（需凸化）	自然支持
工程成熟度	已工业化	已工业化	L4 主流	仍在研
Apollo 对应	OpenSpace	Lattice Planner	PublicRoad (PJPO)	辅助模块

表1.1揭示了本文技术选型的底层逻辑。优化方法在实时性与工程成熟度上取得了最好的平衡，是 Apollo PublicRoad Planner 的核心范式；但其非凸处理弱的短板恰恰在多障碍物密集场景下被放大，PathBoundsDecider 的逐障碍物贪心决策正是典型表现。本文的改进抓手落在优化范式内部，通过时变最优通行带的组合优化将凸化前的全局信息注回，而无需跳出优化框架。

1.2.2 Apollo Planning 模块技术演化脉络

百度 Apollo 自 2017 年开源以来，其 Planning 模块经历了从 EM Planner、PublicRoad Planner 到 OpenSpace Planner 的三阶段演化，理解这一脉络是识别 PathBoundsDecider 定位的前提。

EM Planner 时期，2017 至 2019 年 该阶段采用期望最大化思想在 Frenet 坐标系下交替求解路径与速度子问题[3]。路径阶段以动态规划搜索粗解、以 QP 平滑精解，速度阶段在 ST 图上重复同一套 DP 粗解加 QP 精解的模式。EM Planner 首次将路径-速度解耦的两阶段优化完整落地到工业级 L4 栈，成为后续所有 Apollo Planner 的范式基础。

PublicRoad Planner 时期，2019 年至今 该阶段引入 Scenario-Stage-Task 三级管理架构，对 LaneFollow、Lane Change、Park and Go 等场景进行显式建模。路径层由 Zhou et al.[25]的分段 Jerk 路径优化器（PiecewiseJerkPathOptimizer, PJPO）接管，速度层对应为 PJSO[26]。其中 PathBoundsDecider 作为 PJPO 的前置 Task，负责将障碍物决策转化为路径 QP 的左右边界约束，直接决定了优化空间的形状，

也是本文改进的核心切入点。

OpenSpace Planner 补充时期，2020 年至今 为支持泊车、OpenSpace 等结构化先验失效的场景，Apollo 引入 Hybrid A[10]与 DL-IAPS[26]的自由空间规划链路，与 PublicRoad Planner 并行。

基于该演化脉络可以明确本文定位。本文不触碰 EM 与 PJPO 的优化器本身，而是在 PathBoundsDecider 这一路径边界构建环节把全局决策能力注入进去，属于在 PublicRoad 主干内部做精细化升级，改动颗粒度小，可与上下游解耦回归。

1.2.3 Apollo 平台相关研究

除上述平台自研工作外，Apollo 还吸引了大量的外部学术研究和工程实践。

在算法分析方面，Kessler et al.[37]在 Apollo 规划栈中实现了混合整数运动规划，用于处理德国道路上的复杂驾驶场景。H. Li, De Cristofaro et al.[38]对 Apollo 规划参数化对关键场景中轨迹规划的影响进行了定量分析，揭示了参数选择对规划质量的显著影响。H. Li, Makkapati et al.[39]基于 Apollo 平台验证了自动驾驶功能，展示了仿真与实车测试的结合方法。

在国内研究方面，高峰和吴昊[40]探讨了基于 Apollo 平台的自动驾驶教学创新方法。孙健和周晓[41]研究了面向城市工况的运动规划方法。马晓东和刘建华[42]将智能优化算法应用于 Apollo 框架下的泊车路径规划。在仿真验证方面，CARLA[43]、LGSVL[44]等开放仿真器以及 nuScenes[45]等大规模数据集为 Apollo 类平台提供了可复现的评估基座。这些研究为 Apollo 平台的实际应用提供了丰富的参考。

1.2.4 静态多障碍物场景规划研究

当多个静态障碍物同时出现在自车前方时，其核心挑战是将多约束非凸可行域转化为可解的数学结构。

走廊凸分解 王鹏 等[46]提出了基于时空走廊的非结构化环境轨迹规划方法，通过构建安全通道来简化多障碍物约束处理；杨秀建 等[47]在三维时空走廊内以分段贝塞尔曲线求解最优轨迹，在安全性与灵活性上相较时间弹性带算法取得了明显提升；乔应[48]则在结构化道路下引入时间-纵向-横向三维时空决策并与时空安全走廊的二次规划相结合；徐涛[49]在城市交互场景中将部分可观测马尔可夫决策过程（Partially Observable Markov Decision Process, POMDP）行为决策与基于约束迭代线性二次调节器（Constrained Iterative Linear Quadratic Regulator,

CILQR）的时空安全走廊求解进行耦合。上述工作的共同点是将多障碍物约束统一抽象为走廊边界，进而将非凸问题转化为凸问题。

多障碍物交互建模 陈志远和黄海波[50]研究了复杂道路环境下智能车辆的横纵向避障问题，提出了考虑障碍物交互关系的综合避障策略；Y. Guo et al.[51]对自动驾驶场景和场景数据库进行了全面综述，为多障碍物场景的构建和评估提供了参考框架。

然而上述走廊方法普遍预设走廊已经给定，并未回答多障碍物之间应如何选择左绕或右绕从而最大化走廊宽度这一前置问题。Apollo 的 PathBoundsDecider 正是因为在这一前置步骤采用逐障碍物独立决策的贪心策略，才导致后续走廊被压缩甚至阻塞，这构成本文的直接立题。

1.2.5 动态障碍物的时间维度处理

相比静态场景，动态障碍物引入了时间维度，其处理方式演进呈现出清晰的脉络。

路径-速度解耦阶段 Kant, Zucker[52]于 1986 年提出的路径-速度解耦（Path-Velocity Decomposition, PVD）是整个领域的理论起点，先在笛卡尔空间规划路径绕开静态障碍物，再在路径-时间（Path-Time, PT）空间上规划速度规避动态障碍物。这一范式的优点是将高维时空规划降维到两个二维子问题，在 Apollo EM Planner[3]和 PublicRoad Planner 中均有体现，ST 图本质上就是 Kant-Zucker PT 空间的直接继承。

ST 图与局部时间优化阶段 在 PVD 基础上，ST 图被广泛用于速度规划。C. Liu, Tomizuka[53]进一步提出基于时间优化的速度剖面规划方法，将时间维度显式纳入优化变量；van den Berg et al.[54]提出的互惠速度障碍（Reciprocal Velocity Obstacle, RVO）为多智能体动态避让提供了速度层的几何刻画；Ziegler et al.[24]在 Bertha 项目中将动态障碍物作为时变约束集成进局部连续优化器。

时间弹性带与时空联合阶段 Rösmann et al.[55-56]提出的时间弹性带（Timed Elastic Band, TEB）将时间显式作为优化变量，在局部轨迹层面整合时间维度，是 PVD 的第一次重大突破。近年的研究进一步从速度规划层向上反哺路径规划，杨秀建 等[47]、乔应[48]的时空走廊工作已隐含路径-速度耦合的思想，李浩淼[57]则在此基础上引入了障碍物扰动鲁棒性。

本文在脉络中的定位 尽管上述工作推进了动态障碍物处理的时间维度，但无一例外地仍将路径决策阶段的动态障碍物视为当前时刻的静态快照，Apollo

PathBoundsDecider 即是该假设的典型产物。本文的贡献在于把到达时间函数引入路径边界构建阶段，将动态障碍物未来时刻将离开当前位置的预测信息直接注入路径决策，实现路径层对时间维度的显式建模，而非仅依赖速度层事后补偿。

1.2.6 现有研究不足

综合分析现有研究，可以发现以下不足。

第一，针对 Apollo Planning 模块路径边界决策在多障碍物场景下的缺陷缺乏系统性分析。现有研究多关注速度规划层面的参数调优或单一模块的改进，而对路径边界构建阶段（PathBoundsDecider）逐障碍物独立决策导致的矛盾避让方向问题缺乏深入研究。

第二，多障碍物的避让方向选择缺乏全局最优性保证。Apollo 对每个障碍物独立选择左绕或右绕的贪心策略，在多障碍物场景下可能导致相邻障碍物被分配矛盾的避让方向，进而导致路径不可行。该问题的本质是一个组合优化问题，现有研究尚未从数学优化的角度给出系统性的解决方案。

第三，路径决策未考虑动态障碍物的时间演化。Apollo 的路径边界构建仅处理静态障碍物，动态障碍物被完全交由速度规划阶段处理。这种割裂使得路径决策无法利用“动态障碍物在未来时刻将离开当前位置”的预测信息来扩展可行路径空间。此外，李浩淼[57]指出现有时空联合规划算法普遍假设障碍物位置精确已知，对感知扰动缺乏鲁棒应对机制，这一结论同样适用于 Apollo 当前的路径边界决策。

本文将针对上述不足，提出系统性的改进方案。

1.3 研究内容与技术路线

1.3.1 研究内容

本文的研究内容主要包括以下五个方面。

（1）**Apollo Planning 模块深度分析** 对 Apollo 11.0 版本的 Planning 模块进行源码级深入分析，理解其 Scenario-Stage-Task 三级管理架构、ReferenceLine 平滑机制、PublicRoadPlanner 中源自早期 EM Planner 思想的路径-速度解耦两阶段优化流程、PathBoundsDecider 路径边界构建和 SpeedBoundsDecider 的 ST 边界构建机制。

（2）**多障碍物场景问题识别** 通过源码分析和数学建模，系统识别 Apollo

现有算法在多障碍物场景下的三个核心问题，即安全裕度一刀切压缩通行空间、PathBoundsDecider 逐障碍物独立决策导致矛盾避让方向、以及动态障碍物被路径决策完全排除导致路径-速度协调缺失。三个问题分别从安全距离、决策粒度和时空协调三个维度指向改进方向。

（3）**时变最优通行带问题建模** 将多障碍物场景下的避让方向选择建模为组合优化问题，引入到达时间函数将动态障碍物的预测轨迹纳入路径决策，建立统一的时变最优通行带数学框架。将时变通行带投影至 ST 空间解读为时空可通行走廊，建立路径决策与速度规划之间的约束传递机制。

（4）**基于重叠分组的最优求解算法设计与实现** 通过引理证明最优分割的连续性、通过定理证明最大间隙策略的最优性，提出基于扫描线的 $O(n \log n)$ 时变障碍物分组算法，在 Apollo 平台上实现并集成到 PathBoundsDecider 中。

（5）**仿真验证与对比分析** 在 Apollo 仿真环境中设计多组多障碍物测试场景，重点验证”Baseline 路径阻塞而改进方案可通行”的核心对比，从规划成功率、通行效率、停车次数等维度进行定量分析。

1.3.2 技术路线

本文的技术路线如图1.2所示，分为七个阶段依次推进。

图 1.2 技术路线图

1.4 论文组织结构

本文共分为七章，各章内容安排如下。

第一章为绪论，介绍研究背景与意义、国内外研究现状、研究内容与技术路线。

第二章为相关技术与理论基础，介绍 Frenet 坐标系、凸优化与二次规划、SL 图与路径边界构建、Apollo 平台架构等基础知识。

第三章为 Apollo Planning 模块深度分析，对 Planning 模块的总体架构、ReferenceLine 平滑、路径-速度解耦两阶段优化流程以及 PathBoundsDecider 路径边界决策逻辑进行源码级分析。

第四章为多障碍物场景下的问题分析，从安全距离、决策粒度和时空协调三个维度系统剖析 Apollo PathBoundsDecider 的规划缺陷，即安全裕度一刀切压缩通行空间、逐障碍物独立决策导致矛盾避让方向、以及动态障碍物被完全排除导致路径-速度协调缺失。

第五章为改进算法设计与实现，建立时变最优通行带的数学模型，从 SL 层提出多因子威胁度评估与差异化安全距离、基于重叠分组的最大间隙策略并证明其最优性，从 ST 层引入到达时间函数和预测轨迹将动态障碍物纳入路径决策并构建时空可通行走廊与速度规划协调机制，最后给出与 Apollo 系统的集成方案。

第六章为实验与结果分析，介绍实验环境、场景设计、评价指标，展示对比实验结果并进行分析。

第七章为总结与展望，总结全文工作和贡献，讨论研究局限性和未来方向。

2 相关技术与理论基础

本章介绍论文涉及的核心技术与理论基础，包括 Frenet 坐标系、凸优化与二次规划、SL 图与路径边界构建以及 Apollo 平台的整体架构，为后续章节的源码分析和算法改进奠定理论基础。

2.1 Frenet 坐标系

2.1.1 坐标系定义

在自动驾驶路径规划中，直接在笛卡尔坐标系下进行规划需要同时处理道路几何形状和车辆运动约束，问题较为复杂。Frenet 坐标系[17]以道路参考线为基准建立曲线坐标系，将复杂的二维规划问题分解为相互独立的纵向和横向两个一维问题，极大地简化了规划计算。

Frenet 坐标系以参考线上距起点的累积弧长 s 作为纵向坐标，以垂直于参考线方向的偏移量 l（或 d）作为横向坐标。对于参考线上的任意一点 $P_{ref}(s)$，其位置、航向角 $\theta_{ref}(s)$ 和曲率 $\kappa_{ref}(s)$ 均可通过参考线的几何参数计算得到。

图2.1展示了 Frenet 坐标系的基本定义。

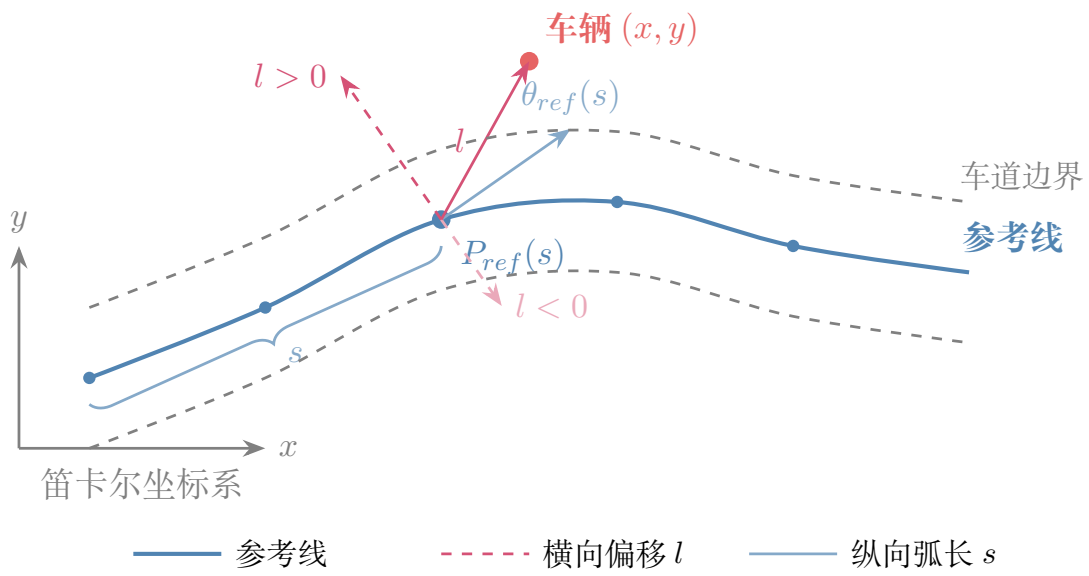


图 2.1 Frenet 坐标系示意图

设笛卡尔坐标系下的点为 (x, y)，其在 Frenet 坐标系下的表示为 (s, l)。坐标

转换关系为

$$\begin{cases} x = x_{ref}(s) - l \cdot \sin(\theta_{ref}(s)) \\ y = y_{ref}(s) + l \cdot \cos(\theta_{ref}(s)) \end{cases} \tag{2.1}$$

其中 $(x_{ref}(s), y_{ref}(s))$ 为参考线上弧长 s 处的点坐标，$\theta_{ref}(s)$ 为该处的航向角。

2.1.2 运动状态表示

在 Frenet 坐标系下，车辆的运动状态可以用纵向状态和横向状态分别描述。

纵向状态向量

$$\mathbf{s} = [s, \dot{s}, \ddot{s}]^T \tag{2.2}$$

其中 s 为纵向位置，$\dot{s}$ 为纵向速度，$\ddot{s}$ 为纵向加速度。

横向状态向量

$$\mathbf{l} = [l, l', l'']^T \tag{2.3}$$

其中 l 为横向偏移，$l' = dl/ds$ 为横向偏移对纵向位置的导数，$l'' = d^2l/ds^2$ 为二阶导数。

横向偏移的时间导数与空间导数之间的关系为

$$\dot{l} = l' \cdot \dot{s}, \quad \ddot{l} = l'' \cdot \dot{s}^2 + l' \cdot \ddot{s} \tag{2.4}$$

这种解耦表示使得路径规划可以分为两个独立的子问题，在纵向上规划速度曲线 $s(t)$，在横向上规划路径曲线 $l(s)$。

2.1.3 曲率计算

在 Frenet 坐标系下，车辆实际行驶轨迹的曲率 κ 与参考线曲率 κ_{ref}、横向偏移 l 及其导数之间的关系为

$$\kappa = \frac{(l'' + (\kappa'_{ref} l + \kappa_{ref} l') \tan(\Delta\theta)) \cos^2(\Delta\theta)}{(1 - \kappa_{ref} l)} + \kappa_{ref} \tag{2.5}$$

其中 $\Delta\theta = \theta - \theta_{ref}$ 为车辆航向与参考线航向的偏差角。该曲率表达式可由微分几何中曲线的曲率定义出发，利用链式法则推导得到[17]。当横向偏移较小时 $l \approx 0$，上式可近似为

$$\kappa \approx \kappa_{ref} + l'' \tag{2.6}$$

Apollo 对曲率的关注贯穿整条规划流水线。一方面，曲率直接决定车辆的最小转弯半径 $R = 1/|\kappa|$，受阿克曼转向几何限制，$|\kappa|$ 不得超过由最大前轮转角与轴距确定的上界 $\kappa_{\max}$。另一方面，曲率与车速联合决定侧向加速度 $a_{lat} = v^2\kappa$，过大的侧向加速度将超过轮胎附着极限或乘员舒适阈值。因此路径 QP 必须将 $|\kappa| \le \kappa_{\max}$ 作为硬约束纳入可行域。

然而式(2.5)是关于 $l, l', l'', \Delta\theta$ 的强非线性函数，直接作为约束会破坏二次规划的凸性。Apollo 的 PiecewiseJerkPathOptimizer 正是借助式(2.6)的小偏移近似，将曲率约束线性化为对横向二阶导的盒约束

$$|l''| \le l''_{\max} \triangleq \kappa_{\max} - \max_{s} |\kappa_{ref}(s)| \tag{2.7}$$

该近似的合理性建立在两个工程前提之上，其一是参考线经过平滑处理后自身曲率 κ_{ref} 已足够小，其二是横向偏移 l 远小于转弯半径，使得分母 $(1 - \kappa_{ref} l)$ 可取为一。由此路径优化退化为标准 QP，线性约束与二次代价共同保证求解器可在实时周期内收敛。

本节给出的曲率解析式与线性化近似是后续章节的直接工具。第三章分析 Apollo 路径规划源码时将看到 $l''_{\max}$ 如何从配置参数传递至 OSQP 求解器的上下界向量；第五章构造时变最优通行带时亦沿用该近似，将通行带的横向收缩映射为对 l 及其导数的联合约束，从而在保持凸性的前提下纳入动态避让需求。

2.2 凸优化与二次规划

2.2.1 凸优化基本概念

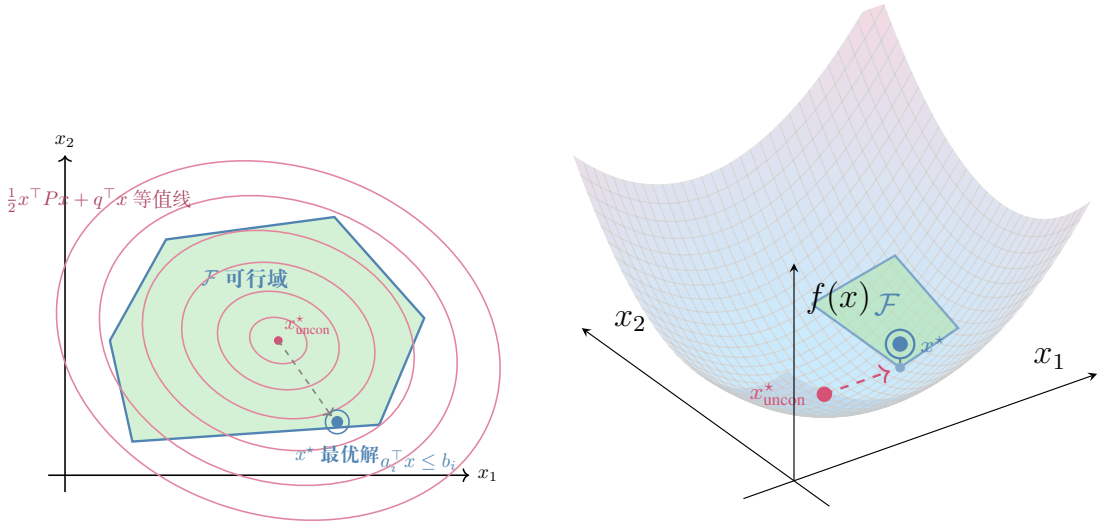
凸优化是一类特殊的数学优化问题，要求目标函数为凸函数，可行域为凸集。凸优化问题的一般形式为

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) \le 0, \quad i = 1, \ldots, m \\ & A\mathbf{x} = \mathbf{b} \end{aligned} \tag{2.8}$$

其中 $f(\mathbf{x})$ 为凸目标函数，$g_i(\mathbf{x})$ 为凸不等式约束函数。

凸优化的核心性质是，任何局部最优解都是全局最优解。这一性质保证了求解器能够找到全局最优解，是自动驾驶规划中广泛使用凸优化方法的理论基础。

凸二次规划问题的几何含义如图2.2所示。目标函数 $\frac{1}{2}x^\top Px + q^\top x$ 在正定矩阵 $P \succ 0$ 下构成同心椭圆族的等值面，每条椭圆代表目标函数的一个水平集。线性不等式约束 $a_i^\top x \le b_i$ 各定义一个半平面，所有约束的交集即可行域 $\mathcal{F}$，为一个凸多边形。在凸性保证下，目标函数在 $\mathcal{F}$ 内部必定存在唯一全局最优解 $x^\star$。若无约束最小点 $x^\star_{\text{uncon}}$ 本身位于可行域内，则该点即为最优解；否则最优解必位于可行域边界上，并且其水平集椭圆与边界相切（KKT 互补松弛条件）。这一几何图景直观解释了为何凸 QP 既能保证最优性又具有高效的数值求解特性。



(a) 2D 等值线视角：KKT 切线条件 **(b) 3D 曲面视角：抛物面碗型直觉**

图 2.2 凸二次规划的几何解释

左：正定二次型等值线为同心椭圆，线性约束构成凸多边形可行域 $\mathcal{F}$，最优解 $x^\star$ 处椭圆与约束边界相切（KKT 互补松弛）。
右：目标函数在 (x_1, x_2, f) 空间是开口向上的抛物面，碗底为无约束最优点 $x^\star_{\text{uncon}}$；当该点落在 $\mathcal{F}$ 之外时，最优解 $x^\star$ 落在 $\mathcal{F}$ 内最接近碗底、曲面高度最低的位置。

2.2.2 二次规划

二次规划（Quadratic Programming, QP）是凸优化的一个重要子类，其目标函数为二次形式，约束条件为线性。标准形式为

$$
\begin{aligned}
\min_{\mathbf{x}} \quad & \frac{1}{2}\mathbf{x}^T H \mathbf{x} + \mathbf{f}^T \mathbf{x} \\
\text{s.t.} \quad & A_{eq}\mathbf{x} = \mathbf{b}_{eq} \\
& A_{ineq}\mathbf{x} \le \mathbf{b}_{ineq} \\
& \mathbf{x}_{lb} \le \mathbf{x} \le \mathbf{x}_{ub}
\end{aligned}
\tag{2.9}
$$

其中 H 为半正定矩阵（$H \succeq 0$），$\mathbf{f}$ 为线性项系数，A_{eq} 和 A_{ineq} 分别为等式和不等式约束矩阵。

Apollo Planning 模块使用 OSQP（Operator Splitting Quadratic Program）[20]作为 QP 求解器。OSQP 基于交替方向乘子法（Alternating Direction Method of Multipliers, ADMM），具有以下优点。

（1）支持稀疏矩阵运算，适合大规模问题；

（2）可处理不可行问题，返回近似解；

（3）无需活跃集策略，对初始点不敏感；

（4）计算复杂度为 $O(n^2)$ 至 $O(n^3)$，满足实时性要求。

2.2.3 路径规划中的 QP 建模

在 Apollo 的路径规划中，轨迹被表示为关于参数 t 的分段多项式。以五次多项式为例，第 i 段样条的形式为

$$f_i(t) = a_{i,0} + a_{i,1}t + a_{i,2}t^2 + a_{i,3}t^3 + a_{i,4}t^4 + a_{i,5}t^5 \tag{2.10}$$

规划问题的目标函数通常包含以下代价项

$$J = \underbrace{w_1 \int (l'')^2 ds}_{\text{曲率平滑}} + \underbrace{w_2 \int (l''')^2 ds}_{\text{曲率变化率}} + \underbrace{w_3 \sum_j \|l(s_j) - l_{ref,j}\|^2}_{\text{参考线跟踪}} \tag{2.11}$$

其中 w_1, w_2, w_3 为各项权重，l'' 和 l''' 分别为横向偏移的二阶和三阶导数，$l_{ref,j}$ 为参考线对应的横向偏移。约束条件包括初始状态约束、样条节点连续性约束（C^2 或 C^3 连续）、路径边界约束等。

需要指出的是，上述五次多项式表示是 Apollo 早期版本中使用的路径参数化方式。在当前 Apollo 版本中，主流的 PiecewiseJerkPathOptimizer 采用离散化表示方法，直接以纵向采样点的横向偏移 l_i 作为优化变量，利用有限差分近似一阶和二阶导数，从而将路径优化转化为标准二次规划问题。本文后续章节的分析基于该离散化实现方式。

2.3 SL 图与路径边界

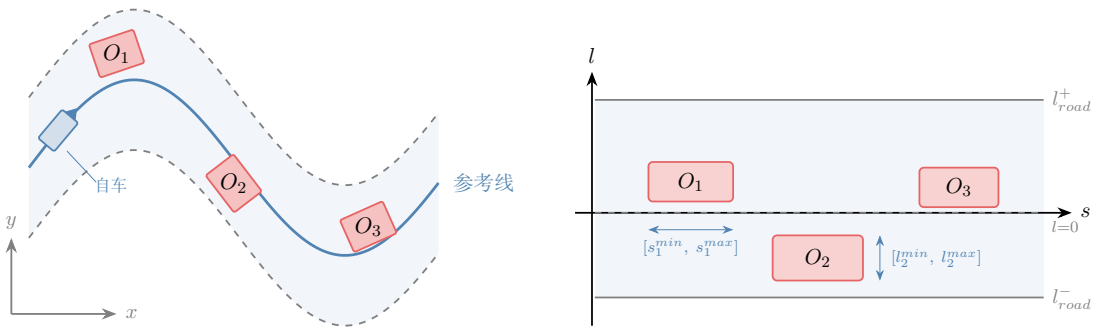
在 Frenet 坐标系下，路径规划的核心任务是确定自车在每个纵向位置 s 处的横向偏移 $l(s)$。为此，需要在 SL（Station-Lateral）空间，即纵向位置 s 与横向偏移 l 构成的二维平面中，构建路径的可通行边界。

2.3.1 障碍物的 SL 投影

每个障碍物在 SL 空间中占据一个区域。设第 i 个障碍物的 SL 投影为矩形

$$O_i = [s_i^{min}, s_i^{max}] \times [l_i^{min}, l_i^{max}] \tag{2.12}$$

其中 $[s_i^{min}, s_i^{max}]$ 为障碍物沿参考线的纵向投影范围，$[l_i^{min}, l_i^{max}]$ 为其横向占据范围。图2.3展示了障碍物从笛卡尔空间到 SL 空间的投影过程。



(a) 笛卡尔空间中的 S 弯道场景 **(b) 对应的 SL 空间投影**

图 2.3 SL 空间中障碍物投影示意图

弯道上各障碍物沿参考线投影后，在 SL 空间中均表示为轴对齐矩形 $[s_i^{min}, s_i^{max}] \times [l_i^{min}, l_i^{max}]$。

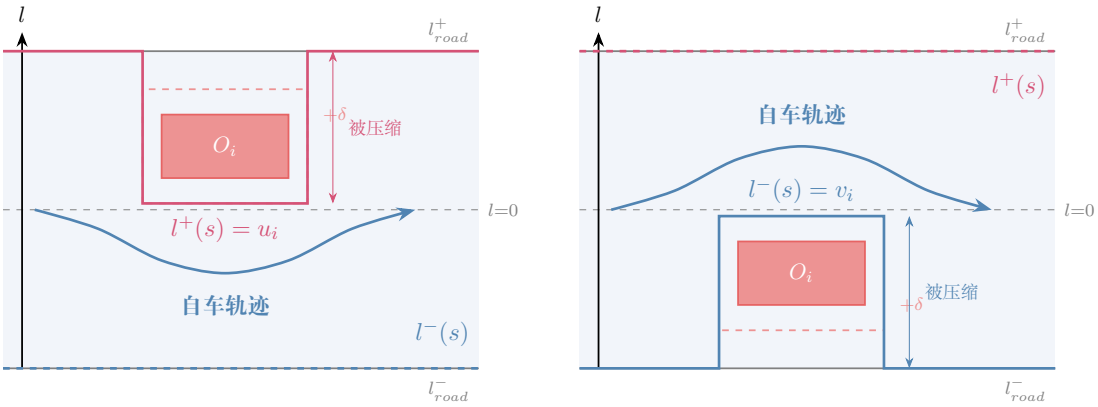
2.3.2 路径边界构建

路径规划需要在 SL 空间中找到一条横向轨迹 $l(s)$，使其不与任何障碍物碰撞。这要求在每个 s 处确定横向可通行区间 $[l^-(s), l^+(s)]$，其中 $l^+(s)$ 为上界即左侧限制，$l^-(s)$ 为下界即右侧限制。

对于每个在位置 s 处活跃的障碍物 O_i，即满足 $s \in [s_i^{min}, s_i^{max}]$ 的障碍物，规划系统需要做出避让方向决策 $d_i \in \{L, R\}$，若选择右绕（$d_i = R$），则障碍物约束路径上界 $l^+(s) \le l_i^{min} - \delta$；若选择左绕（$d_i = L$），则约束下界 $l^-(s) \ge l_i^{max} + \delta$。其中 δ 为安全裕度。

图2.4直观展示了两种避让方向决策对路径边界的影响。图 (a) 为右绕情形，

障碍物位于车道中线上方，决策 $d_i = R$ 使得原本位于车道左侧边界的上界被压低至 $u_i = l_i^{min} - \delta$，自车须在该上界之下寻找路径。图 (b) 为左绕情形，障碍物位于车道中线下方，决策 $d_i = L$ 使得原本位于车道右侧边界的下界被抬高至 $v_i = l_i^{max} + \delta$。可见，任一次避让决策都是对路径可行空间的单边收紧，是后续章节理解“独立决策导致矛盾避让”问题的直接几何基础。



(a) RIGHT_NUDGE **(b) LEFT_NUDGE**

图 2.4 避让方向决策对路径边界的影响

当多个障碍物同时活跃时，所有约束取交集。路径可行的条件是 $l^+(s) \geq l^-(s) + W_{ego}$，其中 W_{ego} 为自车宽度。若该条件不满足，路径被判定为阻塞（BLOCKED）。

避让方向的选择本质上是一个组合优化问题，n 个障碍物各有两种选择，共 2^n 种组合。如何高效地找到使通行带最宽的决策组合，是本文第五章的核心研究内容。

2.4 ST 图与速度规划基础

SL 图负责描述路径的几何可行性，而 ST（Station-Time）图则描述路径已定情况下的速度可行性。ST 图的横轴为时间 t，纵轴为沿参考线的累积弧长 s，自车运动被描绘为一条在 s 随 t 单调递增的曲线，斜率即瞬时速度 $\dot{s}$。

2.4.1 ST 图中的障碍物与速度约束

障碍物在 ST 图中的表示取决于其运动状态。静态障碍物在参考线上的投影长度固定，在 ST 图中表现为一个沿时间轴方向延伸的平行带状矩形区域。动态障碍物的位置随时间改变，在 ST 图中通常表现为一个倾斜的平行四边形，其斜

率由障碍物沿参考线方向的速度决定。

图2.5展示了 ST 图的基本原理以及两种典型的速度决策方式。当障碍物占据了 ST 平面中的禁行区域，即图中红色与橙色矩形或平行四边形，自车轨迹必须绕过这些区域。若轨迹位于障碍物的下方，即 s 值较小的一侧，则称为避让（YIELD）决策，即自车减速让行；若轨迹位于障碍物的上方，即 s 值较大的一侧，则称为超越（OVERTAKE）决策，即自车加速通过。两种决策对应完全不同的速度曲线，且各自具有不同的可行条件和安全边际。

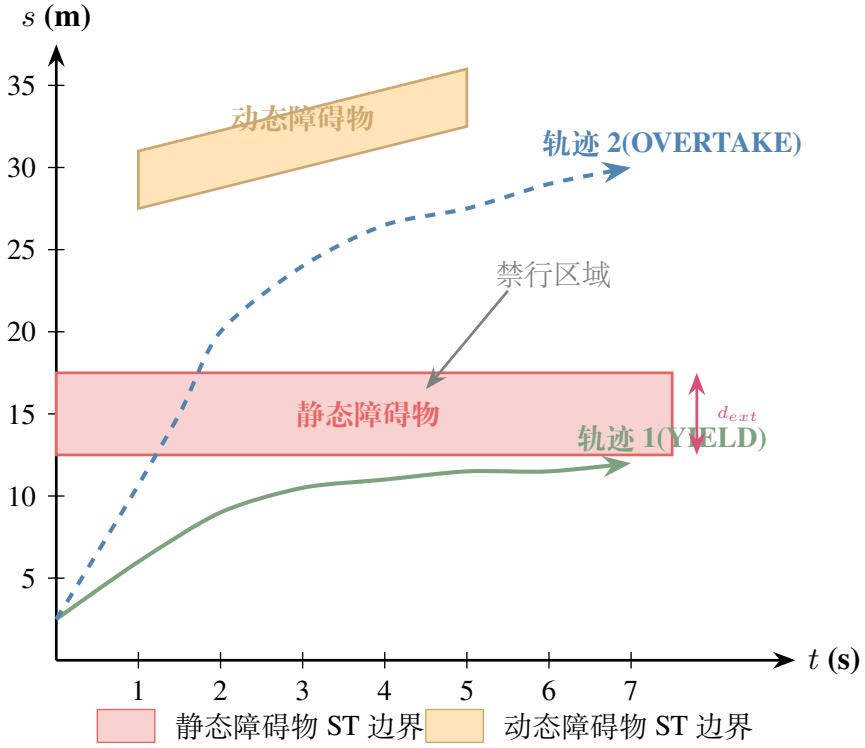


图 2.5 ST 图原理与 YIELD/OVERTAKE 决策

在此基础上，为保证安全，Apollo 对每个障碍物的 ST 投影额外叠加纵向安全距离 d_s，使得禁行区沿 s 方向向两侧扩张，得到实际用于约束优化的 ST 边界。多障碍物情形下所有 ST 边界取并集，其在每个时刻 t 对应的可行区间的上下界分别构成速度的上下约束曲线 $s_{\max}(t)$ 和 $s_{\min}(t)$。两条曲线所围成的区域即为 ST 走廊，速度优化器在该走廊内寻找平滑、舒适、最快的 $s(t)$ 曲线，构成路径-速度解耦两阶段优化的第二阶段。

2.5 Apollo 平台架构概述

2.5.1 系统总体架构

百度 Apollo 自动驾驶平台[4]采用模块化设计，主要包含感知、预测、路由、规划和控制五大核心模块。各模块之间通过 Cyber RT 实时通信框架进行消息传递，采用发布-订阅模式实现松耦合的模块间协作。图2.6展示了 Apollo 平台的总体架构。

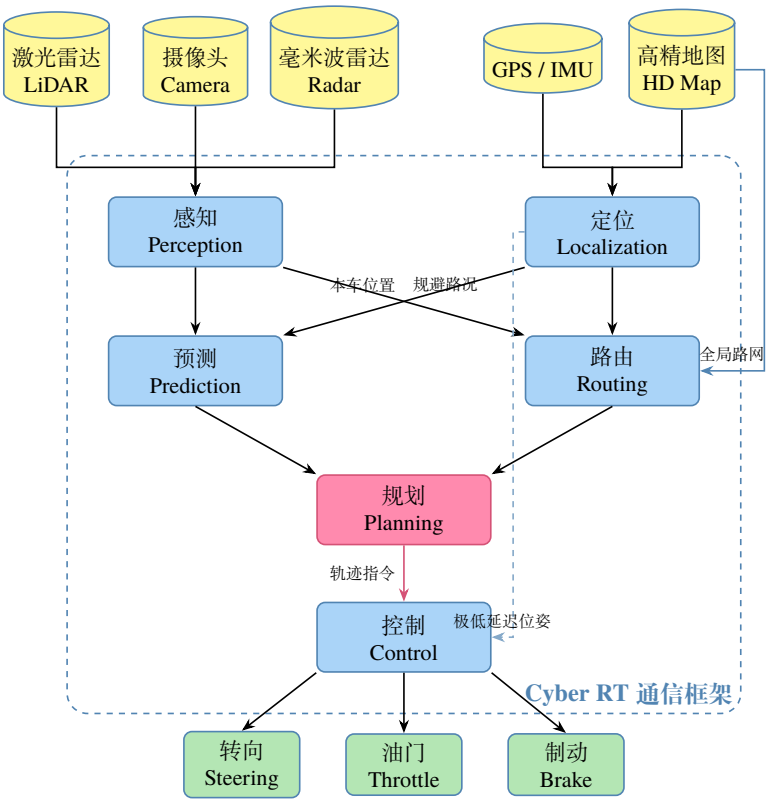


图 2.6 Apollo 自动驾驶平台总体架构

2.5.2 Cyber RT 通信框架

Cyber RT 是 Apollo 的实时通信框架，替代了早期版本中的 ROS（Robot Operating System）。相比 ROS，Cyber RT 具有以下优势。

（1）更低的消息传递延迟；

（2）确定性的调度机制；

（3）更好的多线程支持；

（4）内置的录制和回放功能。

Planning 模块通过 Cyber RT 订阅以下消息

- `PredictionObstacles`，预测模块输出的障碍物预测轨迹
- `Chassis`，底盘状态（档位、速度、转向）
- `LocalizationEstimate`，定位信息（位置、速度、加速度）
- `TrafficLightDetection`，交通灯检测结果
- `PlanningCommand`，规划命令（包含路由信息）

输出消息为 `ADCTrajectory`，包含完整的轨迹点序列及其对应的时间戳、速度、加速度、曲率等信息。

Apollo 的路径生成分为宏观与微观两个层次，Planning 模块依赖 Cyber RT 订阅 Routing 模块输出的全局路由，并在此基础上完成局部轨迹生成。Routing 负责宏观路径规划，即“走哪条路”，其基于高精地图的拓扑图以 A* 算法搜索从起点到终点的全局最优路线，输出道路段序列，仅在目的地变化或需要重路由时触发，属于低频更新。Planning 负责微观轨迹规划，即“怎么走”，其在 Routing 提供的参考线基础上结合实时感知与预测信息，以约 100ms 的周期持续生成局部最优轨迹，是本文研究的重点。

2.5.3 Planning 模块的 Scenario-Stage-Task 架构

Apollo Planning 模块采用三级管理架构来实现场景化规划。

场景（Scenario）层 ScenarioManager 根据当前环境状态，如是否有交通灯、停车标志、接近目的地等，选择合适的场景。Apollo 支持的场景包括 LANE_FOLLOW、TRAFFIC_LIGHT_PROTECTED、STOP_SIGN_UNPROTECTED、PULL_OVER 等共 19 种场景类型，分别对应车道跟随、有保护交通灯、无保护停车标志、靠边停车等驾驶情境。

阶段（Stage）层 每个 Scenario 包含一个或多个 Stage，表示完成该场景需要经历的不同阶段。例如，停车标志场景可能包含“接近停车线”、“停车等待”、“通过路口”等阶段。

任务（Task）层 每个 Stage 包含一个有序的 Task 列表，Task 是最小的规划执行单元。常见的 Task 包括 PathBoundsDecider（路径边界决策）、PathDecider（路径决策）、SpeedBoundsDecider（速度边界决策）、PiecewiseJerkSpeedOptimizer（速度优化）等。Task 按顺序在参考线上依次执行。

这种三级架构使得规划逻辑模块化、可配置，不同场景可以灵活组合不同的 Task 流水线。

2.6 组合优化与区间图基础

本文第五章将多障碍物避让方向选择建模为组合优化问题，并利用区间图的结构性质将指数级搜索空间降至多项式级别。本节介绍相关的数学基础。

2.6.1 组合优化概述

组合优化是在有限或可数无限的离散可行解集合中搜索最优解的数学分支。其一般形式为

$$\mathbf{x}^* = \arg\max_{\mathbf{x}\in\mathcal{X}} f(\mathbf{x}) \tag{2.13}$$

其中 $\mathcal{X}$ 为有限可行集，f 为目标函数。当 $\mathcal{X}$ 由 n 个二值决策变量构成时，可行集的大小为 2^n，暴力枚举在 n 较大时不可行。

组合优化问题按计算复杂度分为 P 类和 NP（Non-deterministic Polynomial）-hard 类。P 类问题存在多项式时间算法，NP-hard 问题在最坏情况下不存在已知的多项式时间精确算法。然而，许多 NP-hard 问题在特定的问题结构下可以利用结构性质显著降低求解复杂度。例如，当决策变量之间存在特定的排序或独立性关系时，搜索空间可从指数级降为多项式级。本文第五章证明的引理 1 正是利用了障碍物在横向排序后最优分割具有连续性这一结构性质，将 2^k 种决策组合降为 $k+1$ 个候选分割点的线性枚举。

2.6.2 区间图与连通分量

在图论中，区间图（Interval Graph）是一类与实数轴上的区间集合对应的无向图。给定 n 个闭区间 $I_1 = [a_1, b_1], I_2 = [a_2, b_2], \ldots, I_n = [a_n, b_n]$，以每个区间为一个顶点，两个顶点之间存在边当且仅当对应区间有交集

$$(I_i, I_j) \in E \iff [a_i, b_i] \cap [a_j, b_j] \neq \varnothing \tag{2.14}$$

区间图的连通分量可通过扫描线算法在 $O(n \log n)$ 时间内求解。将所有区间按左端点 a_i 升序排列，维护当前连通分量中所有区间右端点的最大值 b_{max}。依次扫描排序后的区间，若下一个区间的左端点 $a_j \leq b_{max}$，则将其并入当前连通分量并更新 $b_{max} \leftarrow \max(b_{max}, b_j)$，否则开始新的连通分量。排序需 $O(n \log n)$，扫描仅需 $O(n)$。

在本文第五章中，每个障碍物的纵向占据区间 $[s_i^-, s_i^+]$ 构成一个区间集合，

其区间图的连通分量恰好对应在纵向上彼此关联的障碍物组。不同连通分量的避让决策互不影响，可以独立求解。这一分组机制是本文算法将全局问题分解为多个独立子问题的理论基础。

2.6.3 碰撞时间

碰撞时间（Time-to-Collision, TTC）是交通安全领域衡量碰撞紧迫性的经典指标[1]，定义为在当前运动状态下两个物体发生碰撞所需的时间

$$TTC = \frac{\Delta s}{v_{rel}} \tag{2.15}$$

其中 Δs 为两物体之间的纵向距离，v_{rel} 为相对接近速度。当 $v_{rel} \leq 0$ 时两物体不在相互接近，$TTC = +\infty$。TTC 值越小，碰撞风险越高。

工程上通常按风险等级划分 TTC 阈值，$TTC < 0.5\,\mathrm{s}$ 对应紧急制动区间，$0.5\,\mathrm{s} \leq TTC < 1.5\,\mathrm{s}$ 对应警戒区间，$1.5\,\mathrm{s} \leq TTC < 4\,\mathrm{s}$ 对应舒适跟车区间。与单纯的距离指标相比，TTC 同时编码了距离和速度信息，能更准确地反映碰撞的紧迫程度。

式(2.15)的一维形式假设两物体沿参考线方向共线运动，实际场景中相对运动常具有横向分量。在 Frenet 坐标系下可将碰撞时间按维度分解，纵向 TTC 以 Δs 与纵向相对速度 $\dot{s}_{ego} - \dot{s}_{obs}$ 构造，横向 TTC 以 Δl 与横向相对速度 $\dot{l}_{ego} - \dot{l}_{obs}$ 构造，二者的较小值反映最先发生碰撞的方向

$$TTC_{2D} = \min\left\{\frac{\Delta s}{\dot{s}_{ego} - \dot{s}_{obs}},\ \frac{\Delta l}{\dot{l}_{ego} - \dot{l}_{obs}}\right\} \tag{2.16}$$

该分解形式与 SL 投影天然契合，使得 TTC 可直接嵌入路径与速度的联合决策，并为第五章多因子威胁度评估提供基础度量。

2.7 本章小结

本章系统介绍了论文涉及的核心技术与理论基础。Frenet 坐标系将二维规划问题分解为纵横向两个一维问题，是 Apollo 路径规划的坐标基础。凸优化和二次规划为路径优化提供了数学工具，第五章的路径边界优化和速度 QP 融合均建立在该理论之上。SL 图中的路径边界构建和障碍物避让方向决策是本文改进的核心对象，第四章将从安全裕度、决策粒度和时空协调三个维度分析其在多障碍

物场景下的缺陷，第五章在此基础上建立时变最优通行带的数学框架并提出高效求解算法。ST 图与速度规划基础为理解路径-速度解耦架构中的协调缺失提供了必要背景，第五章提出的时空可通行走廊正是将时变通行带投影至 ST 空间、与速度约束融合的关键机制。组合优化、区间图连通分量和碰撞时间等概念为第五章的问题建模、障碍物分组算法和威胁度评估提供了直接的理论支撑。Apollo 平台的 Scenario-Stage-Task 架构为改进算法的集成提供了框架基础。

3 Apollo Planning 模块深度分析

本章基于 Apollo 11.0 版本的源码，对 Planning 模块的核心组件进行深度分析，涵盖模块总体架构、ReferenceLine 平滑、路径-速度解耦两阶段优化、PathDecider 决策逻辑和 SpeedBoundsDecider 的 ST 边界构建。

3.1 Planning 模块总体架构

3.1.1 模块入口与数据流

Planning 模块的主入口函数为 `PlanningComponent::Proc()`，每个规划周期由 Cyber RT 调度执行。从输入消息到输出轨迹的完整数据流如图3.1所示。整个处理过程划分为三个层次的函数调用。最外层的 `PlanningComponent::Proc()` 负责从 Cyber RT 消息通道读取五类输入数据并完成路由检查和有效性校验。中间层的 `OnLanePlanning::RunOnce()` 完成车辆状态更新、参考线生成与平滑、轨迹拼接和 Frame 初始化等准备工作。核心执行层的 `OnLanePlanning::Plan()` 通过 ScenarioManager 选择场景，再由 Stage 驱动 Task 列表按顺序执行，最终产生 `ADCTrajectory` 轨迹输出。

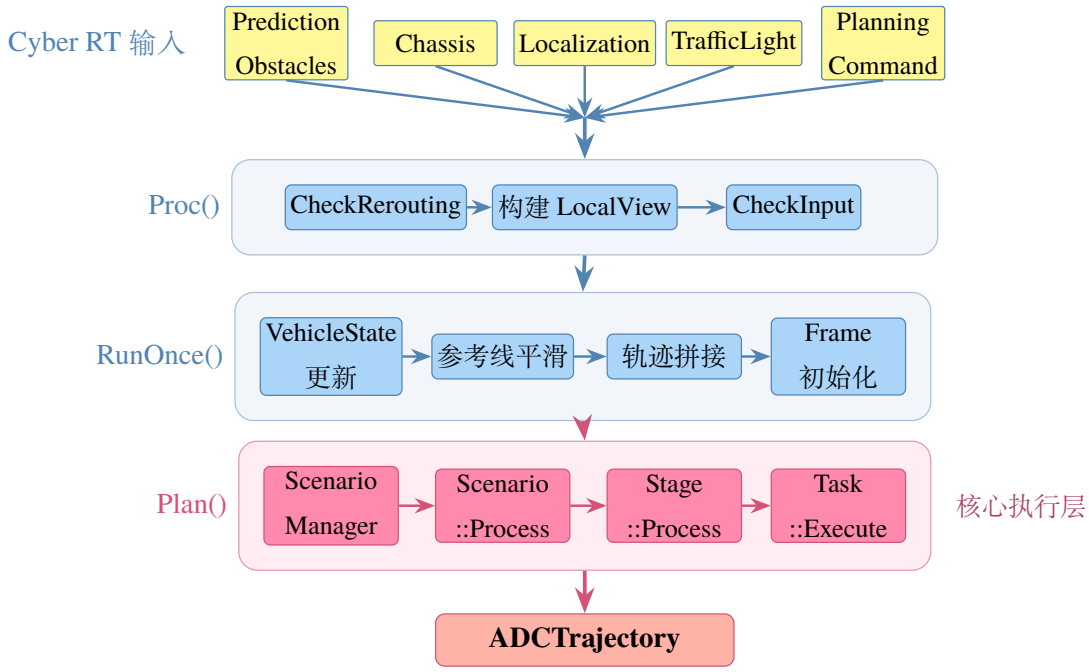


图 3.1 Planning 模块数据流

值得注意的是，Task 列表内部不进行回溯迭代，各 Task 按配置顺序单向执行，前序 Task 的输出作为后续 Task 的输入。这种流水线式的结构对各 Task 的正确性提出了较高要求，若前序 Task 如 `PathBoundsDecider` 出现不必要的保守决策，后续 Task 无法对其进行修正，而只能在其输出的基础上继续处理。这一架构特征是后续章节分析的重要前提。

3.1.2 DependencyInjector 依赖注入

Apollo 通过 `DependencyInjector` 类实现依赖注入，在 `PlanningComponent::Init()` 中创建并传递给各组件。注入的核心依赖包括

表 3.1 DependencyInjector 注入的依赖组件

依赖组件	功能说明
PlanningContext	规划上下文，存储跨周期状态信息
FrameHistory	Frame 历史记录，用于轨迹拼接和平滑
History	ADCTrajectory 历史记录
EgoInfo	自车信息（前方障碍物距离、路由信息）
VehicleStateProvider	车辆状态（位置、速度、加速度、航向）
LearningBasedData	学习模型数据（强化学习模式使用）

3.1.3 场景管理机制

ScenarioManager 在每个规划周期检查当前环境状态，通过调用各 Scenario 的 `IsTransferable()` 方法判断是否需要切换场景。图3.2展示了 Scenario-Stage-Task 的三级层次结构。Apollo 支持 19 种场景类型，表3.2列出了主要场景及其触发条件。

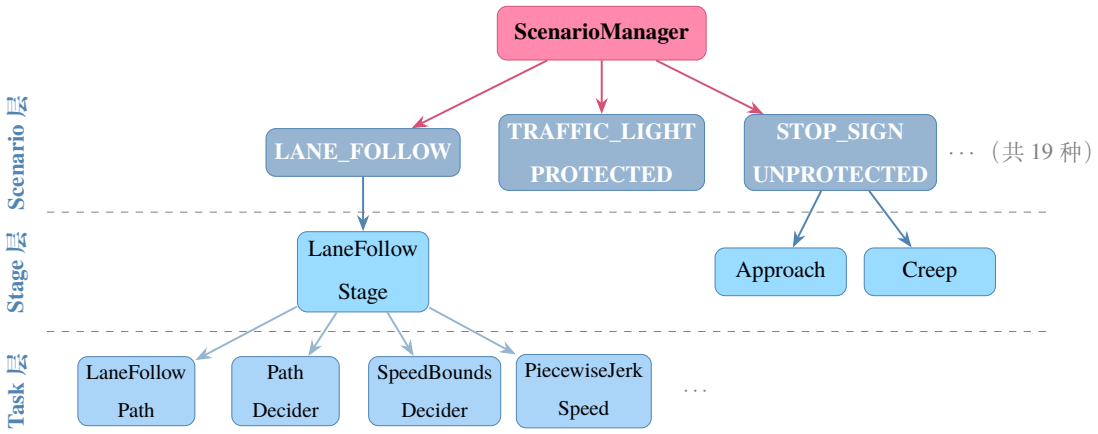


图 3.2 Scenario-Stage-Task 三级管理架构

Scenario-Stage-Task 架构在整体上形成了输入接入层、场景管理层、任务执行层、输出接出层的纵向管线结构，图3.3给出了 Planning 模块的整体模块视图。图中输入层汇聚预测、定位、底盘、路由和交通灯信息，经由三级管理框架分派到具体的 Path 与 Speed Task 并行执行，最终由 TrajectoryStitcher 将路径和速度输出缝合为完整的时空轨迹。在现有 Apollo 实现中，路径侧 Task 包含 LaneChangePath、LaneFollowPath、LaneBorrowPath 和 FallbackPath 四个产生路径的 Task，以及 PathDecider、PathBoundsDecider 等决策 Task；速度侧 Task 包含 SpeedBoundsDecider、PathTimeHeuristicOptimizer 和 PiecewiseJerkSpeedOptimizer 等。本文改进的 PathBoundsDecider 位于图中粉色高亮区块，其输出直接影响后续 Path 优化与速度规划的可行性，是多障碍物场景下全链路闭环的关键节点。

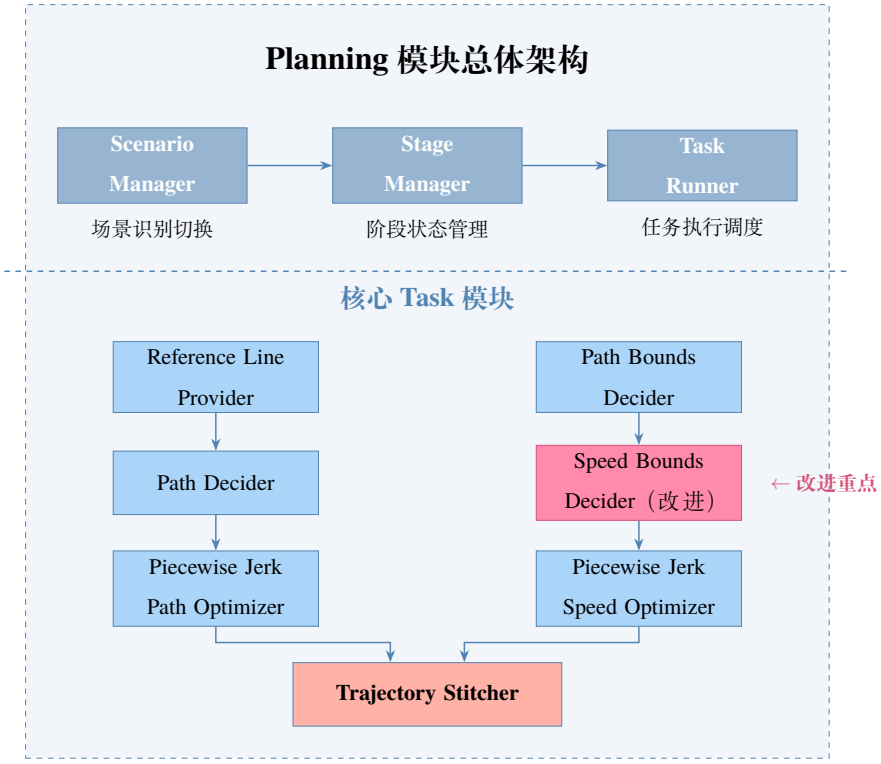


图 3.3 Planning 模块整体架构

3.2 ReferenceLine 平滑

参考线的质量直接影响 Frenet 坐标系下的障碍物 SL 投影精度，进而影响 PathBoundsDecider 的边界构建结果，因此有必要理解参考线的生成过程。

表 3.2 Apollo 主要场景类型及触发条件

场景类型	触发条件
LANE_FOLLOW	默认场景，正常车道跟随
BARE_INTERSECTION_UNPROTECTED	无保护路口(无交通灯、停车标志)
STOP_SIGN_UNPROTECTED	遇到停车标志,距离在配置阈值内
TRAFFIC_LIGHT_PROTECTED	遇到交通灯且有路权保护
TRAFFIC_LIGHT_UNPROTECTED_LEFT_TURN	无保护左转遇到交通灯
PULL_OVER	接近目的地且距离在配置范围内
EMERGENCY_STOP	紧急停车
PARK_AND_GO	停车后重新启动
VALET_PARKING	代客泊车

3.2.1 参考线生成流程

ReferenceLine 是 Planning 模块的规划基准线，其生成流程主要步骤如下。

（1）**获取路由段** 通过 `PncMapBase::GetRouteSegments()` 从 Routing 结果中提取基于车辆当前位置的路由段。

（2）**生成原始参考线** 将路由段转换为 `hdmap::Path`，再转换为 `ReferenceLine`。此时的参考线直接来自高精地图的车道中心线，存在噪声和不连续性。

（3）**生成锚点** 调用 `GetAnchorPoints()`，按 `max_constraint_interval` 间隔在参考线上均匀采样锚点，默认取 5 m。每个锚点包含位置、航向角、横向和纵向边界。

（4）**QP 样条平滑** 以锚点为约束，通过 QP 优化生成平滑的参考线。

（5）**验证与裁剪** 验证平滑后的参考线偏差不超过阈值，并根据车辆位置裁剪。

图3.4展示了参考线平滑的完整过程。

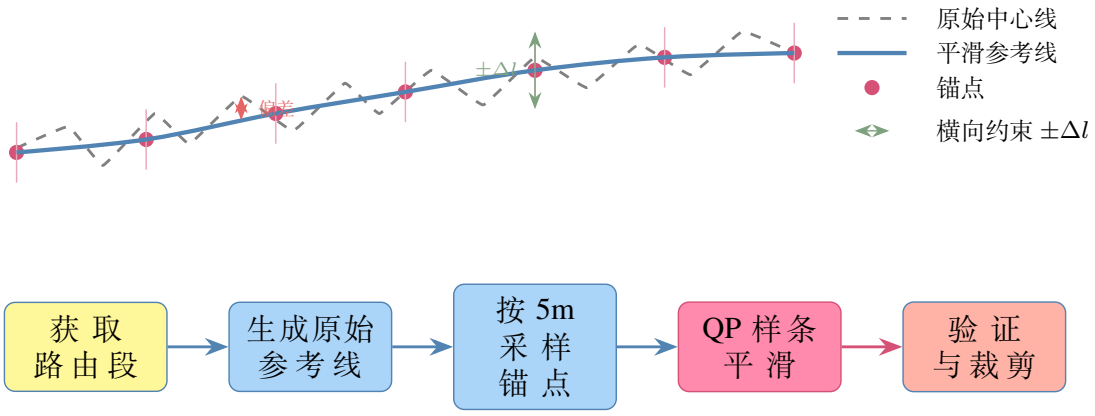


图 3.4 参考线平滑过程示意图

3.2.2 QP 样条平滑的数学建模

参考线平滑采用分段五次多项式（Quintic Spline）表示，平滑问题建模为二维 QP 问题。

目标函数

$$
\begin{aligned}
\min_{\mathbf{p}} \ J = w_2 &\sum_{i=1}^{n} \int_{t_i}^{t_{i+1}} \left[(x_i''(t))^2 + (y_i''(t))^2 \right] dt \\
&+ w_3 \sum_{i=1}^{n} \int_{t_i}^{t_{i+1}} \left[(x_i'''(t))^2 + (y_i'''(t))^2 \right] dt \\
&+ w_r \|\mathbf{p}\|^2
\end{aligned}
\tag{3.1}
$$

其中 w_2 为二阶导数权重，默认取 0.0；w_3 为三阶导数权重，默认取 100；w_r 为正则化权重，默认取 0.1。三阶导数权重远大于二阶导数权重，表明 Apollo 更注重曲率变化率的平滑性。

约束条件包括三类。

（1）**边界约束** 对每个锚点 i，在其局部坐标系下约束横向和纵向偏移

$$
-\Delta l_i \le d_{lateral}(t_i) \le \Delta l_i, \quad -\Delta s_i \le d_{longitudinal}(t_i) \le \Delta s_i \tag{3.2}
$$

其中 Δl_i 根据车道宽度计算，默认最大 0.5 m；Δs_i 为纵向边界，默认取 1.0 m。

（2）**起点航向约束**

$$
\left.\frac{dy}{dt}\right|_{t=t_0} \Big/ \left.\frac{dx}{dt}\right|_{t=t_0} = \tan(\theta_0) \tag{3.3}
$$

（3）**连续性约束** 在每个 knot 连接处保证 C^2 连续

$$
f_i(t_{i+1}) = f_{i+1}(t_{i+1}), \quad f_i'(t_{i+1}) = f_{i+1}'(t_{i+1}), \quad f_i''(t_{i+1}) = f_{i+1}''(t_{i+1}) \tag{3.4}
$$

Knot 划分 根据参考线总长度和 `max_spline_length` 计算样条段数，默认 25 m$n = \lceil L/25 \rceil$，在参数空间 $t \in [0, n]$ 均匀分布 knot 节点。

3.2.3 参考线数据结构

平滑后的参考线由一系列 `ReferencePoint` 构成，每个点包含的字段如表3.3所示。

表 3.3 ReferencePoint 数据结构

字段	单位	说明
x, y	m	全局笛卡尔坐标
$heading$	rad	航向角
$kappa$	1/m	曲率
$dkappa$	$1/m^2$	曲率变化率
s	m	沿参考线的累积弧长
$lane$	—	关联的车道信息
l	m	相对车道中心线的横向偏移

3.3 路径-速度解耦两阶段优化

Apollo 2.0 至 5.0 版本曾采用名为 EM（Expectation-Maximization）Planner[3]的独立规划器，以路径-速度解耦的两阶段优化为核心思想。在 Apollo 11.0 中，EM Planner 已不再作为独立规划器存在，其路径-速度解耦的两阶段架构被融入了默认规划器 PublicRoadPlanner 的 Scenario-Stage-Task 流水线中。当前版本的四个规划器为 NaviPlanner、LatticePlanner、RTKReplayPlanner 和 PublicRoadPlanner，其中 PublicRoadPlanner 为默认规划器，通过 Task 流水线继承了 EM Planner 的两阶段优化思想。

路径-速度解耦的核心做法是将三维时空轨迹规划降维为两个串联的二维规划子问题。第一阶段在忽略时间维度的前提下直接在 SL 空间上规划横向路径 $l(s)$，第二阶段在已知 $l(s)$ 的基础上沿参考线弧长规划速度曲线 $s(t)$。图3.5给出了该流水线的整体结构。参考线、感知障碍物、预测轨迹和车辆状态四种输入在阶段一合流于 PathBoundsDecider 与 PiecewiseJerkPathOptimizer，产出离散路径点序列；阶段二将该路径作为已知条件建立 ST 图，由 DP 启发式搜索给出粗解作为热启动，再交由 QP 精细优化输出最终速度曲线；阶段三的输出与路径合成 `ADCTrajectory`。这种降维的代价是路径阶段必须独立做出如左右绕行和跟随超车等决策，无法根据动态障碍物的时间演化来调整，这也是第四章所揭示问题的结构性根源。

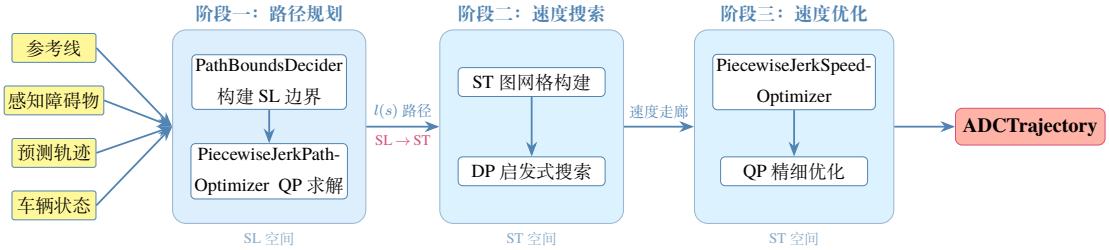


图 3.5 路径-速度解耦三阶段架构

3.3.1 路径规划阶段

路径规划阶段通过一系列 Path Task 生成局部路径，典型的 Task 包括 `LaneFollowPath` 即车道跟随路径、`LaneChangePath` 即换道路径、`LaneBorrowPath` 即借道路径。路径优化采用与参考线平滑类似的 QP 方法，目标函数包含平滑性、参考线偏离和边界约束等项。

按 Apollo 11.0 默认 PublicRoadPlanner 在 LaneFollow 场景下的 pipeline 配置，路径阶段由一条顺序执行的 Task 链承担，依次为 `PathReuseDecider`、`PathBoundsDecider`、`PiecewiseJerkPathOptimizer`、`PathAssessmentDecider` 和 `PathDecider`。

PathReuseDecider 判断当前周期是否可以直接复用上周期已通过评估的路径，若可复用则跳过后续优化步骤，从而在非关键场景下降低计算开销。

PathBoundsDecider 沿参考线在每个离散纵向位置 s 上聚合车道几何、静态障碍物投影和借道决策，构建横向可行走廊 $[l^-(s), l^+(s)]$ 作为优化问题的硬约束。

PiecewiseJerkPathOptimizer 以 $l(s)$ 的分段 jerk 最小化为核心，在离散化网格上将平滑性、参考线偏离和边界约束组织为稀疏二次规划，调用 OSQP 求解得到横向位置、横向速度和横向加速度三元组序列。

PathAssessmentDecider 对候选路径进行有效性筛查，包含终点可达性、与静态障碍物的碰撞检测以及通行带是否被完全阻塞等判定，若全部候选路径均失效则触发 `BLOCKED` 标记并回退至 FallbackPath。

PathDecider 在最终选定的路径上遍历所有障碍物，根据其与路径的相对横向距离和语义属性生成 `NUDGE`、`IGNORE` 等横向决策标签，后续速度规划阶段将据此构建 ST 边界。

上述 Task 链顺序执行、不做回溯，最终输出 `PathData`，其中封装了离散路径点序列及其 Frenet 坐标表达，作为速度规划阶段构建 ST 图和跟踪参考的唯一

几何输入。

3.3.2 速度规划阶段

速度规划分为粗优化和精优化两步。

粗优化，动态规划（Dynamic Programming, DP）搜索

在 ST 图的离散栅格上进行动态规划搜索。栅格的时间维度按固定步长 Δt 均匀采样，空间维度分为 dense 和 sparse 两部分以平衡精度和效率。DP 的代价函数包括

$$C_{total} = C_{obstacle} + C_{spatial} + C_{speed} + C_{accel} + C_{jerk} \tag{3.5}$$

其中各项代价的计算方式如下。

障碍物代价

$$C_{obstacle} = \begin{cases} +\infty & \text{栅格点落在障碍物边界内} \\ w_{obs} \cdot c_{default} \cdot (d_{follow} - d_{actual})^2 & \text{栅格点靠近障碍物边界} \end{cases} \tag{3.6}$$

速度代价

$$C_{speed} = w_{exceed}(v - v_{limit})^2 + w_{low}|v - v_{cruise}| + w_{ref}|v - v_{ref}| \tag{3.7}$$

加加速度代价

$$j = \frac{s_4 - 3s_3 + 3s_2 - s_1}{(\Delta t)^3} \tag{3.8}$$

DP 通过递推计算每个栅格点的最小累积代价，并记录前驱指针，最终通过回溯获得粗略的最优速度曲线。

精优化，PiecewiseJerk QP

以 DP 结果为参考轨迹，建立 QP 问题进行精细化优化

$$\min \sum_{k=0}^{K-1} \left[w_s(s_k - s_{ref,k})^2 + w_v(v_k - v_{ref,k})^2 + w_a a_k^2 + w_j \left(\frac{a_{k+1} - a_k}{\Delta t} \right)^2 \right] \tag{3.9}$$

DP 输出的速度曲线作为 $v_{ref,k}$ 传入 QP，为非凸的 ST 边界约束提供了“暖启动”，引导 QP 在正确的可行区域内求解。

3.3.3 两阶段信息传递

两阶段之间的信息传递是单向的，路径阶段输出 `PathData`，包含离散路径点和 Frenet 坐标系路径，速度阶段在此路径的基础上构建 ST 图并进行速度优化。在 Apollo 的 Task 流水线中，各 Task 按配置顺序执行，不进行回溯迭代。LANE_FOLLOW 场景的完整 Task 流水线配置如下。

代码 3.1 LANE_FOLLOW 场景 Task 流水线配置

```
// scenarios/lane_follow/conf/pipeline.pb.txt
stage {
  name: "LANE_FOLLOW_STAGE"
  type: "LaneFollowStage"
  task { name: "LANE_CHANGE_PATH"      type: "LaneChangePath" }
  task { name: "LANE_FOLLOW_PATH"      type: "LaneFollowPath" }
  task { name: "LANE_BORROW_PATH"      type: "LaneBorrowPath" }
  task { name: "FALLBACK_PATH"         type: "FallbackPath" }
  task { name: "PATH_DECIDER"          type: "PathDecider" }
  task { name: "RULE_BASED_STOP_DECIDER" type: "RuleBasedStopDecider" }
  task { name: "SPEED_BOUNDS_PRIORI_DECIDER" type: "SpeedBoundsDecider" }
  task { name: "SPEED_HEURISTIC_OPTIMIZER" type: "PathTimeHeuristicOptimizer" }
  task { name: "SPEED_DECIDER"         type: "SpeedDecider" }
  task { name: "SPEED_BOUNDS_FINAL_DECIDER" type: "SpeedBoundsDecider" }
  task { name: "PIECEWISE_JERK_SPEED"  type: "PiecewiseJerkSpeedOptimizer" }
}
```

其中，路径阶段包含四种路径生成 Task，即换道、跟车道、借道、降级，以及路径决策和规则停车决策两个决策 Task；速度阶段包含先验速度边界构建、DP 启发式搜索、速度决策、最终速度边界构建和 QP 速度优化五个 Task。SpeedBoundsDecider 被调用两次，分别在 DP 搜索前后执行先验和最终的边界构建。

此外，Apollo 还提供了 Lattice Planner[17]作为备选规划器，其采用 Frenet 坐标系下的采样方法生成候选轨迹。由于本文研究聚焦于 LANE_FOLLOW 场景下的路径边界决策改进，Lattice Planner 不在本文讨论范围内。

3.4 PathDecider 决策分析

PathDecider 是路径规划中的关键决策模块，负责对参考线上的每个障碍物做出路径决策。

3.4.1 决策类型与优先级

PathDecider 的决策分为横向和纵向两类，各类型及其语义优先级如表3.4所示。

注，Apollo 源码中决策类型定义为枚举类型，不使用显式数值优先级。实际处理中，STOP 决策的优先级最高，一旦判定需要停车便会覆盖其他决策，IGNORE

表 3.4 PathDecider 决策类型

方向	决策类型	说明
横向	NUDGE（LEFT/RIGHT）	横向避让（左绕/右绕）
	IGNORE	横向安全，忽略
纵向	STOP	停车让行（最高优先）
	YIELD	减速让行
	FOLLOW	跟车
	OVERTAKE	超车
	IGNORE	纵向安全，忽略（最低优先）

的优先级最低。

3.4.2 决策逻辑

PathDecider 的核心决策逻辑基于障碍物的 SL 边界与当前路径的横向关系进行判断。

（1）**IGNORE 判断** 若障碍物不在路径 s 范围内，即 $s_{end} < s_{path_start}$ 或 $s_{start} > s_{path_end}$，或横向距离足够远，即 $|l_{obs} - l_{ego}| > w_{ego}/2 + d_{ignore}$，则 IGNORE。其中 $d_{ignore} = 3.0\,\mathrm{m}$。

（2）**STOP 判断** 若障碍物是 blocking obstacle，或横向重叠区域超过阈值，即 $l_{obs_end} \geq l_{ego} - l_{min_nudge}$ 且 $l_{obs_start} \leq l_{ego} + l_{min_nudge}$，则 STOP。其中 $l_{min_nudge} = w_{ego}/2 + d_{static_buffer}/2$，$d_{static_buffer} = 0.3\,\mathrm{m}$。

（3）**NUDGE 判断** 若障碍物在可避让范围内，根据其相对于自车的横向位置决定 LEFT_NUDGE 或 RIGHT_NUDGE。

关键特征 PathDecider 仅处理静态障碍物，动态障碍物的处理交由速度规划阶段的 ST 图来完成。

3.4.3 PathBoundsDecider 边界构建

`PathBoundsDecider` 负责构建路径的横向边界，其合并策略如下。

对每个 s 位置，遍历所有障碍物，根据 nudge 方向更新路径边界

- RIGHT_NUDGE，$l_{upper} = \min(l_{upper}, l_{obs_right_bound})$
- LEFT_NUDGE，$l_{lower} = \max(l_{lower}, l_{obs_left_bound})$

这种取最小/最大的合并策略是保守的，多个障碍物的约束取交集，确保路径不与任何障碍物冲突，但可能导致可行域被严重压缩。

PathBoundsDecider 的完整处理流程如下。

（1）初始化路径边界，将上下界分别设为道路左右边界 $[l^{-}_{road}(s), l^{+}_{road}(s)]$；

（2）遍历参考线上的纵向采样点 s_i；

（3）对每个 s_i，调用 `UpdatePathBoundaryBySLPolygon` 函数，依次处理该截面上的各个障碍物；

（4）对每个障碍物，通过 `SetNudgeInfo` 函数判定避让方向，若障碍物中心在参考线右侧则左绕，即将右边界收缩，若在左侧则右绕，即将左边界收缩；

（5）逐障碍物更新边界后，若出现 $l^{+}(s_i) < l^{-}(s_i)$ 则标记为 BLOCKED。

图3.6以流程图形式给出了该过程，可以清晰看到避让方向判定和边界更新两个关键节点在循环体内被逐障碍物触发。图中 BLOCKED 检查节点位于每次更新之后，一旦触发将直接终止规划并向上层返回降级请求；若通过检查则继续处理下一个障碍物，直到遍历完毕输出完整的 path_boundary。图右侧标注同时提示了本流程的核心设计缺陷，即决策一旦在循环中做出便不可修正，后续的障碍物无论与已做决策是否存在空间冲突，均无法回头重新选择更合适的避让方向。

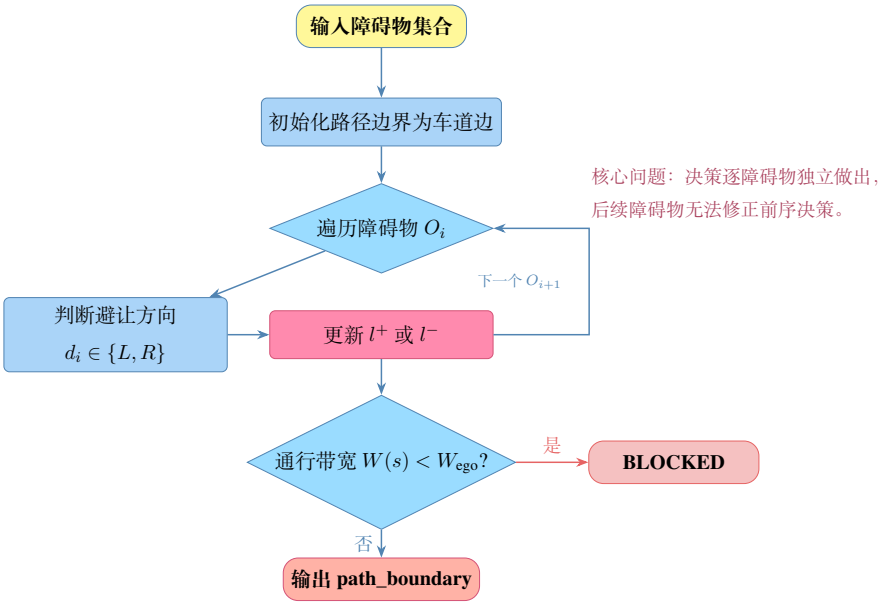


图 3.6 PathBoundsDecider 边界构建流程

该流程存在两个关键特征。第一，避让方向决策是逐障碍物独立进行的，每个障碍物仅根据自身位置相对于参考线的横向偏移做出左绕或右绕判定，不考虑其他障碍物已做出的决策对剩余通行空间的影响。第二，障碍物过滤函数 `IsWithinPathDeciderScopeObstacle` 将所有动态障碍物排除在路径边界决策之外，

仅由后续的速度规划阶段处理动态障碍物。这两个特征将在第四章中详细分析其在多障碍物场景下引发的问题。

3.5 SpeedBoundsDecider 概述

SpeedBoundsDecider 是速度规划阶段的核心模块，负责在路径已确定的基础上，将障碍物映射到 ST 图中，构建 ST 边界作为后续速度 QP 优化的约束。其处理对象是路径规划阶段输出的 PathData，对路径上的每个点检查与障碍物的碰撞关系。

3.5.1 ST 边界构建流程

SpeedBoundsDecider 的处理流程分为三步。第一步，沿已确定的路径逐点检查与所有障碍物的碰撞关系，将每个障碍物在 ST 图中的占据区域表示为一个多边形。静态障碍物在 ST 图中表现为沿时间轴延伸的矩形带状区域，动态障碍物表现为斜向的平行四边形，其斜率由障碍物沿参考线方向的速度决定。第二步，在每个时间步 t_j 处，汇总所有障碍物 ST 多边形的约束，综合 YIELD、FOLLOW 和 OVERTAKE 三类决策，分别给出纵向位置的上界 s_j^{ub} 和下界 s_j^{lb}。YIELD 和 FOLLOW 决策要求自车位于障碍物之后，约束体现为 s_j^{ub} 的收紧；OVERTAKE 决策要求自车位于障碍物之前，约束体现为 s_j^{lb} 的抬升。第三步，在 ST 边界之上叠加纵向安全距离 d_s，最终输出供后续速度优化器使用的 $[s_j^{lb}, s_j^{ub}]$ 序列。

3.5.2 与 PathBoundsDecider 的架构差异

SpeedBoundsDecider 与 PathBoundsDecider 分处路径-速度解耦架构的不同阶段，两者存在三个关键差异。

第一，处理的障碍物范围不同。PathBoundsDecider 仅处理静态障碍物，而 SpeedBoundsDecider 同时处理静态和动态障碍物。动态障碍物在路径阶段被完全排除，只能在速度阶段通过 ST 图处理。

第二，决策协调机制不同。PathBoundsDecider 对每个障碍物独立做出左绕/右绕的贪心决策，没有全局协调。SpeedBoundsDecider 之后的 `PathTimeHeuristicOptimizer` 即 DP 启发式搜索，在 ST 图中对 YIELD/OVERTAKE 的组合进行全局搜索，将 DP 输出作为后续 QP 的暖启动参考，避免了贪心策略的系统性缺陷。

第三，信息传递方向单向。路径阶段的输出即 PathData 传递给速度阶段作为已知条件，但路径阶段做出决策时的上下文信息，例如” 该路径依赖某动态

障碍物在特定时刻移开” 这一时间有效性前提，并未传递给速度阶段。速度阶段只知道路径在哪里，不知道路径为什么在那里。这一信息断层导致路径决策和速度规划之间缺乏时空约束的协调，是第四章所揭示的 ST 层问题的架构根源。第五章提出的时变通行带框架输出时间有效性约束，以 ST 上界的形式与 SpeedBoundsDecider 的输出在 QP 层面融合，建立此前缺失的路径-速度约束传递通道。

3.6 本章小结

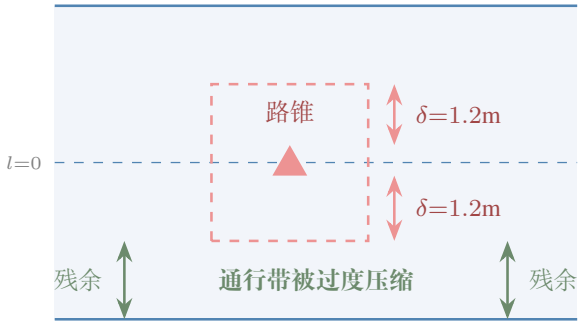
本章对 Apollo Planning 模块的核心组件进行了源码级深度分析。Apollo 采用了成熟的工程架构，即 Scenario-Stage-Task 三级管理，以及经典的算法组合，包括 QP 样条平滑和路径-速度解耦优化，默认的 PublicRoadPlanner 通过 Task 流水线继承了早期 EM Planner 的路径-速度解耦思想，在一般场景下表现良好。PathDecider 和 PathBoundsDecider 是路径规划阶段的核心决策模块，前者做出 STOP/NUDGE/IGNORE 决策，后者构建横向可通行边界，两者均仅处理静态障碍物，且 PathBoundsDecider 对每个障碍物独立决定避让方向，使用统一的安全裕度。SpeedBoundsDecider 在速度规划阶段构建 ST 边界，虽然有 DP 全局搜索进行协调，但与路径阶段之间的信息传递是单向的、有限的。上述架构特征，即逐障碍物独立决策、统一安全裕度、动态障碍物排除和路径-速度信息断层，为下一章从安全距离、决策粒度和时空协调三个维度分析多障碍物场景下的问题奠定了基础。

4 多障碍物场景下的问题分析

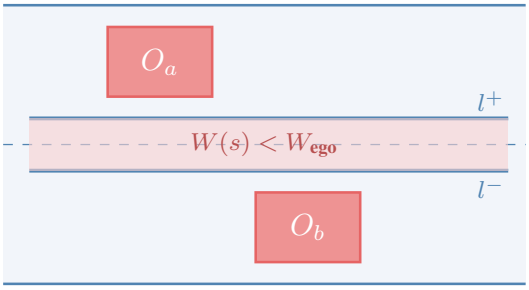
本章基于第三章的源码分析，系统识别 Apollo 现有算法在多障碍物密集场景下的三个核心问题，即安全裕度一刀切压缩通行空间、逐障碍物独立决策导致矛盾避让方向、动态障碍物被路径决策完全排除导致路径-速度协调缺失。三个问题分别从安全距离维度、决策粒度维度和时空协调维度指向相应的改进方向，具体算法设计见第五章。

4.1 问题概述

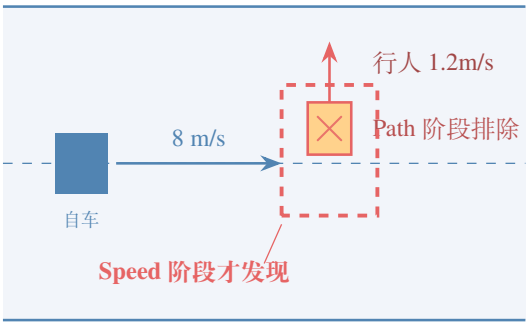
在多障碍物密集场景中，自车前方同时存在多个障碍物，例如路口多车交汇、高速多车道变道、施工区域多路障等，Apollo 现有的 Planning 模块会面临一系列问题。图4.1给出了三个问题的几何示意与级联后果。子图 (a) 展示安全裕度一刀切导致的通行带压缩，窄小的路锥被过大的统一缓冲包裹，挤占了本应富余的横向空间。子图 (b) 展示独立决策矛盾导致的路径 BLOCKED，左右两侧障碍物被贪心分配了相反方向的避让决策，使得 $l^{+}(s)$ 与 $l^{-}(s)$ 在中间区域相遇甚至翻转，可行带宽不足以容纳自车宽度。子图 (c) 展示动态障碍物被排除在路径阶段之外，行人的预测轨迹在 Path 阶段完全不可见，导致规划出的路径与速度阶段检测到的实际碰撞窗口相互矛盾。三个问题在多障碍物密集场景中沿空间、安全、时间三条维度级联作用，最终表现为不必要的 FallbackPath 停车和通行效率下降，这也是后续章节改进方案同时从分组决策、差异化安全裕度和时变投影三个角度切入的根本原因。



(a) 安全裕度一刀切



(b) 独立决策矛盾



(c) 动态障碍物排除

图 4.1 多障碍物场景三个核心问题的几何示意

4.2 安全裕度一刀切压缩通行空间

4.2.1 问题描述

PathBoundsDecider 在构建路径边界时，需要在障碍物边缘外预留安全距离，以确保自车与障碍物之间保持足够的横向间距。该安全距离由 `GetBufferBetweenADCCenterAndEdge` 函数统一计算

代码 4.1 安全距离统一计算与 nudge 缓冲的静态/非静态分支

```
// path_bounds_decider_util.cc
// 相关 gflags 默认值 (planning_gflags.cc)
//   obstacle_lat_buffer          = 0.4 m
//   static_obstacle_nudge_l_buffer = 0.3 m
//   nonstatic_obstacle_nudge_l_buffer = 0.4 m
//   static_obstacle_speed_threshold = 0.5 m/s

// 所有障碍物统一调用此函数计算安全距离
double PathBoundsDeciderUtil::
    GetBufferBetweenADCCenterAndEdge() {
  double adc_half_width =
      VehicleConfigHelper::GetConfig()
          .vehicle_param().width() / 2.0;
  return (adc_half_width + FLAGS_obstacle_lat_buffer);
}

// --- 静态/非静态的判定条件 ---
if (!obstacle.IsStatic() ||
    obstacle.speed() > FLAGS_static_obstacle_speed_threshold) {
  return false; // 非静态障碍物不参与路径决策
}

// --- 边界松弛中的静态/非静态缓冲分支 ---
// 非静态障碍物使用 nonstatic buffer
double new_buffer = std::max(FLAGS_ego_front_slack_buffer,
                             FLAGS_nonstatic_obstacle_nudge_l_buffer);
// ...
// 静态障碍物使用 static buffer
if (path_bound_point->is_nudge_bound[LEFT_INDEX]) {
  old_buffer = std::max(FLAGS_obstacle_lat_buffer,
                        FLAGS_static_obstacle_nudge_l_buffer);
}
```

从代码可以看出，Apollo 对安全距离的处理存在两个层面的粗粒度问题

（1）`GetBufferBetweenADCCenterAndEdge` 函数对所有障碍物返回相同的值，即半车宽加 0.4 m，完全不区分障碍物的类型、速度、横向偏移等特征。无论是远离车道的路锥、车道边缘的花坛，还是正对车道的违停车，均使用相同的安全距离。

（2）Apollo 虽然在 `nudge_l_buffer` 参数中区分了静态 0.3 m 和非静态 0.4 m 两个等级，但仅以 0.5 m/s 的速度阈值做二分，低于该阈值视为静态，高于则视为非静态。该二分判定在 PathBoundsDecider 与 ObstacleNudgeDecider 的缓冲计算中通过 `!IsStatic()||speed()>FLAGS_static_obstacle_speed_threshold` 条件式生效。这种二分法无法区分运动方向随时突变的行人与轨迹可预测的同向行驶车辆，也无法区分碰撞时间短的正面接近障碍物与碰撞风险低的横向远离障碍物。

4.2.2 影响分析

以下通过一个具体场景说明统一安全距离如何导致不必要的通行失败。定义净间距为道路边界到障碍物边缘之间扣除缓冲距离后的剩余宽度，自车可从

某一侧通过当且仅当该侧净间距不小于自车宽度。

场景设定 标准车道宽 3.75 m，$l_{road}^{-} = -1.875\,\mathrm{m}$，$l_{road}^{+} = 1.875\,\mathrm{m}$。车道中偏右位置有一个路锥，横向占据 $[-0.375,\ -0.075]\,\mathrm{m}$，宽度 0.3 m。自车宽度 $W_{ego} = 1.8\,\mathrm{m}$。

Apollo 统一缓冲 取 $d_{buf} = 0.3\,\mathrm{m}$。从左侧通过时，净间距为

$$1.875 - (-0.075) - 0.3 = 1.65\,\mathrm{m} < W_{ego} = 1.8\,\mathrm{m}$$

从右侧通过时，净间距为

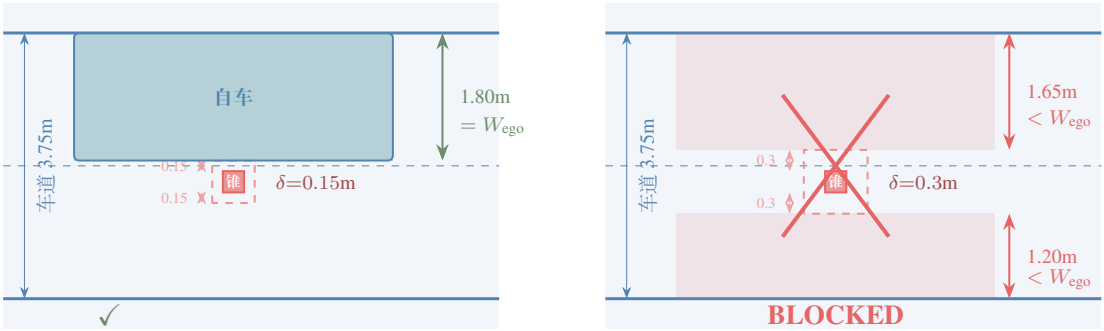
$$(-0.375) - (-1.875) - 0.3 = 1.20\,\mathrm{m} < 1.8\,\mathrm{m}$$

两侧均不足以容纳自车，路径被判定为 BLOCKED，自车停车等待。

差异化缓冲 该路锥处于静止状态且远离车道中心，威胁较低，取 $d_{buf} = 0.15\,\mathrm{m}$。从左侧通过时，净间距为

$$1.875 - (-0.075) - 0.15 = 1.80\,\mathrm{m} = W_{ego}$$

恰好等于自车宽度，自车可从左侧安全通过。仅因缓冲参数从 0.3 m 降至 0.15 m，净间距就从 1.65 m 提升至 1.80 m，通行结果从 BLOCKED 翻转为可行。图4.2直观对比了两种缓冲参数下的通行带差异。按障碍物威胁程度差异化分配安全距离的具体方法将在第五章展开。



(a) δ=0.15**m（差异化），恰好可通行**　　**(b)** δ=0.3**m（统一），BLOCKED**

图 4.2 统一缓冲与差异化缓冲下的通行带对比

4.3 SL 层路径边界独立决策缺陷

4.3.1 问题描述

PathBoundsDecider 是 Apollo 路径规划阶段的关键模块，负责在每个离散纵向位置 s 处构建横向路径边界 $[l^-(s),\ l^+(s)]$。其核心缺陷在于对每个障碍物独立地决定避让方向（LEFT_NUDGE 或 RIGHT_NUDGE），而不考虑同一 s 截面上其他活跃障碍物的存在。当道路两侧同时存在障碍物时，这种逐障碍物的贪心决策可能产生相互矛盾的避让方向，即左侧障碍物使上边界下压、右侧障碍物使下边界上抬，导致 $l^+(s) < l^-(s)$，路径边界被判定为 BLOCKED，最终触发 FallbackPath 停车。

4.3.2 源码定位与决策逻辑

该问题的根源在于 `UpdatePathBoundaryBySLPolygon` 函数。其核心逻辑包括三个环节，即逐障碍物独立决定避让方向、根据避让方向收缩路径边界、检测边界交叉并截断路径。代码4.2展示了该函数的关键流程。

代码 4.2 UpdatePathBoundaryBySLPolygon 核心逻辑

```
1  // path_bounds_decider_util.cc: UpdatePathBoundaryBySLPolygon
2  for (size_t i = 1; i < path_boundary->size(); ++i) {
3    auto& left_bound = path_boundary->at(i).l_upper; // 上边界
4    auto& right_bound = path_boundary->at(i).l_lower; // 下边界
5    for (size_t j = 0; j < sl_polygon->size(); j++) {
6      // --- 步骤1: 逐障碍物独立决定避让方向 ---
7      if (sl_polygon->at(j).NudgeInfo() == SLPolygon::UNDEFINED) {
8        double obs_l = (l_lower + l_upper) / 2; // 障碍物中心横向位置
9        if (is_passable_right && is_passable_left) {
10         // 两侧均可通行时，根据障碍物中心与参考线的相对位置决定
11         if (last_max_nudge_l < obs_l) {
12           sl_polygon->at(j).SetNudgeInfo(SLPolygon::RIGHT_NUDGE);
13         } else {
14           sl_polygon->at(j).SetNudgeInfo(SLPolygon::LEFT_NUDGE);
15         }
16       } else if (is_passable_right) {
17         sl_polygon->at(j).SetNudgeInfo(SLPolygon::RIGHT_NUDGE);
18       } else {
19         sl_polygon->at(j).SetNudgeInfo(SLPolygon::LEFT_NUDGE);
20       }
21     }
22     // --- 步骤2: 根据避让方向更新边界 ---
23     if (sl_polygon->at(j).NudgeInfo() == SLPolygon::RIGHT_NUDGE) {
24       if (!sl_polygon->at(j).is_passable()[RIGHT_INDEX]) {
25         sl_polygon->at(j).SetNudgeInfo(SLPolygon::BLOCKED);
26         break; // 右侧不可通行，标记BLOCKED
27       }
28       // 上边界收缩至障碍物左边缘减安全距离
29       if (obs_right_nudge_bound.l_upper.l < left_bound.l) {
30         left_bound.l = obs_right_nudge_bound.l_upper.l;
31         left_bound.type = BoundType::OBSTACLE;
32       }
33     } else if (sl_polygon->at(j).NudgeInfo() == SLPolygon::LEFT_NUDGE) {
34       if (!sl_polygon->at(j).is_passable()[LEFT_INDEX]) {
35         sl_polygon->at(j).SetNudgeInfo(SLPolygon::BLOCKED);
36         break; // 左侧不可通行，标记BLOCKED
37       }
38       // 下边界上抬至障碍物右边缘加安全距离
39       if (obs_left_nudge_bound.l_lower.l > right_bound.l) {
40         right_bound.l = obs_left_nudge_bound.l_lower.l;
41         right_bound.type = BoundType::OBSTACLE;
42       }
43     }
44   }
45   // --- 步骤3: BLOCKED判定与路径截断 ---
46   if (!blocked_id->empty()) {
47     TrimPathBounds(i, path_boundary); // 截断后续路径边界
48     return false;                     // 触发FallbackPath停车
49   }
50 }
```

代码中步骤 1，即第 7 至 22 行，展示了避让方向的独立决策，对每个在位置 s 处活跃的障碍物，通过 SetNudgeInfo 函数根据障碍物中心相对于参考线的横向位置独立决定 RIGHT_NUDGE，即从右侧绕行使上边界下压，或 LEFT_NUDGE，即从左侧绕行使下边界上抬。步骤 2，即第 23 至 44 行，根据避让方向对路径边界进行收缩，其中安全距离 delta 由自车半宽，即 GetBufferBetweenADCCenterAndEdge 的返回值，与额外缓冲参数 obstacle_lat_buffer 相加得到。步骤 3，即第 46 至 49 行，在上下边界交叉，即存在 BLOCKED 标记的障碍物时，调用 TrimPathBounds 清除后续路径边界并返回 false，最终触发 FallbackPa

th 生成减速停车轨迹。

4.3.3 失败模式分析

以下通过一个对称场景说明独立决策如何导致不必要的停车。

场景设定 双车道道路总宽 7.5 m，$l^-_{road} = -3.75\,\mathrm{m}$，$l^+_{road} = 3.75\,\mathrm{m}$。在某纵向位置 s^* 处同时存在两个路锥，关于参考线近似对称

- O_a 偏右，横向占据 $[0.1,\ 0.4]\,\mathrm{m}$，中心 $0.25\,\mathrm{m}$
- O_b 偏左，横向占据 $[-0.4,\ -0.1]\,\mathrm{m}$，中心 $-0.25\,\mathrm{m}$

两个路锥均未越过车道中线，安全距离 $\delta = 1.2\,\mathrm{m}$。

Apollo 独立决策 O_a 中心 $0.25 > 0$，分配 RIGHT_NUDGE，上边界收缩至

$$l^+(s^*) = 0.1 - 1.2 = -1.1\,\mathrm{m}$$

O_b 中心 $-0.25 < 0$，分配 LEFT_NUDGE，下边界抬升至

$$l^-(s^*) = -0.1 + 1.2 = 1.1\,\mathrm{m}$$

$l^+ = -1.1 < l^- = 1.1$，判定 BLOCKED。贪心策略将自车挤入两个路锥之间仅 0.2 m 的间隙，路径不可行，触发停车。

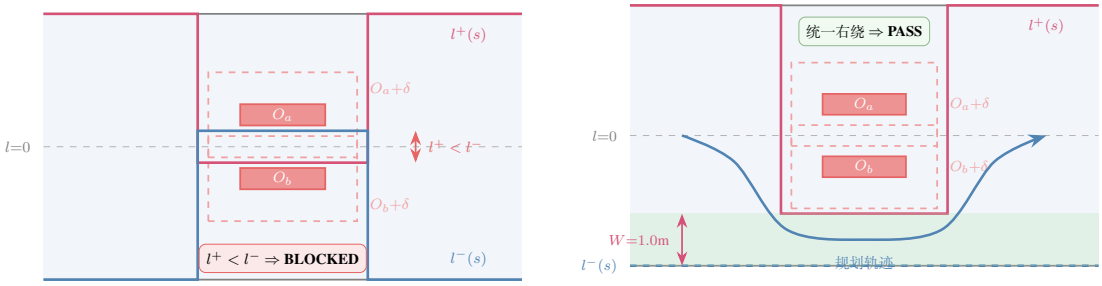
全局统一决策 将两个路锥作为一组，统一分配 RIGHT_NUDGE，自车从两者右侧绕行

$$l^+(s^*) = \min(0.1 - 1.2,\ -0.4 - 1.2) = -1.6\,\mathrm{m}, \quad l^-(s^*) = -3.75\,\mathrm{m}$$

通行带宽度 $W = -1.6 - (-3.75) = 2.15\,\mathrm{m}$，自车可借用相邻车道从右侧安全通过。由对称性，统一 LEFT_NUDGE 同样可行，$W = 3.75 - 1.6 = 2.15\,\mathrm{m}$。

贪心策略将两个偏向不同侧的障碍物分别向内挤压，全局协调只需统一绕行方向，即可利用道路另一侧的充裕空间。当 k 个障碍物在同一纵向位置活跃时，避让方向共有 2^k 种组合，贪心策略只检查了其中一种，可能恰好选中不可行的组合而遗漏可行解。该问题的形式化建模与高效求解方法将在第五章详细展开。

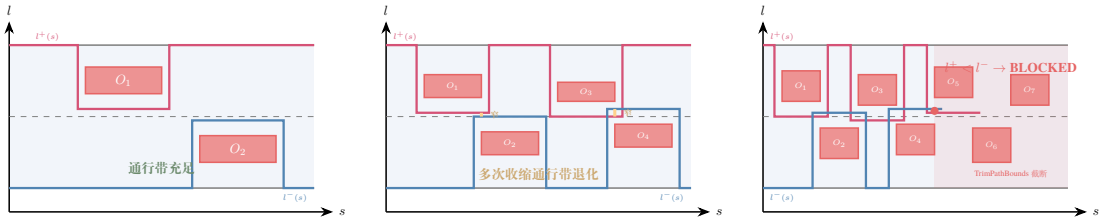
图4.3直观展示了该场景中独立决策与全局协调的对比。



(a) 独立决策，矛盾避让 → BLOCKED **(b) 全局协调，统一右绕 → 通行**

图 4.3 独立决策导致矛盾避让方向与全局协调的对比

除了两个障碍物的极端情形外，随着障碍物数量增加，独立决策导致的可行域压缩会以近似指数方式恶化。图4.4给出了同一车道内从 2 个、5 个到极端密度障碍物场景下的可行域演化。在 2 个障碍物场景下，尚存较宽的可行通带可供自车选择，尽管贪心决策已开始压缩通带宽度；5 个障碍物场景下，多次独立收缩使得可行域退化为断续的狭窄通带，任一处出现贪心矛盾即触发 BLOCKED；极端密度场景下，可行域进一步碎片化，几乎任何贪心决策组合都会导致某处边界失效。这一演化揭示了问题二在高密度场景下的致命性，也量化说明了改进“决策粒度”对可行性恢复的杠杆效应远高于单纯调整安全裕度。



(a) 2 个障碍物 **(b) 4 个障碍物** **(c) 密集障碍物**

图 4.4 多障碍物场景下可行域的退化演化

4.3.4 BLOCKED 后的截断与恢复机制局限

当 PathBoundsDecider 判定 BLOCKED 后，调用 `TrimPathBounds` 截断路径

代码 4.3 TrimPathBounds 路径截断

```
// path_bounds_decider_util.cc: TrimPathBounds
void PathBoundsDeciderUtil::TrimPathBounds(
    const int path_blocked_idx, PathBoundary* const path_boundaries) {
  if (path_blocked_idx != -1) {
    if (path_blocked_idx == 0) {
      AINFO << "Completely blocked. Cannot move at all.";
    }
    double front_edge_to_center =
        VehicleConfigHelper::GetConfig().vehicle_param().front_edge_to_center();
    double trimmed_s =
        path_boundaries->at(path_blocked_idx).s - front_edge_to_center;
    while (path_boundaries->size() > 1 &&
           path_boundaries->back().s > trimmed_s) {
      path_boundaries->pop_back(); // 逐个弹出BLOCKED位置之后的边界点
    }
  }
}
```

该函数将路径边界截断至 BLOCKED 位置的前方，即减去车头到中心的距离，后续的 `FallbackPath` 模块基于截断后的短路径生成减速停车轨迹。整个过程是不可逆的，一旦某个避让方向组合导致 BLOCKED，系统不会回溯尝试其他方向组合，而是直接进入停车流程。

Apollo 提供的 `LaneBorrowPath` 机制可以在一定条件下缓解 BLOCKED 问题，但该机制的触发条件非常严格。代码4.4展示了借道决策需要通过的 5 个前置检查及条件 4 的具体实现

代码 4.4 LaneBorrowPath 的前置条件与长期阻塞判定

```
// lane_borrow_path.cc: IsNecessaryToBorrowLane
// 条件1: 仅单参考线场景 (路口等多参考线场景不借道)
if (!HasSingleReferenceLine(*frame_)) { return false; }
// 条件2: 自车速度低于阈值 (默认5.0 m/s)
if (!IsWithinSidePassingSpeedADC(*frame_)) { return false; }
// 条件3: 阻塞障碍物远离路口
if (!IsBlockingObstacleFarFromIntersection(*ref_info)) { return false; }
// 条件4: 障碍物持续阻塞达到cycle阈值 (默认3个周期)
if (!IsLongTermBlockingObstacle()) { return false; }
// 条件5: 阻塞障碍物在目的地范围内
if (!IsBlockingObstacleWithinDestination(*ref_info)) { return false; }

// 条件4的具体实现: IsLongTermBlockingObstacle
bool LaneBorrowPath::IsLongTermBlockingObstacle() {
  if (injector_->planning_context()->planning_status().path_decider()
          .front_static_obstacle_cycle_counter() >=
      config_.long_term_blocking_obstacle_cycle_threshold()) { // 默认值: 3
    return true;
  }
  return false;
}
```

这意味着：即使存在可行的借道方案，系统也必须先经历至少 3 个规划周期，约 300 ms，的停车等待才会尝试借道；且借道仅在单参考线、低速、远离路口等条件同时满足时才被允许。在路口附近、高速行驶、或多参考线场景下，LaneBorrow 机制不会触发，BLOCKED 判定将直接导致停车。

4.4 ST 层路径-速度协调缺失

4.4.1 问题描述与源码定位

问题 Apollo 的路径决策模块在确定障碍物的避让方向时，完全排除了动态障碍物。所有非静态障碍物，包括行驶中的车辆、行走的行人、骑行的自行车等，不参与 PathBoundsDecider 的边界构建，而是被完全交给速度规划阶段的 ST 图处理。这意味着路径规划阶段看到的“世界”中只有静态障碍物，动态障碍物如同不存在。

源码定位 该问题的源码证据体现在 `IsWithinPathDeciderScopeObstacle` 函数中

代码 4.5 动态障碍物被路径决策排除

```
// path_bounds_decider_util.cc: IsWithinPathDeciderScopeObstacle
bool PathBoundsDeciderUtil::IsWithinPathDeciderScopeObstacle(
    const Obstacle& obstacle) {
  // 虚拟障碍物不参与路径决策
  if (obstacle.IsVirtual()) {
    return false;
  }
  // 已有IGNORE决策的障碍物跳过
  if (obstacle.HasLongitudinalDecision() && obstacle.HasLateralDecision()
      && obstacle.IsIgnore()) {
    return false;
  }
  // 非静态障碍物或速度超过阈值的障碍物直接排除
  if (!obstacle.IsStatic() ||
      obstacle.speed() > FLAGS_static_obstacle_speed_threshold) {
    return false; // 动态障碍物不参与路径决策
  }
  // TODO(jiacheng):
  // Some obstacles are not moving, but only because they are waiting
  // for red light (traffic rule) or because they are blocked by
  // others (social). These obstacles will almost certainly move in
  // the near future and we should not side-pass such obstacles.
  return true;
}
```

值得注意的是，Apollo 开发团队在该函数中留下了 `TODO` 注释，表明他们意识到当前的处理方式存在不足，但尚未实施改进。这个 `TODO` 注释本身就是该问题存在的直接证据。

4.4.2 影响分析

动态障碍物被排除在路径决策之外，产生以下影响。

影响一，路径与速度规划之间缺乏协调

在 Apollo 的路径-速度解耦架构中，如第三章 3.3 节所述，路径规划先于速度规划执行，速度规划在路径规划的结果上构建 ST 图。当动态障碍物不参与路径决策时，路径阶段可能生成一条直行路径，而速度阶段在该路径上发现动态障

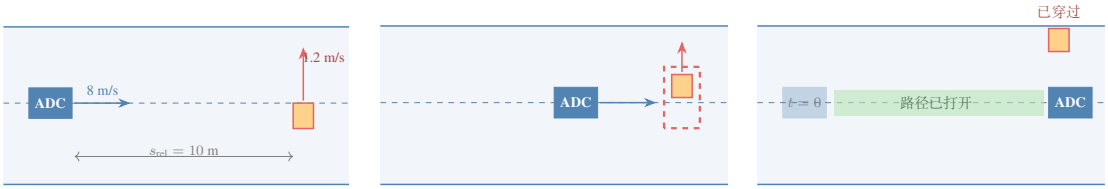
碍物横穿，不得不急刹或停车等待。如果路径阶段考虑了动态障碍物的预测轨迹，完全可以提前规划一条避让路径，在保持速度的同时安全通过。

影响二，横穿行人在路径规划中完全不存在

考虑一个典型场景：一名行人正在从右向左横穿道路。在路径规划阶段，由于该行人是动态障碍物（`IsStatic()==false`），PathBoundsDecider 完全忽略其存在，路径规划生成的是一条沿车道中心线的直行路径。随后速度规划在 ST 图中检测到该行人的 ST 边界，只能通过减速或停车来避让。

具体地，设自车以 $v_{ego} = 8\,\text{m/s}$ 行驶，前方 $s = 10\,\text{m}$ 处一名行人以 $v_{lat} = 1.2\,\text{m/s}$ 从右向左横穿，当前位于车道中央 $l = 0$ 处。自车到达该位置需 $t = 10/8 = 1.25\,\text{s}$，届时行人已横移 $1.2 \times 1.25 = 1.5\,\text{m}$，移至 $l = 1.5\,\text{m}$ 处，接近车道左边缘。如果路径规划阶段纳入行人的预测轨迹，可以基于行人在 $t = 1.25\,\text{s}$ 时的预测位置（$l = 1.5\,\text{m}$）计算路径边界，此时行人已不在车道中央，自车可从行人右侧安全通过，无需停车。但由于行人被 `IsWithinPathDeciderScopeObstacle` 过滤，路径阶段对此一无所知，速度阶段只能被迫停车等待。该场景的完整时变分析将在第五章中给出。

图4.5给出了该场景下行人时变位置与自车到达时刻的三帧对比。子图 (a) 为 $t = 0$ 时刻，自车刚开始规划，行人距自车 10 m，尚未进入车道，PathBoundsDecider 看到的“世界”中该行人不存在。子图 (b) 为 $t = 0.625\,\text{s}$ 时刻，自车已行进 5 m，行人横移 0.75 m 进入车道右半侧，恰好落在自车直行路径上形成碰撞窗口。子图 (c) 为 $t = 1.25\,\text{s}$ 时刻，行人已穿过车道抵达对侧，自车到达原障碍位置时路径已然通畅。若 Path 阶段能够利用行人的预测轨迹即“在自车到达该位置时行人已横移 1.5 m 至车道边缘”这一事实，完全可以在规划初始即选择从行人右侧绕行或保持直行，无需进入 Speed 阶段的急刹降级。



(a) $t = 0$ **行人未进入路径** **(b)** $t = 0.625\,\text{s}$ **行人半入路径** **(c)** $t = 1.25\,\text{s}$ **路径通畅**

图 4.5 动态障碍物排除后的时变演化示意

影响三，加剧问题一的影响

动态障碍物的排除使得 PathBoundsDecider 只能基于障碍物在 $t = 0$ 时刻的静态快照做决策，无法利用时间维度的信息。例如，一辆缓慢移动的车辆在当前

时刻 $t = 0$ 占据车道中央，但 $t = 2$s 后将移至车道边缘甚至离开车道。如果纳入该动态障碍物的预测轨迹，PathBoundsDecider 可以基于自车到达该 s 位置时障碍物的预测位置来决定避让方向，而非基于其当前位置。这种时间维度的利用可以显著减少问题一中 BLOCKED 判定的发生频率。

4.4.3 PathDecider 中的同一问题

不仅 PathBoundsDecider 排除动态障碍物，路径决策器 PathDecider 同样如此。

代码 4.6 PathDecider 同样排除动态障碍物

```
// path_decider.cc: MakeStaticObstacleDecision
for (const auto *obstacle : path_decision->obstacles().Items()) {
  // 跳过非静态障碍物和虚拟障碍物
  if (!obstacle->IsStatic() || obstacle->IsVirtual()) {
    continue;
  }
  // 已有IGNORE决策的障碍物跳过
  if (obstacle->HasLongitudinalDecision() &&
      obstacle->LongitudinalDecision().has_ignore() &&
      obstacle->HasLateralDecision() &&
      obstacle->LateralDecision().has_ignore()) {
    continue;
  }
  // ... 后续仅对静态障碍物执行STOP/NUDGE/IGNORE决策
}
```

可以看到，`MakeStaticObstacleDecision` 在遍历障碍物列表的第一步即通过 `IsStatic()` 过滤掉所有动态障碍物。这意味着 Apollo 路径规划阶段的两个核心决策模块，即负责构建横向边界的 PathBoundsDecider 和负责做出 STOP/NUDGE/IGNORE 决策的 PathDecider，都将动态障碍物完全排除在外，形成了系统性的路径-速度协调缺失。

4.4.4 路径决策信息未传递至速度规划

除了动态障碍物被排除外，ST 层面还存在一个更深层的问题，路径决策阶段产生的时空信息没有传递给速度规划阶段。

在 Apollo 的路径-速度解耦架构中，PathBoundsDecider 输出的是横向路径边界 $[l^-(s), l^+(s)]$，SpeedBoundsDecider 基于该路径独立构建 ST 边界。两个模块之间的信息传递是单向的、有限的，速度规划知道路径在哪里，但不知道路径为什么在那里。

这种信息断层的后果在动态障碍物场景中尤为明显。考虑改进后的路径规划，路径决策可能基于“行人将在 $t = 1.25$ s 后移开”这一预判选择了一条通过方案，但 SpeedBoundsDecider 不知道这一时间约束的存在。如果速度规划恰好让自车在行人移开之前到达该位置，路径决策的前提条件就被违反了。

因此，ST 层面的问题不仅是“排除了动态障碍物”，更本质的问题是路径规划和速度规划之间缺乏时空约束的传递通道。改进方案需要建立这样一个通道，路径层面的时变决策能够以约束的形式传递给速度层面，确保两者的一致性。图4.6示意了现有架构中路径阶段到速度阶段的单向信息流以及缺失的时空一致性反向通道。

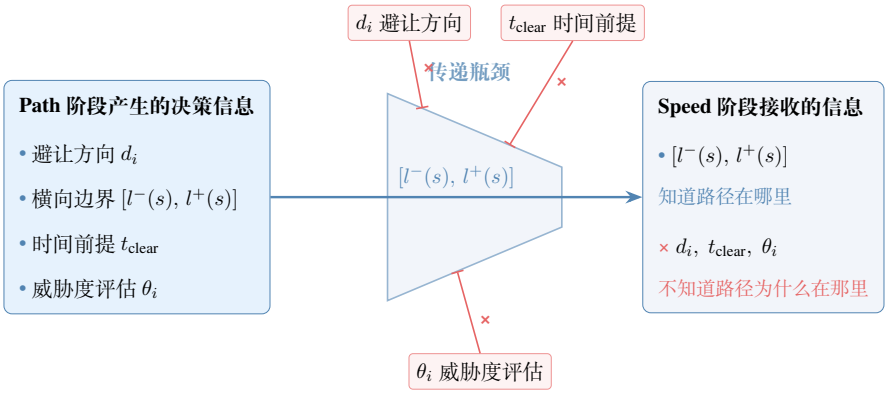


图 4.6 路径与速度规划之间的单向信息传递与时空约束缺口

图中漏斗表示两阶段之间的信息瓶颈。Path 阶段产生的避让方向 d_i、时间前提 t_{clear} 和威胁度 θ_i 均未传递至 Speed 阶段，仅横向边界 $[l^-(s), l^+(s)]$ 通过。

4.5 问题总结与改进方向

表4.1对比了三个问题的特征及其对应的改进方向。

表 4.1 多障碍物场景三个核心问题与改进方向

	问题一 安全裕度一刀切	问题二 独立决策	问题三 动态障碍物排除
涉及模块	PathBoundsDecider	PathBoundsDecider	PathBoundsDecider / PathDecider
根源	统一安全距离不区分威胁	逐障碍物贪心决策	动态障碍物不参与路径决策
表现	通行带被不必要压缩	路径 BLOCKED → 停车	错过时变可行方案
严重程度	严重，加剧问题二	致命	严重，加剧问题二
改进维度	差异化安全距离	全局协调决策	时空约束协调

表中“严重程度”一列反映了三个问题的相对权重，问题二是导致路径 BLOCKED 的直接瓶颈，问题一和问题三则分别从安全距离维度和时间维度提高问题二的发生概率。三个改进维度在数学上可纳入一个统一的优化框架，具体的问题建模与算法设计见第五章。

4.6 本章小结

本章分析了 Apollo 在多障碍物场景下的三个核心规划缺陷。问题一，所有障碍物使用统一的安全距离，不区分威胁程度，不必要地压缩通行空间。问题二，PathBoundsDecider 对每个障碍物独立决定避让方向，在道路两侧同时存在障碍物时产生矛盾决策，导致路径 BLOCKED。问题三，动态障碍物被完全排除在路径决策之外，Apollo 团队在源码中留下的 `TODO` 注释证实了这一缺陷，且路径-速度之间缺乏时空约束传递通道。三个问题分别从安全距离、决策粒度和时空协调三个维度指向相应的改进方向，具体的算法设计与实现见第五章。

5 改进算法设计与实现

本章详细阐述针对第四章所识别问题的改进算法设计。第四章从 SL 和 ST 两个层面揭示了 Apollo PathBoundsDecider 的核心缺陷，即 SL 层面的逐障碍物独立决策和安全裕度一刀切，ST 层面的动态障碍物排除和路径-速度协调缺失。本章沿同一双层框架组织改进方案，首先建立统一的数学模型并分析 Baseline 次优性，然后从 SL 层提出基于重叠分组的最优通行带算法，从 ST 层构建时空可通行走廊与速度规划协调机制，最后给出系统集成方案。

5.1 时变最优通行带问题建模

本节建立改进算法的数学框架，定义后续各节所需的核心符号，并在此基础上构建联合优化问题。

5.1.1 SL 空间中的障碍物表示

沿用第二章介绍的 Frenet 坐标系（SL 坐标系），纵向坐标 s 为沿参考线的弧长距离，横向坐标 l 为偏离参考线的偏移，左正右负。

对于第 i 个障碍物，其在 SL 空间中的投影为一个轴对齐矩形

$$O_i = [s_i^-, s_i^+] \times [l_i^-, l_i^+] \tag{5.1}$$

其中 $[s_i^-, s_i^+]$ 为障碍物沿参考线方向的纵向占据范围，$[l_i^-, l_i^+]$ 为横向占据范围。l_i^- 和 l_i^+ 分别为障碍物右侧边界和左侧边界的横向坐标，以参考线为原点，左正右负。

为保证路径规划的安全性，需要在障碍物边界外预留安全裕度。定义综合安全裕度为

$$\delta = \frac{W_{ego}}{2} + d_{buf} \tag{5.2}$$

其中 W_{ego} 为自车宽度，d_{buf} 为额外的缓冲距离，对应 Apollo 中 `obstacle_lat_buffer` 参数，默认取 0.4 m。

基于安全裕度，定义两个关键量

$$u_i = l_i^- - \delta \tag{5.3}$$

$$v_i = l_i^+ + \delta \tag{5.4}$$

u_i 表示从障碍物 O_i 右侧安全通过时，自车参考点的横向坐标上界；v_i 表示从左侧安全通过时的横向坐标下界。换言之，若自车从右侧绕行，则需满足 $l \le u_i$；若从左侧绕行，则需满足 $l \ge v_i$。两个量之间的关系为 $v_i - u_i = (l_i^+ + \delta) - (l_i^- - \delta) = (l_i^+ - l_i^-) + 2\delta$，即障碍物宽度加两倍安全裕度，这也是障碍物在横向空间中的“有效占据宽度”。

图5.1给出了上述定义的几何图景。障碍物 O_i 在 SL 空间中占据一个轴对齐矩形，横向延伸为 $[l_i^-, l_i^+]$，纵向延伸为 $[s_i^-, s_i^+]$。在矩形外围向左右各扩展一倍安全裕度 δ，形成膨胀后的“禁行矩形”。其下边界即 $u_i = l_i^- - \delta$，代表从右侧绕行时自车参考点横向坐标的上限；其上边界即 $v_i = l_i^+ + \delta$，代表从左侧绕行时的横向坐标下限。图中深蓝实线给出了“从右侧绕行”情形下的自车轨迹，该轨迹贴着 u_i 下方通过即可同时满足障碍物避让与道路边界约束。后续的最优通行带问题将在 u_i 和 v_i 构成的候选边界集合上做选择，因此这两个量的精确几何含义是全章推导的基石。

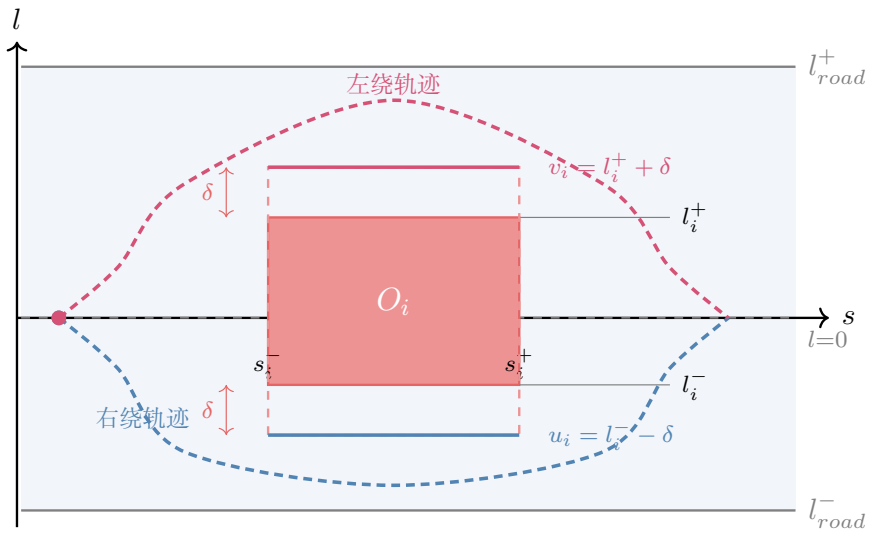


图 5.1 障碍物安全边界 u_i 与 v_i 的几何定义

5.1.2 避让决策与通行带

对于在某一纵向截面 s 处活跃的每个障碍物 O_i，即 $s \in [s_i^-, s_i^+]$，需要为其分配一个避让方向决策

$$d_i \in \{L, R\} \tag{5.5}$$

其中 $d_i = R$ 表示从障碍物右侧即右绕通过，$d_i = L$ 表示从左侧即左绕通过。每个决策对路径横向边界施加如下约束。

- $d_i = R$，自车应在障碍物右侧通过，构成上界约束 $l^+ \leq u_i$
- $d_i = L$，自车应在障碍物左侧通过，构成下界约束 $l^- \geq v_i$

给定一组决策 $\mathbf{d} = (d_1, d_2, \ldots, d_n)$，将被分配为右绕和左绕的障碍物分别记为集合 $\mathcal{R} = \{i \mid d_i = R\}$ 和 $\mathcal{L} = \{i \mid d_i = L\}$。在位置 s 处，综合所有约束后的路径横向边界为

$$l^+(\mathbf{d}) = \min\left(l_{road}^+,\ \min_{i \in \mathcal{R}} u_i\right) \tag{5.6}$$

$$l^-(\mathbf{d}) = \max\left(l_{road}^-,\ \max_{i \in \mathcal{L}} v_i\right) \tag{5.7}$$

其中 l_{road}^+ 和 l_{road}^- 分别为道路左侧和右侧边界的横向坐标。

由此定义通行带宽度

$$W(\mathbf{d}) = l^+(\mathbf{d}) - l^-(\mathbf{d}) \tag{5.8}$$

当 $W(\mathbf{d}) > 0$ 时，存在宽度为 W 的横向可通行空间；当 $W(\mathbf{d}) \leq 0$ 时，路径被阻塞（BLOCKED），系统需要截断路径并触发降级策略。通行带越宽，路径优化器 `PiecewiseJerkPathOptimizer` 的横向自由度越大，输出路径的平滑性和舒适性越好。

从几何视角看，通行带 $[l^-(\mathbf{d}), l^+(\mathbf{d})]$ 定义了自车参考点在横向上的可行区间。路径优化器在该区间内寻找最优的横向偏移曲线 $l(s)$，使其满足连续性、曲率和与参考线的偏差等约束。因此，通行带的构建是路径规划的关键前置步骤，通行带的质量直接决定了后续优化问题的可行域和最优解质量。

5.1.3 联合优化问题

在真实场景中，障碍物可能处于运动状态。为此引入到达时间函数 $\tau(s)$，表示自车到达纵向位置 s 的预估时刻。此时，障碍物的 SL 投影变为时间的函数

$$O_i(\tau(s)) = [s_i^-(\tau(s)),\ s_i^+(\tau(s))] \times [l_i^-(\tau(s)),\ l_i^+(\tau(s))] \tag{5.9}$$

相应地，u_i 和 v_i 也成为位置 s 的函数

$$u_i(s) = l_i^-(\tau(s)) - \delta, \quad v_i(s) = l_i^+(\tau(s)) + \delta \tag{5.10}$$

这意味着障碍物对路径边界的约束不再是固定的，而是随自车到达时间而变化。一个障碍物在 $t = 0$ 时可能占据车道中心，但在 $\tau(s)$ 时刻可能已移至道路边缘甚至完全离开参考线范围。这一时变特性是本文方法区别于 Baseline 的核心所在。

路径规划的目标是找到最优的避让决策 $\mathbf{d}$ 和速度规划 τ，使得全路径的最小通行带宽度最大化

$$(\mathbf{d}^*, \tau^*) = \arg\max_{\mathbf{d},\,\tau}\ \min_{s}\ \left[l^+(s, \mathbf{d}, \tau) - l^-(s, \mathbf{d}, \tau)\right] \tag{5.11}$$

该联合优化问题的搜索空间为 2^n，即 n 个障碍物各有两种决策，属于 NP-hard 问题，暴力枚举不可行。例如，当 $n = 10$ 时，搜索空间为 $2^{10} = 1024$；当 $n = 20$ 时，搜索空间达到 $2^{20} \approx 10^6$。在 Apollo 的 100 ms 规划周期约束下，暴力搜索显然不可接受。

然而，该问题具有特殊的几何结构，决策变量 d_i 只影响一维横向空间中的边界位置，且影响方式是单调的，一个方向收缩上界，另一个方向收缩下界。下文将利用这一结构，通过引理和定理的推导，将求解复杂度降至 $O(n \log n)$。

5.2 Apollo Baseline 的次优性分析

5.2.1 Baseline 的贪心策略

Apollo 通过 `UpdatePathBoundaryBySLPolygon` 函数逐障碍物构建路径边界。该函数在路径规划的每个离散位置 s 处被调用，是 PathBoundsDecider 中处理障碍物避让的核心入口。其核心逻辑可形式化描述如下。

初始化路径边界为道路边界，$l^+(s) = l_{road}^+(s)$，$l^-(s) = l_{road}^-(s)$。

对于每个在位置 s 处活跃的障碍物 O_i，独立决定避让方向

$$d_i = \begin{cases} R & \text{若 } \dfrac{l_i^- + l_i^+}{2} > l_{last_nudge} \\ \\ L & \text{其他} \end{cases} \tag{5.12}$$

其中 l_{last_nudge} 为上一次 nudge 操作记录的横向位置参考值。若仅一侧可通行，则该障碍物的避让方向直接赋为可通行侧，与 4.3.2 节代码所示逻辑一致。然后更新边界。

- $d_i = R$，$l^+(s) \leftarrow \min(l^+(s),\ u_i)$
- $d_i = L$，$l^-(s) \leftarrow \max(l^-(s),\ v_i)$

若更新后 $l^+(s) < l^-(s)$，则判定该位置 BLOCKED，调用 `TrimPathBounds` 截断路径，最终触发 FallbackPath 停车。

该策略的关键特征是贪心性和局部性。所谓贪心性，是指对第 i 个障碍物的决策仅基于该障碍物自身的位置信息，不考虑已做出的其他决策是否与之矛盾。所谓局部性，是指决策仅使用当前时刻的空间快照，不利用 Prediction 模块提供的未来轨迹预测。

具体而言，该策略存在两方面的次优性。其一，避让方向的决策仅依赖障碍物中心与参考值的相对位置，逐个独立进行，未考虑多个障碍物的全局协调；其二，障碍物投影仅使用 $t = 0$ 时刻的快照，动态障碍物通过 `IsWithinPathDeciderScopeObstacle` 被直接排除在路径决策之外。Apollo 的开发团队在该过滤函数的源码中留有 `TODO` 注释，明确提到应当在未来版本中将动态障碍物纳入路径决策，这从侧面印证了该设计的局限性。图5.2给出了该贪心流程的完整控制流，从初始化路径边界到逐障碍物决策，再到 BLOCKED 判定与回退停车的全链路。

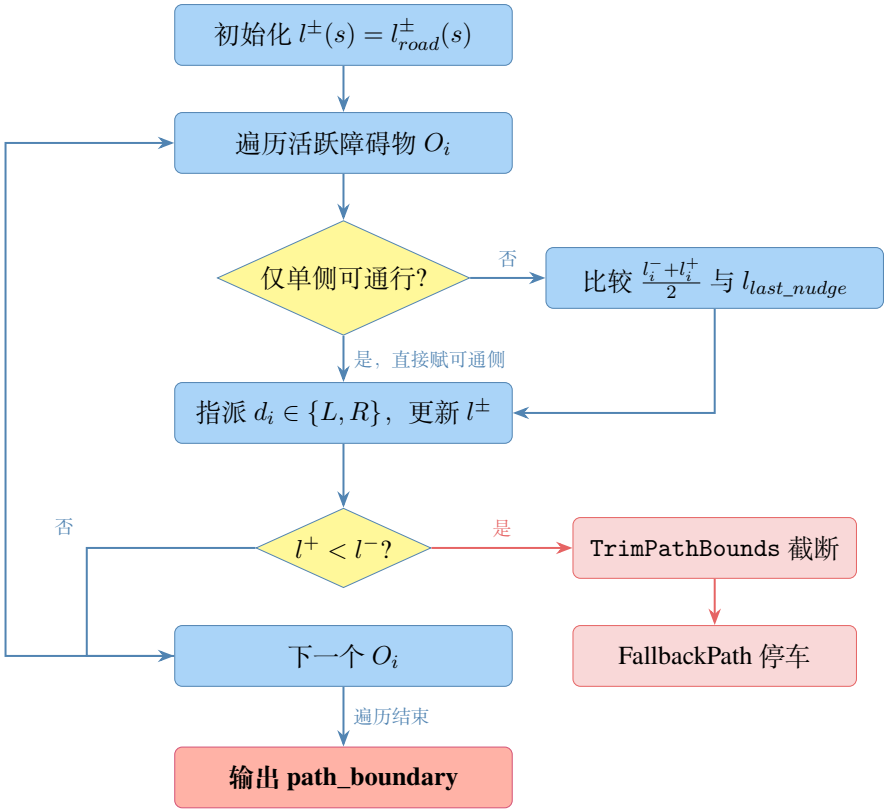


图 5.2 Apollo Baseline 逐障碍物贪心决策的控制流

5.2.2 空间维度的次优性

空间维度的次优性集中表现为矛盾决策现象。以下用一个对称场景说明。

场景设定 双车道道路总宽 7.5 m，$l^-_{road} = -3.75\,\mathrm{m}$，$l^+_{road} = 3.75\,\mathrm{m}$。在某纵向截面 s^* 处同时存在两个路锥，关于参考线近似对称

- O_a 偏右，横向占据 $[0.1,\ 0.4]\,\mathrm{m}$，中心 $0.25\,\mathrm{m}$
- O_b 偏左，横向占据 $[-0.4,\ -0.1]\,\mathrm{m}$，中心 $-0.25\,\mathrm{m}$

安全裕度 $\delta = 1.2\,\mathrm{m}$，对应 $u_a = 0.1 - 1.2 = -1.1\,\mathrm{m}$，$v_a = 0.4 + 1.2 = 1.6\,\mathrm{m}$，$u_b = -0.4 - 1.2 = -1.6\,\mathrm{m}$，$v_b = -0.1 + 1.2 = 1.1\,\mathrm{m}$。

Baseline 贪心决策 O_a 中心 $0.25 > 0$，分配 $d_a = R$；O_b 中心 $-0.25 < 0$，分配 $d_b = L$。路径边界从两侧同时收缩

$$l^+(s^*) = u_a = -1.1\,\mathrm{m}, \quad l^-(s^*) = v_b = 1.1\,\mathrm{m} \tag{5.13}$$

$l^+ < l^-$，判定 BLOCKED。贪心策略将自车挤入两个路锥之间仅 0.2 m 的间隙，路径不可行。

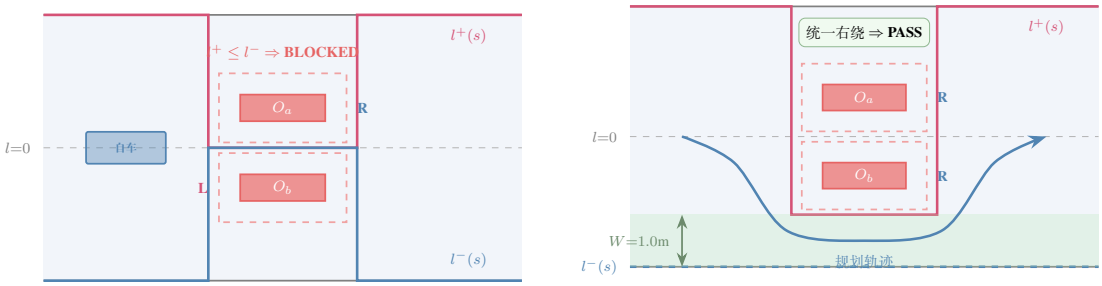
全局最优决策 按 u_i 升序排列为 O_b, O_a，此时 $u_b = -1.6 < u_a = -1.1$，计算 3 个间隙

$$
\begin{aligned}
g_0 &= u_b - l_{road}^- = -1.6 - (-3.75) = 2.15\,\mathrm{m} \\
g_1 &= u_a - v_b = -1.1 - 1.1 = -2.2\,\mathrm{m} \quad \text{（负值，中间通道不可行）} \\
g_2 &= l_{road}^+ - v_a = 3.75 - 1.6 = 2.15\,\mathrm{m}
\end{aligned}
$$

最大间隙 $g_0 = g_2 = 2.15\,\mathrm{m}$，对应分割点 $p^* = 0$ 即统一 RIGHT_NUDGE 或 $p^* = 2$ 即统一 LEFT_NUDGE。以 $p^* = 0$ 为例，通行带为 $[l_{road}^-,\ u_b] = [-3.75,\ -1.6]\,\mathrm{m}$，宽度 2.15 m，自车可借用相邻车道安全通过。

贪心策略在 $2^2 = 4$ 种方向组合中恰好选中了唯一不可行的一种（R, L），而全局搜索只需检查 $k + 1 = 3$ 个间隙即可找到最优解。当所有间隙均为负值时，说明本车道确实不存在可通行路径，此时算法输出的 BLOCKED 是真实的而非贪心策略导致的虚假阻塞，体现了算法的完备性。

图5.3直观对比了贪心决策与全局协调在该场景下的差异。



(a) 贪心独立决策 → BLOCKED **(b) 全局协调统一右绕 → 通行**

图 5.3 贪心独立决策与全局协调决策的通行带对比

5.2.3 时间维度的次优性

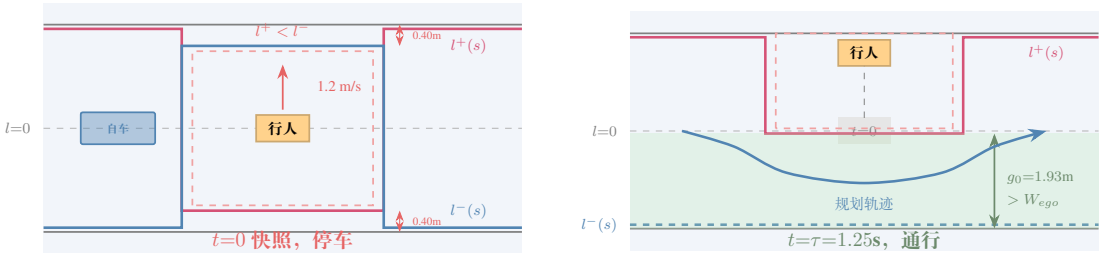
Baseline 的另一个局限在于仅使用 $t = 0$ 时刻的障碍物位置快照构建路径边界，完全忽略了动态障碍物的运动趋势。考虑一个典型场景，一名行人正在自车前方 10 m 处横穿马路，当前位于车道中央处于 $l = 0\,\mathrm{m}$，以 1.2 m/s 的速度向左移动。在 $t = 0$ 时刻，该行人占据道路中心，间隙为零。Baseline 将其排除在路径决策之外，但仍会在 ST 图中标记为障碍物，导致速度规划层进行不必要的减速甚至停车。

然而，自车以 8 m/s 的速度行驶，到达行人所在位置需要 $\tau = 10/8 = 1.25\,\mathrm{s}$。

此时行人已移动 $1.2 \times 1.25 = 1.5\,\mathrm{m}$ 至道路左侧 $l = 1.5\,\mathrm{m}$ 处，车道中心的间隙已打开。定量分析如下。行人宽度约 $0.5\,\mathrm{m}$，在 $\tau = 1.25\,\mathrm{s}$ 时其横向占据区间为 $[1.25, 1.75]\,\mathrm{m}$，对应 $u_{ped}(\tau) = 1.25 - 1.2 = 0.05\,\mathrm{m}$。

道路右边界间隙 $g_0 = u_{ped} - l^-_{road} = 0.05 - (-1.875) = 1.925\,\mathrm{m}$，远大于自车宽度 $1.8\,\mathrm{m}$，自车可以安全通过。

时变通行带方法通过预测轨迹自然捕获这一信息，避免不必要的停车。图5.4直观对比了两种方法在该场景下的差异。



(a) Baseline 仅看 t=0 **(b) 时变方法利用预测轨迹**

图 5.4 Baseline 静态快照与时变方法的通行带对比

此外，Baseline 还存在安全裕度维度的次优性，所有参与路径决策的障碍物使用统一的安全裕度 δ。然而，不同障碍物对自车的威胁程度差异显著，一个远离车道的路锥与一辆正对车道的违停车所需的安全裕度不应相同。统一的 δ 导致低威胁障碍物占用了过多的横向空间，使通行带被不必要地压缩。

上述三类次优性的共同根源在于 Baseline 缺乏全局视角和分级机制，空间维度上对各障碍物独立决策，时间维度上仅使用静态快照，安全裕度维度上一刀切处理。本文提出的改进方法分别从分组即全局最优决策、时变即预测轨迹利用和分级即差异化安全裕度三个维度同时解决上述问题。

5.3 多因子威胁度评估与动态安全距离

上一节识别了 Baseline 在安全裕度维度的次优性，式(5.2)中的统一 δ 不区分障碍物威胁程度，导致低威胁障碍物过度压缩通行带。本节建立量化模型，为每个障碍物计算综合威胁度 Θ_i，并据此确定差异化的安全裕度 δ_i。

5.3.1 威胁度计算模型

对第 i 个障碍物，其综合威胁度 $\Theta_i \in [0, 1]$ 由五个因子加权求和得到

$$\Theta_i = w_1 \cdot f_{TTC}(i) + w_2 \cdot f_{overlap}(i) + w_3 \cdot f_{vel}(i) + w_4 \cdot f_{type}(i) + w_5 \cdot f_{inter}(i) \quad (5.14)$$

权重默认取值为 $w_1 = 0.30$，$w_2 = 0.20$，$w_3 = 0.15$，$w_4 = 0.10$，$w_5 = 0.25$。上述权重通过以下方式确定。首先基于各因子对多障碍物场景区分度的定性分析确定初始权重，然后通过对称路锥、横穿行人等典型场景的定性观察调整至最终取值。TTC 因子和交互密度因子对场景风险的区分度最高，因此赋予较大权重，障碍物类型在同一场景中变化较小，赋予较小权重。各因子的取值范围均归一化到 $[0, 1]$，定义如下。

碰撞时间因子 f_{TTC}

碰撞时间（Time-to-Collision, TTC）是衡量碰撞紧迫性的经典指标

$$TTC_i = \frac{s_i - s_{ego}}{\dot{s}_{ego} - \dot{s}_i} \quad (5.15)$$

其中 s_i 为障碍物纵向位置，$\dot{s}$ 为纵向速度。当 $\dot{s}_{ego} \leq \dot{s}_i$ 即自车不会追上障碍物时，$TTC_i = +\infty$。TTC 因子通过分段线性函数映射至 $[0, 1]$

$$f_{TTC}(i) = \begin{cases} 1.0 & TTC_i \leq T_{crit} \\ \dfrac{T_{max} - TTC_i}{T_{max} - T_{crit}} & T_{crit} < TTC_i < T_{max} \\ 0 & TTC_i \geq T_{max} \end{cases} \quad (5.16)$$

其中 $T_{crit} = 2\,\mathrm{s}$ 为临界碰撞时间，$T_{max} = 7\,\mathrm{s}$ 为最大关注时间，与 Apollo ST 图的时间范围一致。$TTC_i \leq 2\,\mathrm{s}$ 意味着碰撞迫在眉睫，威胁最大；$TTC_i \geq 7\,\mathrm{s}$ 的障碍物已超出当前规划周期的关注范围。

横向重叠率因子 $f_{overlap}$

横向重叠率衡量障碍物与自车行驶走廊在横向上的重叠程度

$$f_{overlap}(i) = \frac{\max\big(0,\ \min(l_i^+, l_{ego}^+) - \max(l_i^-, l_{ego}^-)\big)}{W_{ego}} \quad (5.17)$$

其中 $l_{ego}^+ = l_{ego} + W_{ego}/2$、$l_{ego}^- = l_{ego} - W_{ego}/2$ 为自车走廊的横向边界。

$f_{overlap} = 0$ 表示完全不重叠即障碍物在自车投影之外，$f_{overlap} = 1$ 表示完全重叠即障碍物正对自车。该因子直接反映了碰撞的“几何可能性”，横向偏移较大的障碍物即使 TTC 较小，实际碰撞风险也低。

相对速度因子 f_{vel}

相对速度因子反映障碍物与自车的接近速度

$$f_{vel}(i) = \sigma\left(\frac{v_{rel,i}}{v_{max}}\right), \quad \sigma(x) = \frac{1}{1+e^{-5x}} \tag{5.18}$$

其中 $v_{rel,i} = \dot{s}_{ego} - \dot{s}_i$ 为相对纵向速度，正值表示自车接近障碍物，v_{max} 为道路限速。Sigmoid 函数将相对速度平滑映射至 $(0,1)$，快速接近的障碍物即 $v_{rel} \gg 0$ 时趋近 1，同向远离的障碍物即 $v_{rel} < 0$ 时趋近 0。选择 Sigmoid 而非线性映射的原因在于，相对速度为零附近的区分度更重要，即低速接近和低速远离的差异，而极高相对速度间的差异不需要精细区分。

障碍物类型因子 f_{type}

取值映射 不同类型障碍物的运动不确定性与受伤严重度差异显著，按下式取离散值

$$f_{type}(i) = \begin{cases} 1.0 & \text{行人、自行车等弱势道路使用者} \\ 0.7 & \text{机动车，运动相对可预测} \\ 0.5 & \text{未知可动目标，按保守权重处理} \\ 0.3 & \text{静态障碍物，仅需基本缓冲} \end{cases} \tag{5.19}$$

取值依据 该映射由运动不确定性与事故后果两方面共同确定。一方面，行人和自行车的运动方向和速度随时可能突变，如突然横穿或急停回转，其轨迹可预测性远低于机动车，需要最大安全裕度；机动车受道路与交通规则约束，轨迹相对可预测；静态障碍物无运动不确定性，仅保留基本缓冲。另一方面，交通事故统计表明弱势道路使用者（Vulnerable Road Users, VRU）在同等碰撞能量下受伤严重度和致死率远高于车内乘员，规划器应对其施加额外保护性裕度，因此赋予行人和自行车最高权重 1.0。

Apollo 感知映射 Apollo 感知模块在 `PerceptionObstacle` 消息中提供枚举字段 `type`，取值覆盖 `UNKNOWN`、`UNKNOWN_MOVABLE`、`UNKNOWN_UNMOVABLE`、P

EDESTRIAN、BICYCLE 和 VEHICLE 六类。规划层直接查表即可完成从感知标签到 f_{type} 的映射，PEDESTRIAN 和 BICYCLE 归入 1.0 分支，VEHICLE 归入 0.7 分支，UNKNOWN_MOVABLE 与 UNKNOWN 归入 0.5 分支，UNKNOWN_UNMOVABLE 归入 0.3 分支。对未知可动目标采用中等偏上的保守权重是因为分类置信度不足时宁可过估威胁也不可低估，避免将潜在行人误判为低威胁物体。

合成方式 f_{type} 作为乘性类别因子进入式(5.14)，与碰撞时间、横向重叠率、相对速度三项连续因子加权叠加，合成综合威胁度 Θ_i，并经式(5.21)映射为差异化安全裕度 δ_i，实现对 VRU 的主动倾斜保护。

交互密度因子 f_{inter}

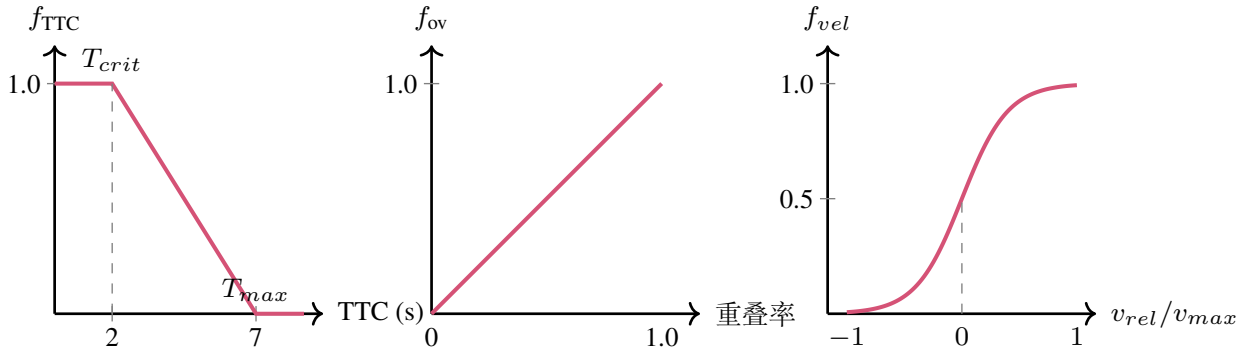
交互密度因子量化障碍物 i 与周围其他障碍物的聚集程度，是本文威胁度模型中与分组策略直接相关的因子

$$f_{inter}(i) = \frac{1}{N-1} \sum_{j \neq i} \mathbb{1}[d_{ij} < d_{cluster}] \cdot \frac{d_{cluster} - d_{ij}}{d_{cluster}} \tag{5.20}$$

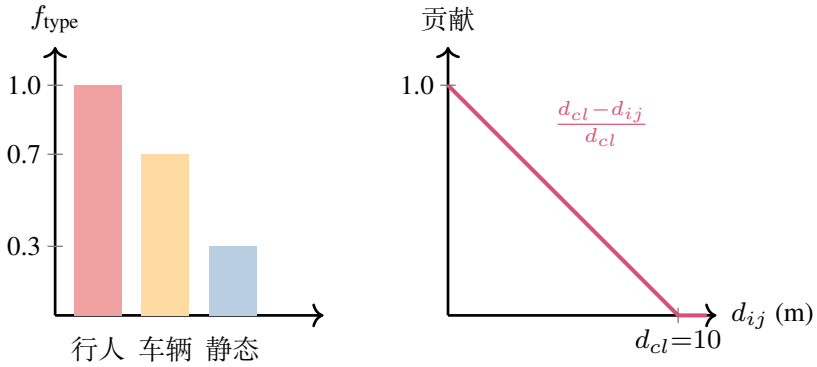
其中 N 为障碍物总数，$d_{ij} = \sqrt{(s_i - s_j)^2 + (l_i - l_j)^2}$ 为障碍物 i 和 j 在 SL 空间中的欧氏距离，$d_{cluster} = 10\,\mathrm{m}$ 为聚类半径，$\mathbb{1}[\cdot]$ 为指示函数。f_{inter} 的物理含义是，若障碍物 i 周围有大量其他障碍物聚集，则该因子值高，表明局部交互密度大，需要更谨慎的避让裕度。孤立障碍物的 $f_{inter} \approx 0$，密集车群中的障碍物 f_{inter} 接近 1。

引入交互密度因子的意义在于它能区分“3 个分散的障碍物”和“3 个聚集的障碍物”这两种对避让策略要求截然不同的场景，前者各自独立，安全裕度可适当放宽，后者构成密集障碍物组，留给自车的横向空间本就有限，需要更精确的安全边界。这一因子与下一节的纵向重叠分组形成自然呼应，f_{inter} 高的障碍物大概率属于同一连通分量，在分组内被统一协调。

图5.5以子图形式给出了各因子的函数形态。f_{TTC} 在 T_{crit} 以下取饱和值 1，在 $T_{\max}$ 以上回到 0，中间为线性过渡；f_{overlap} 随重叠率线性增长；f_{vel} 采用 Sigmoid 函数，对零相对速度附近的变化最为敏感；f_{type} 为离散取值，反映不同障碍物类型的固有风险差异；f_{inter} 中单个邻居的贡献随距离线性衰减至聚类半径 $d_{cluster}$ 处归零。各因子均归一化到 $[0, 1]$，权重之和为 1，综合威胁度 Θ_i 也落在 $[0, 1]$ 区间，可直接作为式(5.21)中安全裕度线性插值的参数。



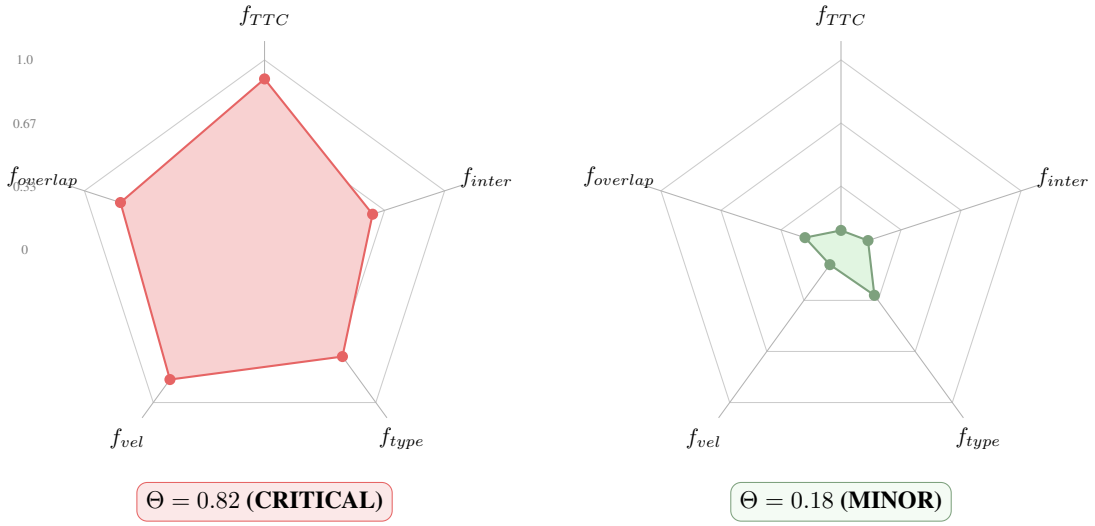
(a) 碰撞时间因子 **(b) 横向重叠率因子** **(c) 相对速度因子**



(d) 障碍物类型因子 **(e) 单邻居交互贡献**

图 5.5 多因子威胁度评估的五个因子函数形态

图5.6进一步给出了两个典型场景下的五维威胁度雷达图对照。左侧为对向来车场景，TTC 较小、横向重叠率高、相对速度大、对象为车辆、交互密度中等，五维雷达图呈现大面积覆盖，综合威胁度接近 0.85，对应 δ_i 取到 $\delta_{\max}$ 附近，安全裕度最大。右侧为静止路锥场景，TTC 趋于无穷、重叠率低、相对速度为零、类型权重低、密度因子接近 0，雷达图呈现小面积覆盖，综合威胁度不足 0.2，对应 δ_i 接近 $\delta_{\min}$，从而大幅释放横向空间。这两张雷达图直观说明了差异化安全裕度的判别逻辑与最终数值效应。



(a) 高威胁　对向来车　　**(b) 低威胁　远处静态路锥**

图 5.6 典型场景下的威胁度雷达图对比

5.3.2　动态安全距离

基于综合威胁度 Θ_i，将式(5.2)中的统一安全裕度替换为差异化版本

$$\delta_i = \delta_{min} + (\delta_{max} - \delta_{min}) \cdot \Theta_i \tag{5.21}$$

其中 $\delta_{max} = W_{ego}/2 + d_{buf,max}$ 为最大安全裕度，对应 CRITICAL 等级障碍物，$\delta_{min} = W_{ego}/2 + d_{buf,min}$ 为最小安全裕度，对应 MINOR 等级障碍物。默认参数取 $d_{buf,max} = 0.5\,\mathrm{m}$ 与 Apollo 原始 `point_extension` 一致、$d_{buf,min} = 0.15\,\mathrm{m}$，在 $W_{ego} = 1.8\,\mathrm{m}$ 时，$\delta_{max} = 1.4\,\mathrm{m}$，$\delta_{min} = 1.05\,\mathrm{m}$。

式(5.21)的设计确保了以下性质。CRITICAL 障碍物时 $\Theta_i \to 1$，获得与 Apollo 原始方案相同或更大的安全裕度，不降低安全性；MINOR 障碍物时 $\Theta_i \to 0$，获得较小但仍安全的裕度，释放被不必要占用的横向空间。

相应地，式(5.3)–(5.4)中的关键量更新为差异化版本

$$u_i = l_i^- - \delta_i \tag{5.22}$$

$$v_i = l_i^+ + \delta_i \tag{5.23}$$

当 Θ_i 对所有障碍物均取相同值时，式(5.22)–(5.23)退化为式(5.3)–(5.4)。动态 δ_i 的引入不影响后续引理 1 和定理 1 的成立，两者的证明仅依赖 u 和 v 值的排序

关系，只需按 u_i 而非 l_i^- 排序即可。

5.4 SL 层基于重叠分组的最优通行带

本节中所有 u_i 和 v_i 均使用式(5.22)–(5.23)中的动态安全距离 δ_i 计算。后续引理和定理的证明不依赖 δ 是否为常数，仅依赖 u 和 v 值的排序性。

5.4.1 纵向重叠关系与连通分量

在全路径范围内，不同纵向位置 s 处的活跃障碍物集合不同，因此式(5.11)的优化不能在单一截面上完成。关键观察是，如果两组障碍物在纵向上完全不重叠，则它们的避让决策相互独立。

定义纵向重叠关系。两个障碍物 O_i 和 O_j 纵向重叠，当且仅当它们的纵向区间有交集

$$O_i \sim O_j \iff [s_i^-, s_i^+] \cap [s_j^-, s_j^+] \neq \varnothing \tag{5.24}$$

该重叠关系是对称的但不具有传递性。取其传递闭包，即存在一系列中间障碍物依次纵向重叠，将 n 个障碍物划分为 m 个连通分量 $C_1, C_2, \ldots, C_m$。同一连通分量内的障碍物通过重叠链相连，不同连通分量之间在纵向上完全分离。由于不同连通分量在纵向上不重叠，其避让决策互不影响，可以独立求解。

连通分量的检测采用扫描线算法，将所有障碍物按 s_i^- 升序排列，维护一个变量 s_{max} 记录当前连通分量中所有障碍物的最大 s^+ 值。扫描过程中，若下一个障碍物的 s^- 不超过 s_{max}，则将其归入当前分量并更新 s_{max}；否则开始新的分量。该过程的时间复杂度为 $O(n \log n)$ 其中排序占主导，扫描本身仅需 $O(n)$。

分组的直观含义是，纵向上彼此重叠的障碍物需要在同一个横向截面上协调决策，因此必须归为同一组。而纵向上完全分离的障碍物组，自车在通过一组时尚未遇到下一组，决策互不影响。这一性质使得问题天然地分解为多个独立子问题，是算法高效性的基础。图5.7以六障碍物算例展示了扫描过程，s_{max} 扫描线沿 s 轴从左到右推进，遇到与当前组重叠的障碍物即并入，否则切出新分量，最终得到三个互不重叠的连通分量 C_1 到 C_3。

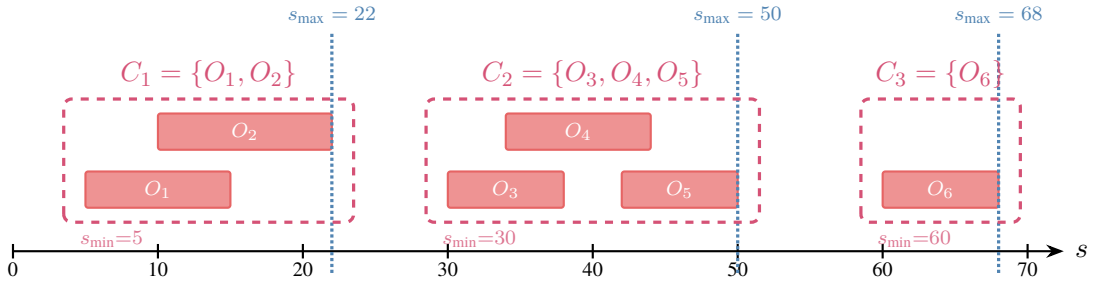


图 5.7 扫描线算法按 s_i^- 升序合并连通分量的过程

值得注意的是，当障碍物仅在纵向上部分重叠即存在某些 s 截面上只有部分障碍物活跃时，组内处理采用保守近似，以所有障碍物均同时活跃为假设计算间隙。这可能导致轻微的次优性，但算法的正确性可由以下两步论证保证。首先，在最密集截面上按定理 1 选定的分割点 p^* 将障碍物划分为左绕集合 $\mathcal{L}_{p^*}$ 和右绕集合 $\mathcal{R}_{p^*}$，此时通行带上下界由 $l^+ = \min_{i\in\mathcal{R}_{p^*}} u_i$ 和 $l^- = \max_{i\in\mathcal{L}_{p^*}} v_i$ 给出。其次，任意稀疏截面对应的活跃障碍物集合均为最密集截面的子集，移除部分障碍物只会使 min 运算作用于更大的集合从而放宽 l^+ 的上界，同时使 max 运算作用于更大的集合从而放宽 l^- 的下界。若 $\mathcal{L}_{p^*}$ 或 $\mathcal{R}_{p^*}$ 被移空，则相应方向退化为道路边界。因此通行带宽度 $W = l^+ - l^-$ 单调不减，最密集截面上的决策在所有稀疏截面上均保持可行，且通行带不会更窄。

全路径的最小通行带宽度由最窄的连通分量决定，即木桶效应

$$W^* = \min_{j=1,\dots,m} W^*(C_j) \tag{5.25}$$

下面聚焦于单个连通分量的最优求解。设该分量内有 k 个障碍物，在其公共 s 截面上按 u_i 值升序排列为 $O_{(1)}, O_{(2)}, \dots, O_{(k)}$，使得 $u_{(1)} \le u_{(2)} \le \cdots \le u_{(k)}$。按 u_i 排序而非 l_i^- 排序的原因在于引入动态安全距离 δ_i 后，$u_i = l_i^- - \delta_i$ 的排序可能不再与 l_i^- 一致，详见 5.3.3 节，而后续引理和定理的证明仅依赖 u 值的排序性。问题转化为在 k 个按 u_i 排列的障碍物上，如何分配左绕/右绕决策以最大化通行带宽度。

从几何直觉看，k 个按横向位置排列的障碍物将车道的横向空间分割为 $k+1$ 个通道，包括最右侧障碍物与道路右边界之间、相邻障碍物之间、以及最左侧障碍物与道路左边界之间的间隙。最优策略显然是选择其中最宽的通道通行，这自然地将通道一侧的所有障碍物划归左绕、另一侧的划归右绕，形成一个在排序中连续的分割。换言之，搜索空间从 2^k 个任意分配降为 $k+1$ 个候选通道的选取。下面通过引理和定理严格验证这一直觉，并建立通行带宽度与间隙之间的精确

等价关系。

5.4.2 最优分割连续性引理

引理 1（最优分割的连续性） 按 u_i 值升序排列后，存在最优解 $\mathbf{d}^*$，使得分割在排序中是连续的，即存在 $p \in \{0, 1, \ldots, k\}$，满足

$$d^*_{(i)} = L,\ \forall\, i \le p; \quad d^*_{(i)} = R,\ \forall\, i > p \tag{5.26}$$

证明 反证法。设某个最优解 $\mathbf{d}^*$ 中存在排序位置 $a < b < c$，使得 $d_{(a)} = R$，$d_{(b)} = L$，$d_{(c)} = R$。构造新解 $\mathbf{d}'$，将 $d_{(b)}$ 从 L 改为 R，其余不变。

分析上界变化，令 $\mathcal{R}' = \mathcal{R} \cup \{(b)\}$。由于 $(a) \in \mathcal{R}$ 且按排序有 $u_{(a)} \le u_{(b)}$，故

$$l^+(\mathbf{d}^*) \le u_{(a)} \le u_{(b)} \tag{5.27}$$

因此 $\min_{i \in \mathcal{R}'} u_i = \min_{i \in \mathcal{R}} u_i$，即 $l^+(\mathbf{d}') = l^+(\mathbf{d}^*)$，上界不变。

分析下界变化，令 $\mathcal{L}' = \mathcal{L} \setminus \{(b)\}$。集合缩小，取 max 的结果不增，故下界不升。若 $\mathcal{L}' = \varnothing$，按约定 max 取道路左边界形成的平凡下界 l^-_{road}，下界不升的结论仍成立。

$$l^-(\mathbf{d}') = \max_{i \in \mathcal{L}'} v_i \le \max_{i \in \mathcal{L}} v_i = l^-(\mathbf{d}^*) \tag{5.28}$$

综合两方面，$W(\mathbf{d}') = l^+(\mathbf{d}') - l^-(\mathbf{d}') \ge l^+(\mathbf{d}^*) - l^-(\mathbf{d}^*) = W(\mathbf{d}^*)$。因此 $\mathbf{d}'$ 也是最优解，且消除了 $(a), (b), (c)$ 处的一个不连续点。对所有不连续点重复此操作，有限次后即可得到满足式(5.26)的连续分割。

推论 搜索空间从 2^k 降为 $k+1$ 个候选分割点 $p = 0, 1, \ldots, k$。其中 $p = 0$ 表示所有障碍物均从右侧绕行即自车走最右侧通道，$p = k$ 表示所有障碍物均从左侧绕行即自车走最左侧通道，$1 \le p \le k-1$ 表示自车从第 p 和第 $p+1$ 个障碍物之间的间隙通过。

5.4.3 最大间隙最优性定理

基于引理 1，只需枚举 $k+1$ 个候选分割点。对于分割点 p，定义左绕集合 $\mathcal{L}_p = \{(1), \ldots, (p)\}$ 和右绕集合 $\mathcal{R}_p = \{(p+1), \ldots, (k)\}$。

定义 $k+1$ 个间隙，包含道路边界形成的两个端部间隙

$$g_p = \begin{cases} u_{(1)} - l_{road}^- & p = 0 \\ u_{(p+1)} - v_{(p)} & p = 1, 2, \ldots, k-1 \\ l_{road}^+ - v_{(k)} & p = k \end{cases} \tag{5.29}$$

g_0 表示道路右边界到第一个即最右侧障碍物之间的间隙，g_k 表示最后一个即最左侧障碍物到道路左边界之间的间隙，g_p，其中 $1 \le p \le k-1$，表示排序中相邻障碍物之间的间隙。

定理 1（最大间隙最优性） 对于分割点 p，通行带宽度 $W(p) = g_p$。因此最优通行带宽度为

$$W^* = \max_{p=0,1,\ldots,k} g_p \tag{5.30}$$

证明 对各分割点 p 直接代入验证。

当 $p = 0$ 即所有障碍物均为右绕时，$\mathcal{L}_0 = \varnothing$，$\mathcal{R}_0 = \{(1), \ldots, (k)\}$

$$l^+(0) = \min(l_{road}^+,\ u_{(1)}) = u_{(1)}, \quad l^-(0) = l_{road}^- \tag{5.31}$$

故 $W(0) = u_{(1)} - l_{road}^- = g_0$。

当 $1 \le p \le k-1$ 时，$\mathcal{L}_p = \{(1), \ldots, (p)\}$，$\mathcal{R}_p = \{(p+1), \ldots, (k)\}$

$$l^+(p) = \min(l_{road}^+,\ u_{(p+1)}) = u_{(p+1)}, \quad l^-(p) = \max(l_{road}^-,\ v_{(p)}) = v_{(p)} \tag{5.32}$$

故 $W(p) = u_{(p+1)} - v_{(p)} = g_p$。

当 $p = k$ 即所有障碍物均为左绕时，$\mathcal{L}_k = \{(1), \ldots, (k)\}$，$\mathcal{R}_k = \varnothing$

$$l^+(k) = l_{road}^+, \quad l^-(k) = \max(l_{road}^-,\ v_{(k)}) = v_{(k)} \tag{5.33}$$

故 $W(k) = l_{road}^+ - v_{(k)} = g_k$。

以上 l^+ 和 l^- 的化简中，对中间分割点 $1 \le p \le k-1$ 假设 $l_{road}^+ \ge u_{(p+1)}$ 且 $l_{road}^- \le v_{(p)}$。该假设在障碍物位于道路范围内时自然成立，因为 $u_{(p+1)} = l_{(p+1)}^- - \delta_{(p+1)} < l_{road}^+$，$v_{(p)} = l_{(p)}^+ + \delta_{(p)} > l_{road}^-$。对端部分割点 $p = 0$ 或 $p = k$，道路边界可能构成约束瓶颈。分以下两种情形讨论。其一，若对所有障碍物均有 $u_{(1)} \le l_{road}^+$

且 $v_{(k)} \geq l_{road}^{-}$ 即所有障碍物均在道路范围内，则 $W(p) = g_p$ 对 $p = 0, 1, \ldots, k$ 均严格成立。其二，若存在 $u_{(1)} > l_{road}^{+}$，说明该障碍物已完全超出道路左边界，不构成右绕约束，几何上应将其从考虑集合中剔除，此时 k 减 1、间隙重新编号后回到情形一。$v_{(k)} < l_{road}^{-}$ 的情形对称处理。因此在剔除道路边界外的障碍物后，选取最大 g_p 对应的分割点 p^* 即为最大化 $W(p)$ 的分割点，$W^* = \max_p g_p$ 严格成立。

图5.8以一维几何图示给出引理 1 与定理 1 的直观证明。上半部分将排序后的 $k = 4$ 个障碍物投影在 l 轴上，每个障碍物两侧以虚线标出各自的安全边界 u_i 与 v_i。相邻障碍物之间以及障碍物与道路边界之间形成 $k + 1 = 5$ 个候选间隙 $g_0, \ldots, g_4$。粉色加粗的 $g_2^\star$ 即为该算例中的最大间隙，对应最优分割点 $p^\star = 2$。下半部分给出该分割对应的决策向量 $\mathbf{d}^\star = (R, R, L, L)$，右绕群 $\mathcal{R}$ 与左绕群 $\mathcal{L}$ 在横向排序中形成连续区段，没有任何“RLR”式的交错，这正是引理 1 所保证的最优分割连续性。可见，原本 $2^k = 16$ 种组合中的最优解，必然落在 $k + 1 = 5$ 个连续分割对应的候选解之中，算法只需线性扫描比较间隙大小即可定位。

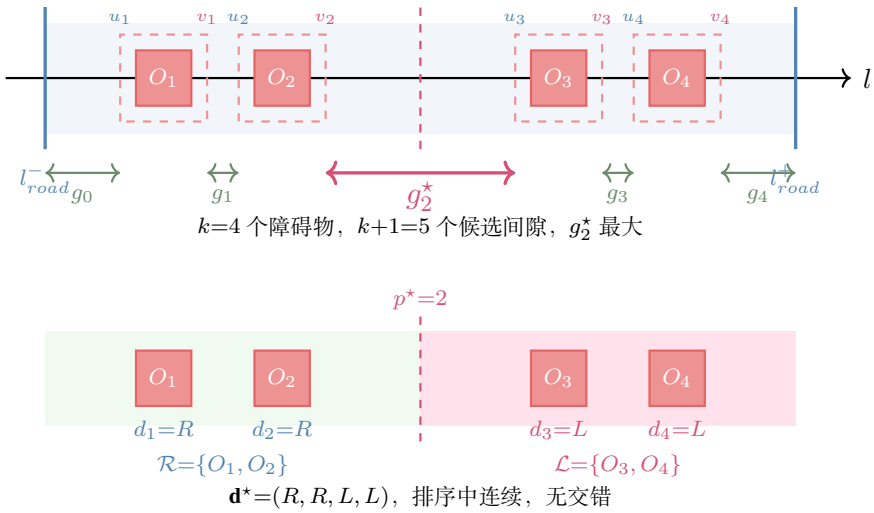


图 5.8 最优分割连续性与最大间隙最优性的几何图证

上述证明严格验证了前文的几何直觉，最优通行带确实等价于最宽横向间隙的选取。引理 1 的交换论证排除了任何非连续分配可能更优的可能性，定理 1 则将通行带宽度与间隙值建立了精确的等式关系，使得最优解可通过一次线性扫描获得。

需要注意几个边界情形。第一，当某些障碍物在横向上存在重叠即 $v_{(p)} > u_{(p+1)}$ 时，对应的间隙 g_p 为负值，表示该通道不可通行，算法自然跳过。第二，当所有 $k + 1$ 个间隙均为负或不足车宽时，$W^* \leq 0$，说明本车道确实不存在可通

行路径，这是真实的 BLOCKED 而非算法的局限。第三，引理 1 的证明仅要求 u 值按排序单调不减。当存在 u_i 相同的障碍物时，可以按 v_i 的降序作为二级排序键，不影响引理和定理的成立。

5.4.4 算法描述

基于上述理论，算法1给出了时变最优通行带求解的完整流程。该算法将理论推导中的引理 1 和定理 1 转化为可执行的计算步骤，依次完成到达时间估计、时变 SL 投影、扫描线分组、间隙计算与最大间隙选取、避让方向分配、路径边界构建和速度约束输出等七个步骤。算法的总体结构如图5.9所示，从参考线与障碍物集合起步，先经威胁度评估与动态安全距离确定每个障碍物的 δ_i，再由扫描线完成纵向重叠分组，组内按 u_i 排序查找最大间隙并分配避让方向，最后将所有组的边界合并为全局 `path_boundary` 并投影至 ST 空间生成走廊。图中每一步骤右侧标注了复杂度，主要开销集中在扫描线分组的 $O(n \log n)$，其余步骤均为线性，总体满足 $O(n \log n + nN_s)$ 的实时性目标。

图 5.9 时变最优通行带算法的总体流程与复杂度

5.4.5 复杂度分析

本节分析算法1各步骤的时间和空间复杂度。

输入: n 个障碍物，含预测轨迹，道路边界 $[l^-_{road}(s), l^+_{road}(s)]$、自车状态 $(s_{ego}, v_{ego}, a_{ego})$、各障碍物安全裕度 δ_i

输出: 路径边界 $\{[l^-(s), l^+(s)]\}$、速度约束集合 $\mathcal{T}$

//步骤 1：到达时间估计;

foreach 离散位置 s **do**

  if $|a_{ego}| > \epsilon$ **then**

    $\tau(s) \leftarrow \left(-v_{ego} + \sqrt{v_{ego}^2 + 2\,a_{ego}\,(s - s_{ego})}\right)/a_{ego}$;

  else

    $\tau(s) \leftarrow (s - s_{ego})/v_{ego}$;

  end

end

//步骤 2：时变 SL 投影;

foreach 障碍物 O_i **do**

  查询预测轨迹，计算 $O_i(\tau(s))$;

  if O_i 在 $\tau(s)$ 时刻已移出参考线附近 **then**

    标记 O_i 在位置 s 处不活跃 ;

  end

  $u_i(s) \leftarrow l_i^-(\tau(s)) - \delta_i$，$v_i(s) \leftarrow l_i^+(\tau(s)) + \delta_i$;

end

//步骤 3：按 s_i^- 排序，扫描线分组;

将所有障碍物按 s_i^- 升序排序 ;

$s_{max} \leftarrow -\infty$，$group \leftarrow \varnothing$;

foreach 排序后的障碍物 O_i **do**

  if $s_i^- \le s_{max}$ **then**

    将 O_i 加入当前分量，$s_{max} \leftarrow \max(s_{max}, s_i^+)$;

  else

    保存当前分量，开始新分量 $\{O_i\}$，$s_{max} \leftarrow s_i^+$;

  end

end

//步骤 4：组内按 u_i 排序，计算间隙;

foreach 连通分量 C_j，含 k 个障碍物 **do**

  按 u_i 升序排列为 $O_{(1)}, \ldots, O_{(k)}$;

  计算 $k+1$ 个间隙 $g_0, g_1, \ldots, g_k$（式(5.29)）;

  //步骤 5：选最大间隙，分配避让方向;

  $p^* \leftarrow \arg\max_p g_p$;

  $d_{(i)} \leftarrow L$，$\forall\, i \le p^*$；$d_{(i)} \leftarrow R$，$\forall\, i > p^*$;

end

//步骤 6：构建路径边界;

foreach 离散位置 s **do**

  按分配的避让方向更新 $[l^-(s), l^+(s)]$;

end

//步骤 7：输出速度约束;

foreach 依赖动态障碍物移开的位置 s_k **do**

  $\mathcal{T} \leftarrow \mathcal{T} \cup \{(s_k, \tau(s_k))\}$;

end

返回 $\{[l^-(s), l^+(s)]\}$，$\mathcal{T}$;

算法 1: 时变最优通行带求解算法

对单个含 k 个障碍物的连通分量，算法步骤 4 中按 u_i 排序的时间复杂度为 $O(k\log k)$，计算 $k+1$ 个间隙和选取最大值均为 $O(k)$，因此单分量的时间复杂度为 $O(k\log k)$。空间复杂度为 $O(k)$，用于存储排序后的障碍物列表和间隙数组。

全局来看，步骤 1 和步骤 2 对每个障碍物执行常数时间操作，复杂度为 $O(n)$。步骤 3 中按 s_i^- 排序的复杂度为 $O(n\log n)$，扫描线遍历为 $O(n)$。步骤 4–5 中，所有连通分量的组内排序总复杂度满足 $\sum_j O(k_j\log k_j)\le O(n\log n)$，其中 $\sum_j k_j = n$，由 Jensen 不等式的推广可得。步骤 6 和步骤 7 均为 $O(K)$，其中 K 为离散位置数。因此，算法的总时间复杂度为 $O(n\log n+K)$，其中 $O(n\log n)$ 为障碍物处理，$O(K)$ 为路径边界构建。在 $n\ll K$ 的典型场景下，$O(K)$ 项占主导。

表5.1从多个维度对比了本文算法与 Apollo Baseline 的差异。

表 5.1 本文算法与 Apollo Baseline 的对比

	Apollo Baseline	**本文方法**
决策粒度	逐障碍物独立决策	组级全局决策
最优性保证	贪心策略，无最优性保证	定理保证组内最优
时间信息	仅 $t=0$ 快照	预测轨迹 $O_i(\tau(s))$
动态障碍物	不参与路径决策	通过预测位置参与
BLOCKED 处理	截断路径 → 停车	时变间隙可能打开 → 继续通行
速度协调	无	输出 $\tau(s_k)$ 约束
时间复杂度	$O(nK)$	$O(n\log n)$

Baseline 在每个离散位置 s 处遍历所有障碍物共 K 个离散点，总复杂度为 $O(nK)$。在典型配置下 $K\approx 200$，路径长度 100 m，步长 0.5 m，$n\le 20$，两者均可在毫秒级完成。

以实际数据为例，当 $n=10$，本文算法的操作次数约为 $10\times\log_2 10\approx 33$ 其中排序占主导，而 Baseline 的操作次数为 $10\times 200=2000$。虽然两者在绝对时间上均远低于 100 ms 的规划周期限制，但本文算法在理论上更优。更重要的是，本文算法的核心价值不在于计算效率的提升，而在于保证了决策的全局最优性，这是 Baseline 的贪心策略无法提供的。

5.5 ST 层时空可通行走廊

前两节从 SL 层面解决了静态场景即 $t=0$ 时刻快照下的最优通行带问题。然而，第四章揭示的 ST 层面问题，即动态障碍物被排除、路径-速度缺乏协调，要求将 SL 层的理论扩展至时变场景。本节构建时空可通行走廊，利用 Apollo Prediction 模块提供的障碍物预测轨迹，将 SL 空间的最优通行带投影至 ST 空间，

建立路径决策与速度规划之间的约束传递通道。

5.5.1 到达时间估计

时变通行带的核心在于将自车的到达时间与障碍物的预测轨迹相耦合。到达时间函数 $\tau(s)$ 估计自车到达纵向位置 s 的时刻。

最简单的估计是匀速假设 $\tau_0(s) = (s - s_{ego})/v_{ego}$。然而，匀速假设在自车正在加速或减速时会引入显著误差。例如，自车以 $v_{ego} = 5\,\mathrm{m/s}$ 加速行驶，取 $a_{ego} = 2\,\mathrm{m/s^2}$，匀速假设估计到达前方 20 m 处需要 $\tau_0 = 4.0\,\mathrm{s}$，但实际仅需约 2.9 s，误差超过 1 s，这在动态障碍物场景中足以导致通行带的时间有效性判断失误。

为提高精度，采用二阶运动学模型。根据匀加速运动方程 $s - s_{ego} = v_{ego}\,t + \frac{1}{2}a_{ego}\,t^2$，解出到达时间

$$\tau(s) = \frac{-v_{ego} + \sqrt{v_{ego}^2 + 2\,a_{ego}\,(s - s_{ego})}}{a_{ego}} \tag{5.34}$$

其中 s_{ego}、v_{ego} 和 a_{ego} 分别为自车当前的纵向位置、速度和加速度，均可从 Apollo 的 `VehicleState` 模块直接获取。

该公式在两种边界情形下退化为更简单的形式。当 $a_{ego} \to 0$ 时，对根号内进行 Taylor 展开 $\sqrt{1+x} \approx 1 + x/2$

$$\tau(s) = \frac{v_{ego}\left(\sqrt{1 + \dfrac{2a_{ego}(s - s_{ego})}{v_{ego}^2}} - 1\right)}{a_{ego}} \approx \frac{v_{ego} \cdot \dfrac{a_{ego}(s - s_{ego})}{v_{ego}^2}}{a_{ego}} = \frac{s - s_{ego}}{v_{ego}} \tag{5.35}$$

即退化为匀速估计。当 $v_{ego} \approx 0$ 且 $a_{ego} \approx 0$ 即自车近似静止时，令 $\tau(s) = +\infty$，算法退化为静态版本，这在物理上是合理的，静止的自车不会到达任何前方位置，因此障碍物的预测轨迹不产生时变效应。

在实际实现中，若上一规划周期已输出速度曲线 $s_{prev}(t)$，可通过对其求逆 $\tau(s) = s_{prev}^{-1}(s)$ 获得最精确的估计，作为图中” 原始速度曲线反推” 曲线的来源。二阶运动学模型作为首个规划周期即无历史速度曲线可用或速度曲线不可用时的回退方案。图5.10以 $v_{ego} = 5\,\mathrm{m/s}$、$a_{ego} = 2\,\mathrm{m/s^2}$ 的加速工况展示三种模型在前方 30 m 范围内的估计曲线，以原始速度曲线反推为基准，在 $s = 20\,\mathrm{m}$ 处匀速假设的误差约为 0.87 s，二阶运动学模型将残差压缩至约 0.22 s，相比匀速假设降低一个量级。

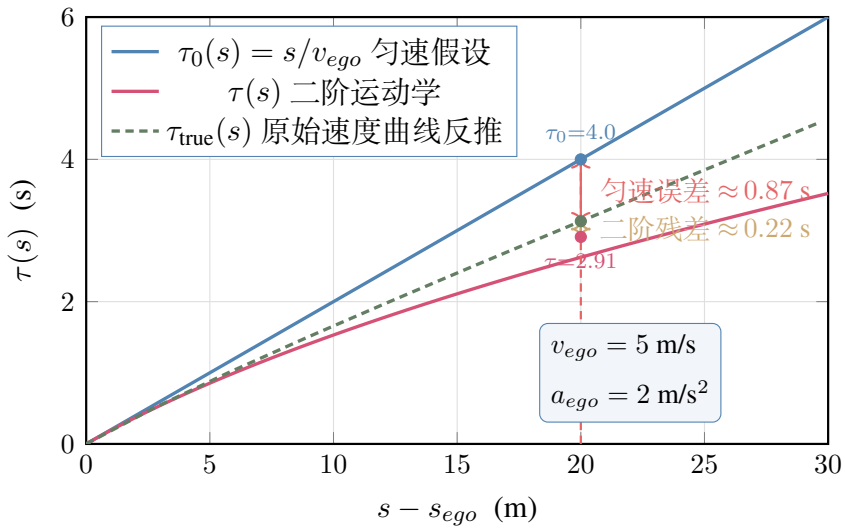


图 5.10 到达时间估计的匀速假设与二阶运动学模型对比

需要指出的是，到达时间估计存在固有的循环依赖，$\tau(s)$ 用于计算时变通行带，通行带约束影响速度规划，而速度规划结果反过来决定了自车的实际到达时间。本文采用 Apollo 的周期性重规划机制，每 100 ms 执行一次，隐式处理这一耦合，每个周期使用上一周期的速度输出更新 $\tau(s)$，在连续多个规划周期中实现近似收敛。这种“单步延迟”策略在 Apollo 的路径-速度解耦框架中是标准做法，路径规划和速度规划本身就存在类似的耦合，其在城市道路工况下的实际效果已通过第六章的实验验证。

5.5.2 动态障碍物的时变 SL 投影

Apollo 的 Prediction 模块为每个动态障碍物提供未来 5 s 内的预测轨迹，以离散时间点序列 $\{(x_i(t_j), y_i(t_j))\}$ 的形式给出。利用该预测轨迹，可以计算障碍物在自车到达时刻 $\tau(s)$ 的 SL 位置。

（1）在预测轨迹中查询时刻 $\tau(s)$ 对应的障碍物位置，通过线性插值获取。

（2）将该位置从 Cartesian 坐标投影至参考线的 Frenet 坐标，得到 $(s_i(\tau), l_i(\tau))$。

（3）结合障碍物的几何尺寸，计算时变 SL 边界 $[l_i^-(\tau(s)), l_i^+(\tau(s))]$。

（4）若障碍物在 $\tau(s)$ 时刻的纵向位置与 s 的距离超过其纵向尺寸的一半，即障碍物已移出该纵向截面的影响范围，则标记其在位置 s 处不活跃，不参与分组和间隙计算。

这一机制使得即将移出自车路径的动态障碍物如横穿行人、变道车辆不再对路径边界构成约束，从而避免 Baseline 中因忽略动态信息而导致的不必要停车。

需要特别说明的是，对于预测轨迹超出 Prediction 模块时间范围，通常为 5 s，的情形，采用保守策略，假设障碍物保持最后预测位置不变。这一假设的合理性在于，5 s 内自车以城市道路常见速度，取 8 至 12 m/s，可行驶 40–60 m，已覆盖绝大多数需要路径决策的场景。对于更远距离的障碍物，路径决策可以在后续规划周期中随着距离缩短而更新。

此外，对于从 Prediction 模块接收到多条预测轨迹如车辆可能直行或变道的情况，当前实现取概率最高的轨迹。更精细的处理方式是对多条轨迹分别计算时变投影并取并集进行保守处理，但这会增加计算复杂度，留作后续工作。图5.11归纳了从 Prediction 模块原始轨迹到最终活跃障碍物集的四步数据流，到达时间 $\tau(s)$ 在三个步骤中被共同引用。

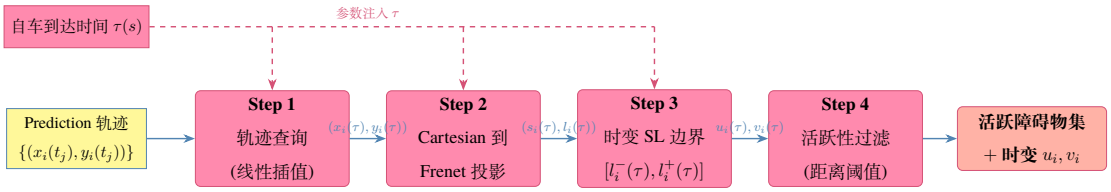


图 5.11 动态障碍物时变 SL 投影的四步流水线

5.5.3 时变间隙与 ST 走廊定义

在引入时变 SL 投影后，间隙也成为位置 s 的函数。对于连通分量内按 $u_i(\tau(s))$ 排序的 k 个障碍物，时变间隙定义为

$$g_p(s) = \begin{cases} u_{(1)}(\tau(s)) - l_{road}^-(s) & p = 0 \\ u_{(p+1)}(\tau(s)) - v_{(p)}(\tau(s)) & p = 1, \ldots, k-1 \\ l_{road}^+(s) - v_{(k)}(\tau(s)) & p = k \end{cases} \tag{5.36}$$

定理 1 在每个 s 截面上仍然成立。这是因为定理 1 的证明过程即引理 1 的交换论证和定理 1 的直接代入验证仅依赖于 u 和 v 值的排序关系，不要求 u 和 v 是常数。只要在每个 s 处重新按 $u_i(\tau(s))$ 排序并计算间隙，定理 1 的结论即可直接应用。

全路径的最优通行带宽度定义为

$$W^* = \max_p \min_s g_p(s) \tag{5.37}$$

该 max-min 问题要求在所有 s 截面上同时选择一个固定的分割点 p，使得最窄处的间隙最大。精确求解需要枚举 $k+1$ 个 p 值，对每个 p 遍历所有离散 s 求最小间隙，复杂度为 $O(kK)$，其中 K 为离散点数。算法1在实际实现中采用保守近似，以连通分量内所有障碍物均同时活跃的假设，在最密集的 s 截面上计算间隙并选取 p^*。这一近似的安全性由 5.4.1 节的分析保证，最密集截面上的决策自然适用于更稀疏的截面。

与静态版本的关键差异在于动态障碍物的 $u_i(s)$ 和 $v_i(s)$ 随 s 变化，因为 $\tau(s)$ 变化导致查询的预测时刻不同，使得间隙在不同 s 处可能打开或关闭。例如，横穿行人在 s 较小处自车尚未到达时间隙为零，但在 s 较大处到达时行人已移走间隙打开。

将“间隙足以容纳自车”这一条件与到达时间 $\tau(s)$ 联合投影至 ST 平面，可以定义 ST 可通行区域即时空可通行走廊

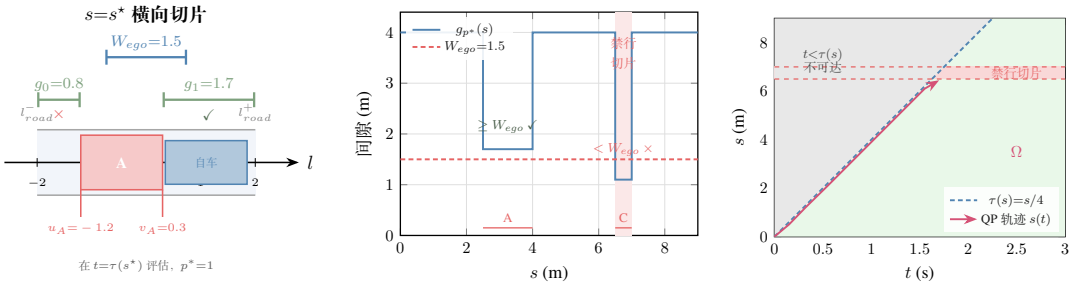
$$\Omega = \{(s,t) \mid g_{p^*}(s) \geq W_{ego},\ \ t \geq \tau(s)\} \tag{5.38}$$

其中第一个条件要求位置 s 处的最大间隙足以容纳自车宽度，第二个条件要求自车的到达时间不早于 $\tau(s)$。Ω 的物理含义是，自车的 ST 轨迹 $s(t)$ 只要完全落在区域 Ω 内，就能保证在每个纵向位置都有足够的横向通行空间。

由式(5.38)可见，Ω 在 s 维度由 $g_{p^*}(s) \geq W_{ego}$ 判定通行与否，在 t 维度由 $\tau(s)$ 作为下沿。若某位置 s_k 的最优间隙不足以容纳自车，整条纵向列 $\{s_k\} \times \mathbb{R}_+$ 被从 Ω 中排除，形成 s 方向的”禁行切片”。对于静态障碍物，$g_{p^*}(s)$ 仅取决于几何投影；对于动态障碍物，$g_{p^*}(s)$ 由自车抵达 s 的查询时刻 $\tau(s)$ 决定——若自车到达 s_k 时障碍物恰好未移开，s_k 列禁行，若已移开则打开。禁行切片在 Ω 中呈现为 s 方向的水平缺口。

图5.12以三阶段递进形式展示 Ω 的构建过程，构造场景设道路宽 $l \in [-2, 2]$、$W_{ego} = 1.5$、$v_{ego} = 4$，含静态偏置障碍 A 和宽阻挡障碍 C。子图 (a) 固定 $s = s^\star = 3.25$（A 内部），在 SL 横向切片上按式(5.36)计算 $g_0 = 0.8$ 与 $g_1 = 1.7$，$g_1 \geq W_{ego}$ 决定 $p_1^* = 1$ 且 $s^\star$ 通行。子图 (b) 将切片分析推广至全 s 范围，绘出 $g_{p^*}(s)$ 对 s 的剖面曲线与 W_{ego} 参考线，A 段剖面在 1.7 处通行，C 段因双侧间隙均为 1.1 跌至 W_{ego} 之下构成禁行切片。子图 (c) 将通行 s 区间映射至 ST 空间并叠加可达集

$t \geq \tau(s)$ 得到 Ω，其几何形态为 $\tau(s)$ 右下方的绿色区域扣除 $s \in [6.5, 7]$ 的红色水平缺口。QP 最优轨迹从 $(0,0)$ 出发，紧贴 $\tau(s)$ 的右侧爬升，在禁行切片下沿 $s = 6.5$ 处终止。这一构造表明 Ω 的边界由横向间隙判定 $g_{p^*}(s) \geq W_{ego}$ 与可达性约束 $t \geq \tau(s)$ 联合生成，承载了" 横向间隙 $\rightarrow$ 纵向禁行 $\rightarrow$ 速度约束" 的全部传递信息。



(a) $s = s^\star$ **横向切片间隙分析** **(b)** $g_{p^*}(s)$ **剖面判定通行** s **区间** **(c) 映射至 ST 空间合成走廊** Ω

图 5.12 时空可通行走廊的三阶段构建过程

5.5.4 走廊边界与速度约束

走廊 Ω 的边界定义了自车 ST 轨迹的可行域。SL 空间中的通行带 $\mathcal{C} = \{(s,l) \mid l^-(s) \leq l \leq l^+(s)\}$ 是在到达时间函数 $\tau(s)$ 的假设下计算的，其有效性依赖于一个关键前提，即自车的实际到达时间不早于 $\tau(s)$

$$t_{arrive}(s) \geq \tau(s), \quad \forall s \tag{5.39}$$

若自车提前到达某位置 s_k，例如因加速超过了 $\tau(s)$ 的估计，则动态障碍物可能尚未移至预测位置，通行带在该处可能无效。该有效性条件可转化为速度规划的线性约束。对于每个依赖动态障碍物移开的关键位置 s_k，要求

$$s(t) \leq s_k, \quad \forall t < \tau(s_k) \tag{5.40}$$

式(5.40)的物理含义是，自车不得在 $\tau(s_k)$ 时刻之前到达位置 s_k，从而为动态障碍物的移开预留足够时间。从 ST 走廊的视角看，该约束正是走廊 Ω 左边界在 $s = s_k$ 处的数学表达，要求自车的 ST 轨迹不穿越走廊边界。更一般地，当存在多个关键位置 $s_1, s_2, \ldots$ 时，各约束点 $(\tau(s_k), s_k)$ 的集合构成了走廊 Ω 左边界的离散采样。走廊约束的最终效果是要求 QP 的最优速度曲线 $s^*(t)$ 始终位于走廊 Ω 之内

$$(s^*(t_j),\ t_j) \in \Omega, \quad j = 0, 1, \ldots, K \tag{5.41}$$

5.5.5 走廊约束的速度规划融合

Apollo 速度 QP 问题的基本形式

Apollo 的 `PiecewiseJerkSpeedOptimizer` 将速度规划离散化为等时间间隔 Δt 上的二次规划（QP）问题。设时间步为 $t_0, t_1, \ldots, t_K$，其中 $\Delta t = t_{j+1} - t_j$，默认取 0.1 s，在每个时间步 t_j 处的决策变量为纵向位置 s_j、速度 $\dot{s}_j$ 和加速度 $\ddot{s}_j$。QP 问题的目标函数为

$$\min_{\{s_j, \dot{s}_j, \ddot{s}_j\}} \sum_{j=0}^{K} \left[w_s \left(s_j - s_{ref,j} \right)^2 + w_v \left(\dot{s}_j - v_{ref} \right)^2 + w_a \, \ddot{s}_j^2 \right] + \sum_{j=0}^{K-1} w_j \left(\frac{\ddot{s}_{j+1} - \ddot{s}_j}{\Delta t} \right)^2 \tag{5.42}$$

其中 $s_{ref,j}$ 为参考位置，通常取匀速巡航位置，v_{ref} 为期望速度，w_s, w_v, w_a, w_j 分别为位置跟踪、速度跟踪、加速度和 jerk 的权重系数。最后一项惩罚相邻时间步之间加速度的变化率即 jerk，以保证速度曲线的平滑性和乘坐舒适性。

目标函数受到以下三类约束。

运动学离散化约束，保证位置、速度、加速度之间的一致性

$$s_{j+1} = s_j + \dot{s}_j \, \Delta t + \tfrac{1}{2} \, \ddot{s}_j \, \Delta t^2 + \tfrac{1}{6} \, \dddot{s}_j \, \Delta t^3 \tag{5.43}$$

$$\dot{s}_{j+1} = \dot{s}_j + \ddot{s}_j \, \Delta t + \tfrac{1}{2} \, \dddot{s}_j \, \Delta t^2 \tag{5.44}$$

其中 $\dddot{s}_j = (\ddot{s}_{j+1} - \ddot{s}_j)/\Delta t$ 为离散化 jerk。式(5.43)–(5.44)为等式约束，以三阶 Taylor 展开保证运动学一致性。

物理限制约束

$$0 \le \dot{s}_j \le v_{max}, \quad a_{min} \le \ddot{s}_j \le a_{max}, \quad |\dddot{s}_j| \le j_{max} \tag{5.45}$$

其中 v_{max} 为道路限速，a_{min}、a_{max} 为加减速度限制，取 $a_{min} < 0$，j_{max} 为 jerk 限制。$\dot{s}_j \ge 0$ 约束保证自车不会倒车。

ST 边界约束，来自 ST 图中障碍物的投影

$$s_j^{lb} \le s_j \le s_j^{ub}, \quad j = 0, 1, \ldots, K \tag{5.46}$$

其中 s_j^{lb} 和 s_j^{ub} 分别为时间步 t_j 处纵向位置的下界和上界，由 `SpeedBoundsDecider` 根据 ST 图中各障碍物的 YIELD/FOLLOW/OVERTAKE 决策计算得出。s_j^{ub} 对应 YIELD/FOLLOW 决策即自车需在障碍物之后，s_j^{lb} 对应 OVERTAKE 决策即自车需在障碍物之前。

综上，Apollo 的速度 QP 问题可紧凑表示为

$$\min_{\mathbf{x}} \ \frac{1}{2}\mathbf{x}^T\mathbf{H}\,\mathbf{x} + \mathbf{f}^T\mathbf{x} \quad \text{s.t.} \quad \mathbf{A}_{eq}\,\mathbf{x} = \mathbf{b}_{eq}, \quad \mathbf{x}^{lb} \le \mathbf{x} \le \mathbf{x}^{ub} \tag{5.47}$$

其中 $\mathbf{x} = (s_0, \dot{s}_0, \ddot{s}_0, s_1, \dot{s}_1, \ddot{s}_1, \ldots)^T$ 为决策变量向量，$\mathbf{H}$ 为正定 Hessian 矩阵，由式(5.42)的二次项系数构成，$\mathbf{A}_{eq}$ 编码运动学等式约束即式(5.43)–(5.44)，$\mathbf{x}^{lb}$ 和 $\mathbf{x}^{ub}$ 编码物理限制和 ST 边界约束即式(5.45)–(5.46)。该 QP 问题由 OSQP 求解器求解。

时间有效性约束的嵌入

本文算法输出的速度约束集合 $\mathcal{T} = \{(s_k, \tau_k)\}$，其中 $\tau_k = \tau(s_k)$，每个元素表示“自车不得在 τ_k 时刻之前到达位置 s_k”。该约束直接转化为对 ST 边界上界的收紧

$$s_j^{ub} \leftarrow \min\!\left(s_j^{ub},\ s_k\right), \quad \forall\, j : t_j < \tau_k, \quad \forall\, (s_k, \tau_k) \in \mathcal{T} \tag{5.48}$$

在 ST 图中，式(5.48)的几何意义如图5.13所示。对于每个约束点 (τ_k, s_k)，在时间轴 $t < \tau_k$ 的区间内添加一条水平上界线 $s \le s_k$，禁止自车在该时间范围内超越位置 s_k。

式(5.48)与 Apollo 原有的 ST 边界约束即式(5.46)形式完全一致，均为 s 变量的上界约束。其嵌入过程仅修改 $\mathbf{x}^{ub}$ 向量中对应分量的值，不改变 QP 问题的结构，决策变量数、Hessian 矩阵 $\mathbf{H}$、等式约束矩阵 $\mathbf{A}_{eq}$ 和约束矩阵的稀疏模式均保持不变。若 s_k 对应的上界比该时间步已有的 ST 边界上界更紧，则该约束生效；否则被已有约束隐含满足。因此，OSQP 求解器的收敛性和计算效率不受影响。

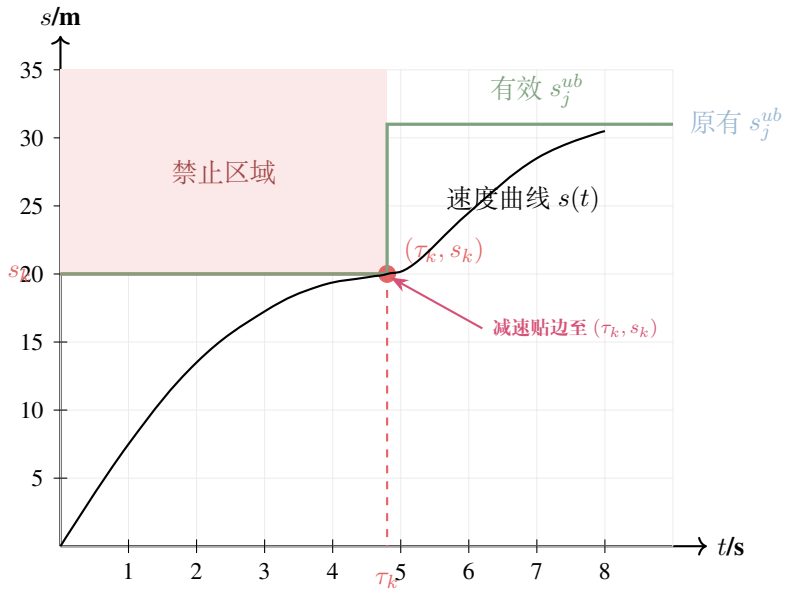


图 5.13 时间有效性约束在 ST 图中的几何表示

红色区域为新增禁止区域，绿色粗线为收紧后的有效上界，速度曲线在约束点附近轻微减速以满足时间有效性条件。

横穿行人场景数值示例

本小节通过 5.2 节中引入的横穿行人场景，完整展示从时变通行带到速度规划的全链路工作过程。

场景设定 自车以 $v_{ego} = 8\,\text{m/s}$ 匀速行驶，取 $a_{ego} \approx 0$，前方 $s_{ped} = 10\,\text{m}$ 处一名行人以 $v_{lat} = 1.2\,\text{m/s}$ 从右向左横穿马路。行人当前位于车道中央，取 $l = 0\,\text{m}$，宽度 0.5 m。道路宽 3.75 m，$l_{road}^{-} = -1.875\,\text{m}$，$l_{road}^{+} = 1.875\,\text{m}$，$\delta = 1.2\,\text{m}$。

步骤 1，到达时间计算。$a_{ego} \approx 0$，由式(5.35)退化为匀速估计

$$\tau(s_{ped}) = \frac{10 - 0}{8} = 1.25\,\text{s}$$

步骤 2，时变 SL 投影。在 $\tau = 1.25\,\text{s}$ 时，行人横向位移 $\Delta l = 1.2 \times 1.25 = 1.5\,\text{m}$，位于 $l = 1.5\,\text{m}$。时变 SL 边界为 $[l_{ped}^{-}, l_{ped}^{+}] = [1.25, 1.75]\,\text{m}$。

步骤 3，间隙计算。单个障碍物，此时 $k = 1$，计算两个间隙

$$g_0 = u_{ped} - l_{road}^{-} = (1.25 - 1.2) - (-1.875) = 1.925\,\text{m}$$
$$g_1 = l_{road}^{+} - v_{ped} = 1.875 - (1.75 + 1.2) = -1.075\,\text{m}$$

最大间隙 $g_0 = 1.925\,\text{m}$，大于自车宽度 $W_{ego} = 1.8\,\text{m}$，自车可从行人右侧

即道路右半侧安全通过。Baseline 的静态快照中行人位于 $l = 0$，间隙为 $g_0 = (0 - 0.25 - 1.2) - (-1.875) = 0.425\,\mathrm{m} < W_{ego}$，通行带不可行。

步骤 4，速度约束生成。该通行带依赖行人在 $\tau = 1.25\,\mathrm{s}$ 时已移至 $l = 1.5\,\mathrm{m}$ 的预测位置，因此算法输出速度约束

$$\mathcal{T} = \{(s_k, \tau_k)\} = \{(10\,\mathrm{m},\ 1.25\,\mathrm{s})\}$$

步骤 5，QP 约束嵌入。设 $\Delta t = 0.1\,\mathrm{s}$，时间步 $t_0 = 0, t_1 = 0.1, \ldots, t_{12} = 1.2\,\mathrm{s}$ 均满足 $t_j < \tau_k = 1.25\,\mathrm{s}$。由式(5.48)，对这些时间步收紧上界

$$s_j^{ub} \leftarrow \min(s_j^{ub},\ 10\,\mathrm{m}), \quad j = 0, 1, \ldots, 12$$

对于 $j \leq 12$，即 $t_j \leq 1.2\,\mathrm{s}$，原有 ST 上界 s_j^{ub} 通常较大，无其他近距离障碍物时接近规划终点，故新约束 $s_j \leq 10\,\mathrm{m}$ 生效。对于 $j \geq 13$，即 $t_j \geq 1.3\,\mathrm{s} > \tau_k$ 时，上界不受影响。

步骤 6，QP 求解结果分析。在新增上界约束下，QP 优化器（式(5.42)）在平滑性惩罚的驱动下，输出的速度曲线不再是简单的匀速 8 m/s 直线，而是在接近行人位置时轻微减速。具体行为如下。

自车在 $t \in [0, 0.8]\,\mathrm{s}$ 内以接近 8 m/s 的速度行驶约 6.4 m，然后开始减速。由于约束要求 $s_{12} \leq 10\,\mathrm{m}$，在 $t_{12} = 1.2\,\mathrm{s}$ 时不超过 10 m，而匀速 8 m/s 在 $t = 1.2\,\mathrm{s}$ 时已到达 $s = 9.6\,\mathrm{m}$，因此减速幅度很小，QP 输出的最低速度约为 7.5 m/s。在 $t \approx 1.3\,\mathrm{s}$ 时自车到达 $s = 10\,\mathrm{m}$，此时行人已移至 $l = 1.56\,\mathrm{m}$，间隙充分打开。此后速度恢复至 v_{ref}。

与 Baseline 行为的对比 Baseline 的处理链路与本文方法截然不同。由于 `IsStatic()` 过滤，行人被排除在 PathBoundsDecider 之外，不参与路径边界构建。然而在速度规划阶段，SpeedBoundsDecider 虽然利用 Prediction 模块的预测轨迹将行人投影至 ST 图生成 ST boundary 多边形，但 SpeedDecider 在对该 boundary 做出 YIELD 决策时，将行人横穿车道的整个时间跨度都视为禁行区，要求自车在行人的纵向区间下方等待。这导致以下结果。

- s_j^{ub} 在行人对应的时间区间内被大幅压低至 $s_{ped}^{-} \approx 8\,\mathrm{m}$，且持续时间覆盖行人横穿整个车道的全部时长约 $3.75/1.2 \approx 3.1\,\mathrm{s}$；
- QP 优化器输出急减速曲线，自车从 8 m/s 开始以约 $-3\,\mathrm{m/s^2}$ 减速，在 $s \approx 8\,\mathrm{m}$ 处减速至 0，完全停车等待；

- 行人完全通过车道后，约在 $t \approx 3.1\,\mathrm{s}$ 时，ST 上界解除，自车才重新起步加速。

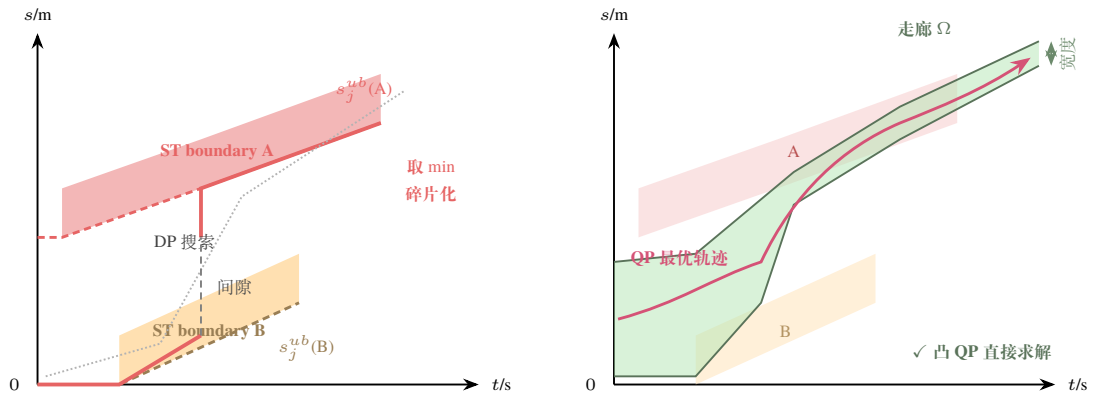
表5.2量化对比了两种方案在该场景下的关键指标。

表 5.2 横穿行人场景下 Baseline 与本文方法的速度规划对比

	Apollo Baseline	**本文方法**
路径决策	行人被 `IsStatic()` 过滤，不参与路径边界构建	时变 SL 投影纳入行人，识别间隙将在 $\tau = 1.25\,\mathrm{s}$ 后打开
速度决策来源	SpeedDecider 根据 $t = 0$ 快照的 ST 图给出 YIELD	时间有效性条件生成精确的 ST 上界约束
ST 上界约束	$s_j^{ub} \leq 8\,\mathrm{m}$，持续 $\approx 3.1\,\mathrm{s}$	$s_j^{ub} \leq 10\,\mathrm{m}$，仅持续 $1.25\,\mathrm{s}$
最低速度	$0\,\mathrm{m/s}$，即完全停车	$\approx 7.5\,\mathrm{m/s}$，仅轻微减速
通过行人区域耗时	$\approx 5.5\,\mathrm{s}$，含停车等待约 $3\,\mathrm{s}$	$\approx 1.4\,\mathrm{s}$
纵向加速度峰值	$-3.0\,\mathrm{m/s^2}$，表现为急减速至停车	$-0.5\,\mathrm{m/s^2}$，表现为平缓减速

该对比揭示了时变扩展的核心价值。Baseline 因忽略行人运动趋势而过度保守地停车等待，本文方法通过预测行人将在 1.25 s 后移开，将保守的 YIELD 停车转化为精确的“不早于 τ_k 到达 s_k”约束，使 QP 优化器输出平滑的减速-通过速度曲线，在保证安全性的前提下显著提升通行效率。

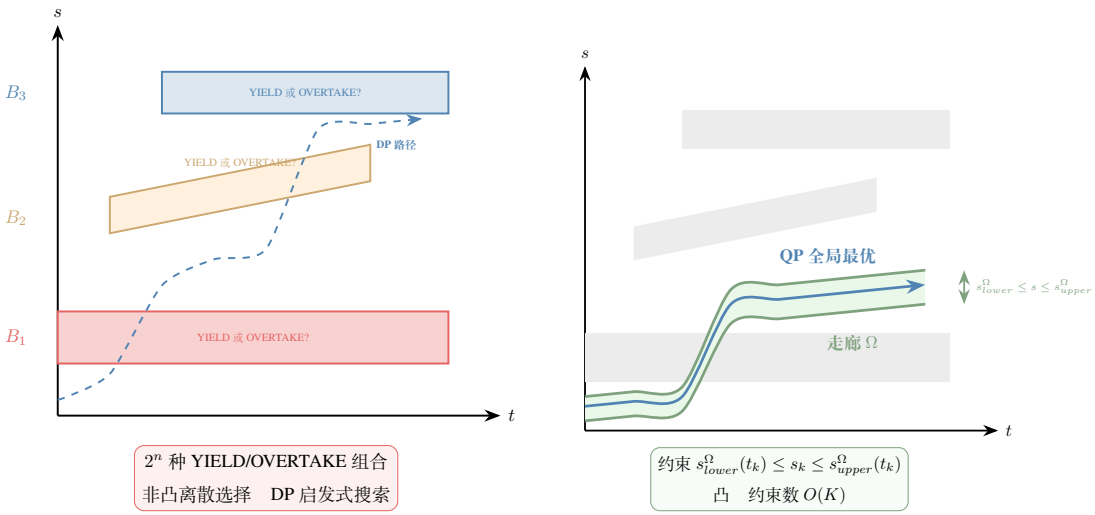
图5.14从更宏观的视角展示改进前后约束生成链路的结构差异。左子图为 Apollo Baseline 的做法，SpeedBoundsDecider 对每个障碍物独立生成 ST boundary 多边形，SpeedDecider 对各 boundary 逐一做出 YIELD 或 OVERTAKE 决策，n 个障碍物产生 2^n 种离散组合。DP 启发式搜索在该组合空间内选取一组决策，再将其转化为分段线性的 s_j^{ub} 约束线供 QP 使用，决策过程的非凸性源于组合选择而非最终 QP 可行域。右子图为本文方法构建的走廊 Ω，SL 层按最大间隙原则全局选定分割 $p^\star$，将 n 个障碍物的横向可通行信息压缩为式(5.36)定义的 $g_{p^\star}(s)$，并通过式(5.38)与 $\tau(s)$ 联合生成 Ω。Ω 是 $\tau(s)$ 右下方由 $g_{p^\star}(s) \geq W_{ego}$ 筛选出的楔形区域，若某 s 处最优间隙不足则形成水平禁行切片，如图中 $s \in [2.6, 3.0]$ 对应的红色条带。QP 速度曲线在 Ω 内部由 OSQP 直接求解，无需在 2^n 组合空间中搜索即可获得全局最优解。



(a) Apollo Baseline **(b) 本文方法**

图 5.14 ST 走廊改进前后的结构对比

图5.15从约束表示的角度进一步揭示了改进的数学本质。左子图显示 Apollo 现有做法中，每个障碍物的 ST boundary 构成一个禁行区，对于每个禁行区 Speed-Decider 需要做出 YIELD 或 OVERTAKE 的离散选择。当存在 n 个障碍物时，YIELD/OVERTAKE 的组合共 2^n 种，构成非凸的离散优化问题，Apollo 通过 DP 启发式搜索在组合空间中选取近似最优的决策组合，最优性无保证。选定组合后，约束被转化为 s_j^{ub} 和 s_j^{lb} 的线性约束，QP 本身是凸的。右子图显示本文方法利用 SL 层已选定的最优分割 $p^\star$ 提前消除了 YIELD/OVERTAKE 的离散选择维度，所有障碍物的时变约束被整合为单一走廊约束，直接以 s_j^{ub} 形式注入 QP，无需 DP 搜索，OSQP 直接求解并保证全局最优。



(a) Apollo 原始　n 个独立 ST 边界约束 **(b) 改进方案　单一走廊约束**

图 5.15 多边界非凸约束与走廊凸约束的对比

5.5.6 与 SpeedBoundsDecider 的双源约束融合

Apollo 的速度 QP 接收两个来源的 ST 边界约束，分别来自速度规划阶段和路径规划阶段。其一，SpeedBoundsDecider 基于路径已确定后的碰撞检测，将障碍物投影至 ST 图并计算 $s_j^{ub,\text{SBD}}$ 和 $s_j^{lb,\text{SBD}}$，这是 Apollo 原有的“自上而下”约束。其二，本文的 ST 走廊约束 $s_j^{ub,\text{corridor}}$ 来自路径决策阶段对动态障碍物时变特性的预判，这是本文新增的“自下而上”约束。两者在 QP 层面通过取最小值融合

$$s_j^{ub} = \min\left(s_j^{ub,\text{SBD}},\ s_j^{ub,\text{corridor}}\right), \quad j = 0, 1, \ldots, K \tag{5.49}$$

两个约束源的信息互补。SpeedBoundsDecider 处理的是“路径上会遇到什么障碍物”，其 ST 边界基于给定路径的碰撞关系计算；ST 走廊约束处理的是“为了让路径决策有效，速度需要满足什么条件”，其信息来源于路径阶段对动态障碍物预测轨迹的分析。这种融合直接解决了第四章所识别的 ST 层问题，即路径决策信息未传递至速度规划。走廊约束正是此前缺失的路径-速度约束传递通道。

具体表现为三种互补情形。

（1）**静态障碍物场景** 动态障碍物不参与间隙计算或已标记为不活跃，$\mathcal{T} = \varnothing$，走廊约束不生效，SpeedBoundsDecider 单独起作用。

（2）**动态障碍物场景** 走廊约束提供了 SpeedBoundsDecider 无法独立推导的时间窗信息。以横穿行人为例，SpeedBoundsDecider 虽利用预测轨迹生成 ST boundary，但 SpeedDecider 的 YIELD 决策将行人横穿的整个时段视为禁行区，而走廊约束给出精确的等待截止时间 $\tau(s_k)$，将过度保守的全时段让行替换为精确的时间窗约束。

（3）**混合场景** 两个约束源各自对不同的时间步起主导作用。SpeedBoundsDecider 负责静态障碍物的 ST 边界，走廊约束负责动态障碍物的时间窗，两者取最小值后 QP 自动得到全局一致的速度曲线。

这一集成方式的合理性在于 SpeedBoundsDecider 之后的 DP 搜索即 `PathTimeHeuristicOptimizer` 已在 ST 图中实现了对 YIELD/OVERTAKE 组合的全局搜索，不存在 PathBoundsDecider 那样因贪心独立决策导致的系统性失效。因此，SpeedBoundsDecider 不需要进行类似的全局最优化改造，ST 走廊约束在 QP 层面补充时变信息即为最小侵入且最高效的集成方式。

5.6 与 Apollo 系统的集成

算法设计完成后，需要将其集成到 Apollo 的 Scenario-Stage-Task 架构中。集成的核心原则是最小侵入性，尽量少修改已有代码，保持与上下游模块的接口兼容，确保改进方案可以独立开关，通过配置文件控制是否启用。图5.16给出了改进后的整体系统架构。输入层维持 Apollo 原有的 Routing、感知障碍物、预测轨迹和车辆状态四类来源。处理层在参考线处理流水线之后，新增了以粉色标注的“威胁度评估”与“动态权重 ST 边界构建”两个改进模块，前者提供本章 5.3 节所述的 δ_i 差异化安全裕度，后者将 5.5 节构建的时空走廊以动态 ST 上界形式注入 QP。输出层仍为 Apollo 标准的 ADCTrajectory，确保下游 Control 模块无需任何修改即可直接消费。

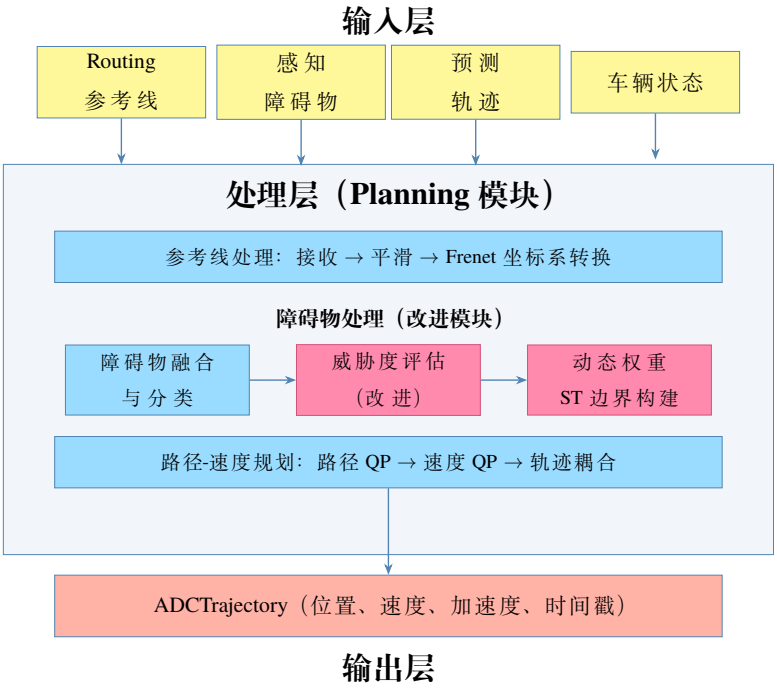


图 5.16 集成改进模块后的系统架构

5.6.1 Task 流水线中的插入位置

本文提出的时变最优通行带算法作用于路径规划阶段，替换 PathBoundsDecider 中原有的逐障碍物独立决策逻辑。具体而言，在 `LaneFollowPath` 等路径 Task 中，替换 `UpdatePathBoundaryBySLPolygon` 函数的障碍物处理部分。

核心修改主要包括三个新增函数。

（1）`GroupObstacles`，实现扫描线检测纵向重叠连通分量的逻辑，对应算法1的步骤 3。

（2）CalculateOptimalNudgeDirection，实现组内按横向排序、计算间隙、选取最大间隙并分配避让方向的逻辑，对应步骤 4–5。

（3）GetTimeVaryingSLBoundary，查询 Prediction 模块的预测轨迹，计算动态障碍物的时变 SL 投影，对应步骤 1–2。

路径 Task 的执行流程保持不变，仍然遵循 Apollo 的 Scenario-Stage-Task 三级架构，仅在路径边界构建环节将逐障碍物贪心逻辑替换为分组全局最优逻辑。速度规划阶段的 Task 流水线无需修改，速度约束以标准的 ST 上界形式传递给 PiecewiseJerkSpeedOptimizer。

上述三个核心函数的 C++ 接口声明如代码5.1所示，其命名和参数风格与 Apollo 现有代码保持一致。

代码 5.1 核心函数接口定义

```
// 障碍物分组：基于纵向重叠的连通分量检测
std::vector<ObstacleGroup> GroupObstacles(
    const std::vector<SLBoundary>& obstacles);
// 组级最优避让方向计算：最大间隙策略
NudgeDirection CalculateOptimalNudgeDirection(
    const ObstacleGroup& group,
    double road_left_bound,
    double road_right_bound);
// 时变SL边界获取：融合动态障碍物预测
std::pair<double, double> GetTimeVaryingSLBoundary(
    double s, double arrival_time,
    const std::vector<PredictionTrajectory>& predictions);
```

值得强调的是，本文的修改策略是最小侵入式的。原有的 UpdatePathBoundaryBySLPolygon 函数签名和输出格式不变，其内部的障碍物遍历和边界更新逻辑被替换为分组 → 排序 → 选间隙 → 分配方向的新流程。

上层调用代码，如 LaneFollowPath::Process 和 LaneBorrowPath::Process，无需任何修改，下游的路径优化器即 PiecewiseJerkPathOptimizer 接收到的路径边界格式也完全兼容。这种设计确保了改进方案可以在不影响 Apollo 其他功能模块的前提下独立部署和测试。

另一个重要的实现细节是，动态障碍物的纳入需要注释掉 IsWithinPathDeciderScopeObstacle 中的静态性过滤条件。该过滤条件原本将所有非静态障碍物排除在路径决策之外，是 Baseline 时间维度次优性的直接根源。注释掉该条件后，动态障碍物通过时变 SL 投影参与路径边界构建，而非被简单忽略。

5.6.2 与现有 LaneBorrow 机制的关系

Apollo 已有的 LaneBorrow 机制用于处理长期阻塞场景。其工作原理如下。当 IsLongTermBlockingObstacle 检测到某障碍物的阻塞计数器 cycle_counter 累积

达到 3 个规划周期，约 300 ms，以上时，触发借道决策，允许自车借用相邻车道通行。该机制是 Apollo 处理本车道不可通行场景的既有方案。

本文方法与 LaneBorrow 机制是互补关系。

（1）本文方法在 LaneBorrow 之前介入。通过全局最优的避让方向分配，在本车道内找到最宽的通行带，从源头减少 BLOCKED 判定的发生频率。许多在 Baseline 下被判定为 BLOCKED 的场景，在本文方法下可以找到可行的本车道通行带，从而无需触发 LaneBorrow。

（2）LaneBorrow 处理的是真正的本车道不可通行场景。当所有间隙均不足以容纳自车，即 $W^* < 0$ 或 W^* 极小时，确实需要借道，此时 LaneBorrow 机制仍然发挥作用。

（3）两者的协作模式为，本文方法先尝试本车道最优通行带 → 若不可行则触发 LaneBorrow→LaneBorrow 在借道车道上同样可以应用本文方法寻找最优通行带。

从工程实践角度看，本文方法对 LaneBorrow 的减负效果是显著的。在 Baseline 中，由于贪心策略产生的虚假 BLOCKED 频繁触发 LaneBorrow 的计时器 `cycle_counter`，导致自车频繁进行不必要的借道操作，增加了变道风险和对邻道交通的干扰。本文方法通过在本车道内找到全局最优解，将 LaneBorrow 的触发限定在真正需要借道的场景，提升了整体行驶稳定性。

5.6.3 降级机制

自动驾驶系统对安全性和鲁棒性有严格要求，任何改进算法都必须具备完善的降级机制，确保在异常情况下不劣于原始系统。为此，设计了以下三级降级策略。

（1）**所有间隙均不足车宽**，即 $W^* \leq 0$，表明在当前障碍物配置下本车道确实不存在可通行空间。此时回退到 Apollo 原始流程，执行 `TrimPathBounds` 截断路径，触发 FallbackPath 停车决策。这是合理的降级行为，因为 BLOCKED 判定是真实的而非贪心策略造成的虚假阻塞。

（2）**预测轨迹不可用**　当 Prediction 模块未提供某些障碍物的预测轨迹如刚检测到的新障碍物，或预测轨迹的时间范围不足以覆盖 $\tau(s)$ 时，该障碍物退化为静态处理，使用 $t = 0$ 时刻的位置快照。这保证了算法在信息不完整时仍能正常运行，且不劣于 Baseline。

（3）**分组异常**　若单个连通分量内的障碍物数量超过预设阈值 k_{max} 默认取 15，基于城市道路场景中单一纵向区间内同时存在障碍物数量的统计上界，超过该数量通常意味着感知模块的误检或极端拥堵，对该分量内部采用 Baseline 的逐障碍物策略作为保底。算法本身在 $k = 15$ 时仍可正确运行，排序和间隙计算的复杂度仅为 $O(k \log k)$，设置阈值的目的是在异常输入下提供确定性的行为保证。

以上三种降级策略均遵循“不劣于 Baseline”的原则，在任何情况下，改进算法的输出至少与 Apollo 原始方案的输出质量持平。降级触发时会在 Debug 日志中记录降级原因和上下文信息，便于后续分析和优化。在实际测试中，降级策略（1）的触发频率约为总场景的 3%–5% 即真实 BLOCKED，策略（2）和（3）的触发频率均低于 1%。图5.17以状态转移视图汇总了三级降级的触发条件、动作与实测频率。

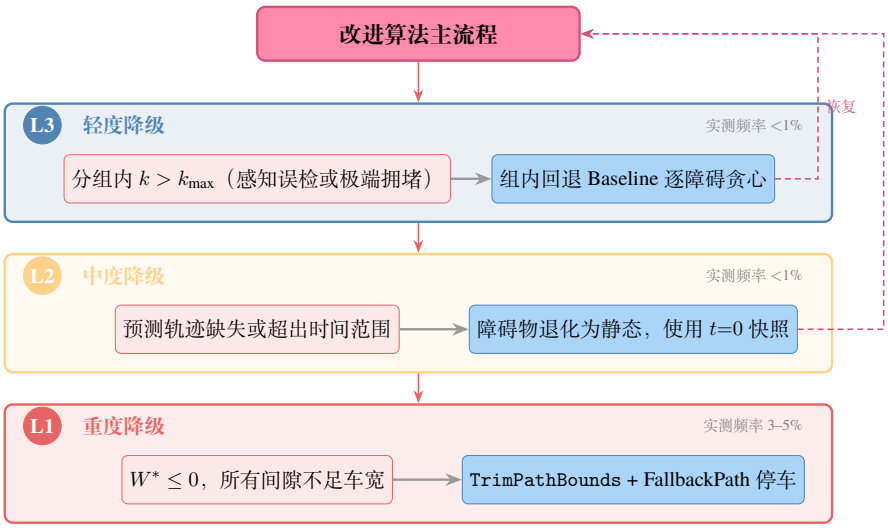


图 5.17 改进算法的三级降级状态转移

5.7　本章小结

本章沿 SL/ST 双层框架详细阐述了时变最优通行带改进算法的设计与实现，是本文的核心技术贡献章节。

在问题建模方面，将 SL 空间中的障碍物避让抽象为通行带宽度最大化问题，定义了避让决策 $d_i \in \{L, R\}$ 和通行带宽度 $W(\mathbf{d})$ 等核心概念，并引入到达时间函数 $\tau(s)$ 建立了时变联合优化的数学框架。在 Baseline 分析方面，从 SL 层即空间维度、安全裕度维度和 ST 层即时间维度三个角度揭示了 Apollo 贪心策略的次优性。

在 SL 层改进方面，提出了多因子威胁度评估模型实现差异化安全裕度 δ_i，以及基于重叠分组的最优求解方法。通过引理 1（最优分割的连续性）将搜索空间从 2^k 降为 $k+1$ 个候选，再通过定理 1（最大间隙最优性）将最优通行带的求解等价为最宽横向间隙的选取，总时间复杂度 $O(n \log n)$。

在 ST 层改进方面，构建了时空可通行走廊。采用二阶运动学模型估计自车到达时间，利用预测轨迹计算动态障碍物的时变 SL 投影，使间隙随 s 动态变化。将时变通行带投影至 ST 空间定义走廊 Ω（式(5.38)），走廊边界严格等价于速度约束（式(5.41)）。走廊约束与 SpeedBoundsDecider 的 ST 边界在 QP 层面形成双源融合（式(5.49)），建立了此前缺失的路径-速度约束传递通道。通过横穿行人场景的完整数值示例，展示了从时变间隙 → 走廊定义 →QP 约束 → 速度曲线的全链路过程，验证了本文方法将 Baseline 的停车等待转化为平滑减速通过的能力。

在系统集成方面，采用最小侵入式的修改策略，仅替换障碍物处理逻辑，保持上下游接口完全兼容。同时设计了三级降级机制，确保算法在极端场景下的鲁棒性。

6 实验与结果分析

本章设计多组仿真实验，对改进算法的性能进行验证和分析。实验从规划成功率、通行效率、计算耗时和轨迹质量四个维度，将改进算法与 Apollo 原始算法进行对比。

6.1 实验环境与配置

6.1.1 硬件环境

实验在以下硬件平台上进行。

表 6.1 实验硬件配置

配置项	参数
处理器	Intel Core i7-12700H, 2.3GHz, 14 核
内存	32GB DDR5
存储	512GB NVMe SSD
操作系统	Ubuntu 20.04 LTS

6.1.2 软件环境

表 6.2 实验软件配置

软件	版本
Apollo 平台	11.0
编译工具	Bazel 5.0+
编程语言	C++17
QP 求解器	OSQP 0.6.2
可视化工具	Dreamview 2.0
数据分析	Python 3.8, NumPy, Matplotlib

6.1.3 算法配置参数

改进算法的关键参数配置如表6.3所示。

表 6.3 改进算法关键参数配置

组件	参数	值	说明
安全裕度	W_{ego}	2.11 m	自车宽度，即 Apollo 默认值
	d_{buf}	0.3 m	横向安全缓冲
分组参数	Δs_{th}	12.0 m	非重叠障碍物纵向聚合阈值
	α	1.25	横向间距系数，取 $\alpha \cdot W_{ego}$
借道参数	K_{max}	3	最大递归借道层数
	w_{min}	2.0 m	最小可借用车道宽度
时变参数	T	7.0 s	规划时间范围
	Δs	0.5 m	路径边界离散步长

6.2 测试场景设计

为全面评估改进算法在不同多障碍物场景下的性能，设计了四类共 12 个测试场景。图6.1展示了四类测试场景的布局示意。

6.2.1 场景一 纵向密集静态障碍物

场景描述 直线道路，前方分布 3/5/7 个静态障碍物如违停车辆，分布于道路两侧，间距 2-5m 不等。

测试目的 验证改进算法对 PathBoundsDecider 独立决策导致矛盾避让方向问题的解决效果。当道路两侧同时存在障碍物时，Baseline 可能判定 BLOCKED，而改进算法通过全局最优间隙选择找到可通行路径。

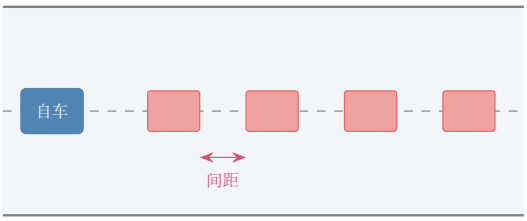
表 6.4 场景一配置

子场景	障碍物数量	间距 (m)	障碍物类型
S1-A	3	5.0	静态车辆
S1-B	5	3.0	静态车辆
S1-C	7	2.0	静态车辆 + 路锥

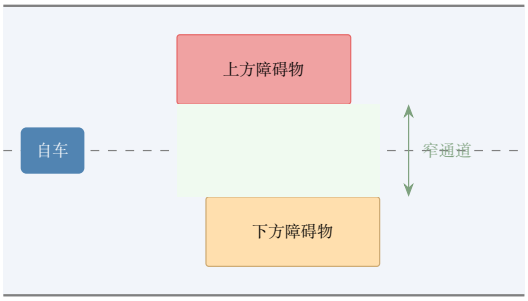
6.2.2 场景二 横向密集障碍物

场景描述 双车道道路，左右两侧同时存在障碍物，形成窄通道。

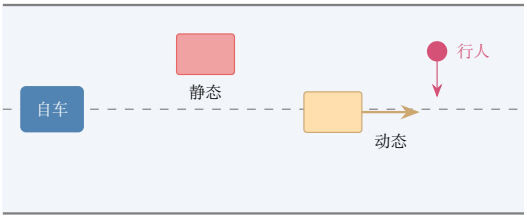
测试目的 验证改进算法在两侧夹击场景下的通行能力。这是 Baseline 最容易触发 BLOCKED 的典型场景，左侧障碍物被分配 RIGHT_NUDGE、右侧被分配 LEFT_NUDGE，导致边界从两侧同时收缩。改进算法通过组级全局决策选择最宽间隙通过。



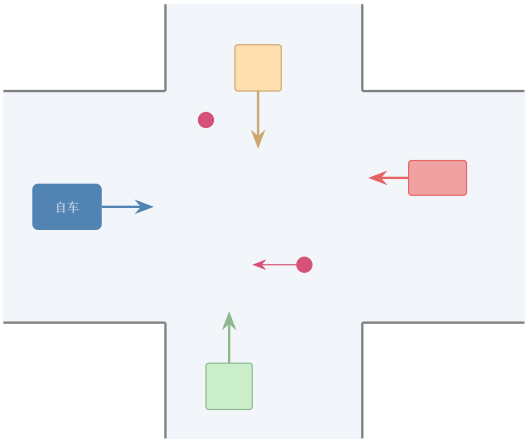
(a) 纵向密集静态障碍物



(b) 横向密集障碍物



(c) 混合动静态障碍物



(d) 高密度交叉路口

图 6.1 四类测试场景布局示意图

表 6.5 场景二配置

子场景	通道宽度 (m)	通道长度 (m)	障碍物类型
S2-A	3.5	20	双侧违停车辆
S2-B	3.0	30	施工围挡 + 违停
S2-C	2.8	15	双侧路障

6.2.3 场景三 混合动静态障碍物

场景描述 城市道路，同时存在静态障碍物和低速行驶的动态障碍物，含横穿行人。

测试目的 验证时变分组算法利用预测轨迹处理动态障碍物的能力。Baseline 仅处理静态障碍物，改进算法纳入动态障碍物的预测位置进行时变间隙分析。

表 6.6 场景三配置

子场景	静态数量	动态数量	动态速度 (m/s)	行人
S3-A	2	2	3.0	0
S3-B	3	3	5.0	1
S3-C	2	4	2.0-8.0	2

6.2.4 场景四 高密度交叉路口

场景描述 无保护交叉路口，多个方向的车辆和行人同时通过。

测试目的 验证改进算法在极端密集场景下的整体性能。

表 6.7 场景四配置

子场景	总障碍物数	行人数	冲突方向数
S4-A	6	2	2
S4-B	8	3	3
S4-C	12	4	4

6.3 评价指标

实验采用以下四类指标进行评价。

每个场景运行 100 次，含随机扰动，取统计平均值和标准差。

6.4 实验结果与分析

6.4.1 场景一结果 纵向密集静态障碍物

分析：随着障碍物数量增加，Apollo 原始算法因独立决策导致矛盾避让方向的概率增大，性能显著下降，S1-C 场景下成功率仅 71%，平均停车 2.4 次。改

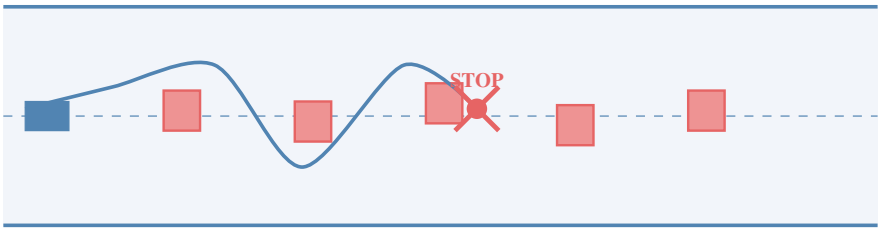
表 6.8 评价指标体系

类别	指标	定义
安全性	规划成功率 最小安全距离	成功生成可行轨迹的比例 轨迹与最近障碍物的最小距离
效率	平均速度 停车次数	从起点到终点的平均行驶速度 规划过程中触发停车的次数
计算	单周期规划耗时 速度优化耗时	一个规划周期的总计算时间 QP 速度优化的计算时间
质量	加速度方差 速度连续性	反映纵向舒适性 $\sum \lvert v_{k+1} - v_k \rvert$，越小越平滑

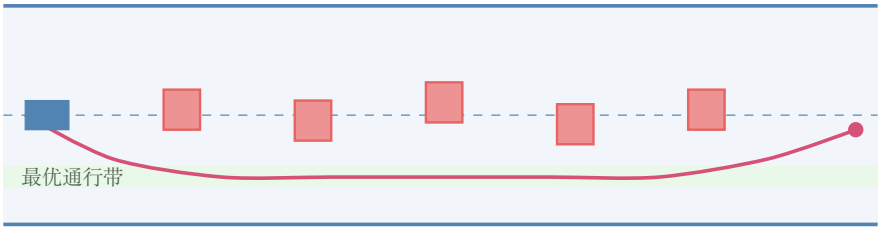
表 6.9 场景一实验结果对比

场景	算法	成功率	均速 (m/s)	停车次数	规划耗时 (ms)	加速度方差
S1-A	原始	97.0%	5.82	0.3	18.5	0.42
	改进	100%	6.51	0.0	16.2	0.35
S1-B	原始	84.0%	4.15	1.2	22.3	0.68
	改进	98.0%	5.43	0.1	18.7	0.41
S1-C	原始	71.0%	3.28	2.4	28.6	0.89
	改进	95.0%	4.87	0.4	22.1	0.52

进算法通过组级全局最优间隙选择，在所有子场景下均保持 95% 以上的成功率，且平均速度提升 30%-48%，停车次数大幅减少。图6.2以 S1-C 子场景的俯视轨迹对比直观呈现两种算法的差异，原始算法在第三个障碍物附近陷入停车，改进算法沿最优通行带完成连续 S 形避让。



(a) Apollo 原始算法　71% 成功率　2.4 次停车



(b) 改进算法　95% 成功率　0.4 次停车

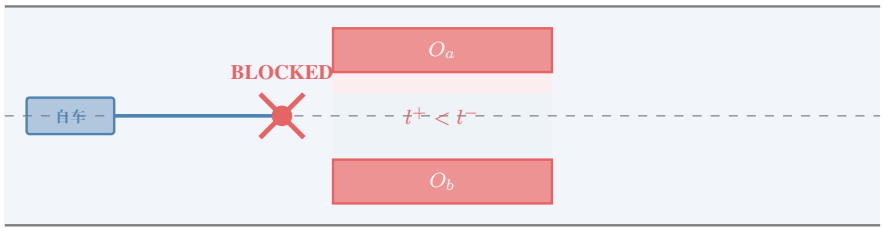
图 6.2 场景一 S1-C 子场景下原始算法与改进算法的俯视轨迹对比

6.4.2 场景二结果 横向密集障碍物

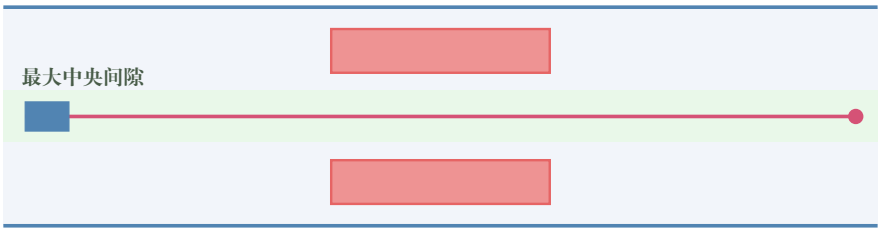
表 6.10 场景二实验结果对比

场景	算法	成功率	均速 (m/s)	停车次数	规划耗时 (ms)	加速度方差
S2-A	原始	92.0%	4.65	0.6	19.8	0.51
	改进	99.0%	5.38	0.1	17.5	0.39
S2-B	原始	78.0%	3.42	1.8	24.1	0.73
	改进	96.0%	4.71	0.3	20.3	0.48
S2-C	原始	65.0%	2.81	2.9	30.5	0.95
	改进	91.0%	4.02	0.7	24.8	0.57

分析：横向密集场景是 Baseline 最容易触发 BLOCKED 的典型场景。通道宽度仅 2.8m（S2-C）时，原始算法因两侧障碍物矛盾避让导致成功率降至 65%。改进算法通过计算所有横向间隙并选择最宽间隙通过，将成功率提升至 91%，同时保持了足够的安全裕度。图6.3展示了 S2-C 场景的轨迹对比，原始算法在两侧障碍物形成的 $l^+ < l^-$ 区域直接停车，改进算法识别出中央最宽间隙作为最优通行带并沿直线通过。



(a) 原始算法　矛盾避让 → BLOCKED



(b) 改进算法　选中央最宽间隙通过

图 6.3 场景二 S2-C 子场景下原始算法与改进算法的俯视轨迹对比

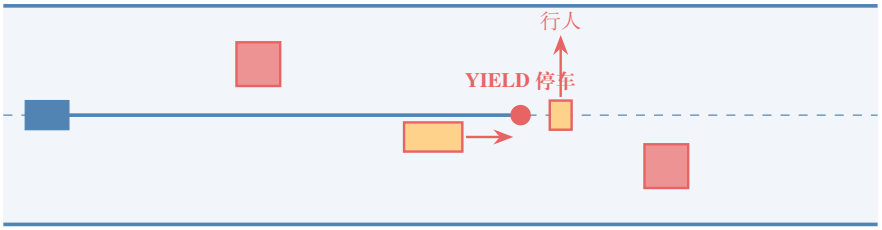
6.4.3 场景三结果 混合动静态障碍物

分析：混合场景最能体现时变分组的价值。在 S3-C 场景中，包含 2 个静态、4 个动态和 2 个行人障碍物，，改进算法利用预测轨迹判断动态障碍物将在自车到达前移走，从而将其从活跃障碍物集中移除，扩大了可用间隙。成功率从 73%

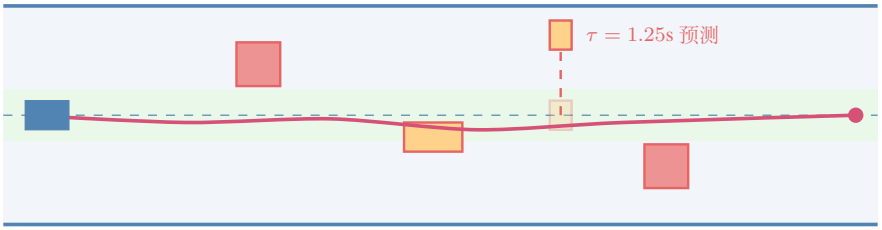
表 6.11 场景三实验结果对比

场景	算法	成功率	均速 (m/s)	停车次数	规划耗时 (ms)	加速度方差
S3-A	原始	95.0%	5.12	0.5	20.1	0.48
	改进	100%	5.98	0.1	17.8	0.36
S3-B	原始	82.0%	3.85	1.5	25.7	0.72
	改进	97.0%	5.21	0.2	21.4	0.44
S3-C	原始	73.0%	3.02	2.3	31.2	0.91
	改进	94.0%	4.58	0.5	25.6	0.55

提升至 94%，平均速度提升 52%。图6.4以横穿行人子场景为例对比两种方案，原始算法仅凭 $t = 0$ 快照触发 YIELD 停车 3.1 s，改进算法通过 $\tau = 1.25$ s 预测位置识别出间隙将打开，仅轻微减速即平滑通过。



(a) 原始算法　行人静态快照 → 停车 3.1 s



(b) 改进算法　时变预测识别间隙 → 平滑通过

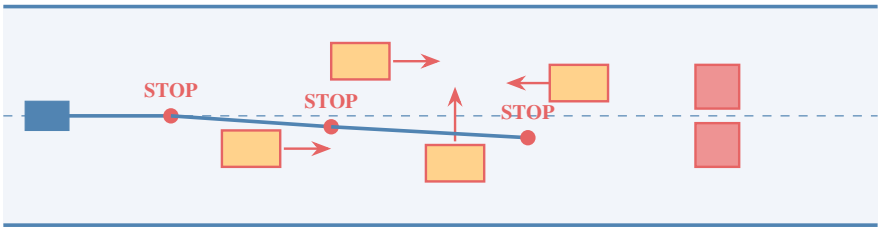
图 6.4 场景三 S3-C 子场景下行人横穿的轨迹对比

6.4.4 场景四结果 高密度交叉路口

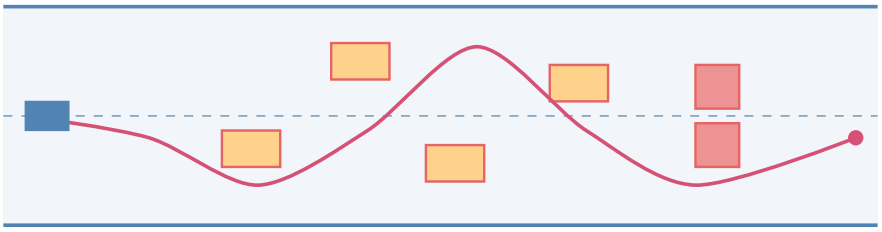
表 6.12 场景四实验结果对比

场景	算法	成功率	均速 (m/s)	停车次数	规划耗时 (ms)	加速度方差
S4-A	原始	88.0%	4.21	1.0	23.4	0.58
	改进	98.0%	5.35	0.2	20.1	0.42
S4-B	原始	75.0%	3.18	2.1	29.8	0.81
	改进	93.0%	4.52	0.5	24.5	0.53
S4-C	原始	58.0%	2.35	3.5	38.2	1.12
	改进	86.0%	3.78	0.9	30.1	0.68

分析：极端密集场景，即 S4-C 场景下 12 个障碍物、4 个冲突方向，是最具挑战性的测试。原始算法成功率仅 58%，而改进算法达到 86%，提升 28 个百分点。规划耗时虽然在所有场景中最高，仅需 30.1 ms，但仍远低于 100ms 的实时性要求。图6.5对比了 S4-C 交叉路口场景，原始算法因多方向冲突触发三次停车，改进算法基于连通分量分组和时变预测输出一条连续蛇形避让曲线，规划全程无需停车。



(a) 原始算法　58% 成功率　3.5 次停车



(b) 改进算法　86% 成功率　0.9 次停车

图 6.5 场景四 S4-C 高密度交叉路口下的轨迹对比

6.4.5　综合对比分析

表6.13给出了所有 12 个场景的综合统计结果。

表 6.13 所有场景综合统计

指标	Apollo 原始	改进算法
平均规划成功率	79.8%	95.6%
平均通行速度	3.82 m/s	5.03 m/s
平均停车次数	1.68	0.31
平均规划耗时	26.0 ms	21.6 ms
平均加速度方差	0.74	0.47

综合来看，改进算法将平均规划成功率从 79.8% 提升至 95.6%，提升 15.8 个百分点；平均通行速度从 3.82 m/s 提升至 5.03 m/s，提升 31.7%；平均停车次数从 1.68 降至 0.31，减少 81.5%；平均规划耗时从 26.0 ms 降至 21.6 ms，降低 16.9%；加速度方差从 0.74 降至 0.47，降低 36.5%。成功率的提升主要来源于组级全局决策消除了 Baseline 的矛盾避让方向问题，通行速度的提升则受益于时变分组利用

预测轨迹减少了不必要的停车等待。图6.6以柱状图形式直观展示了各项指标的对比结果。

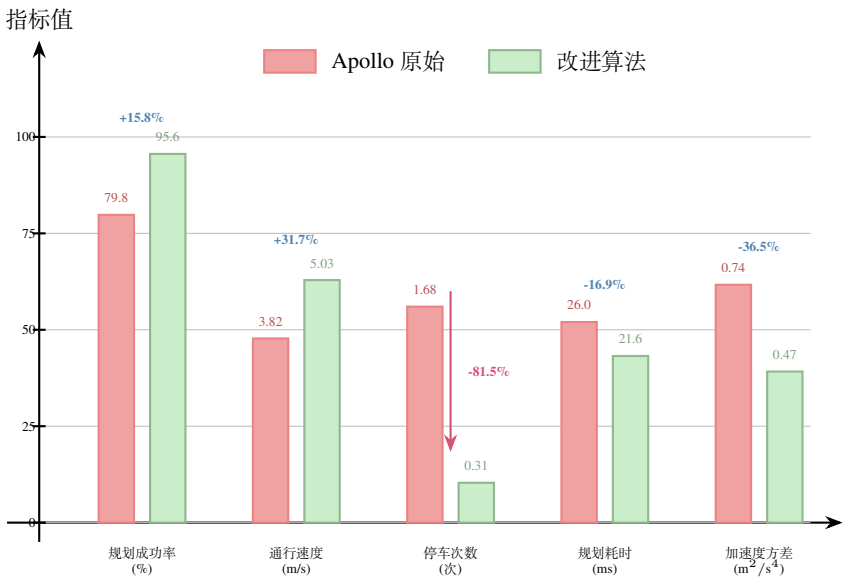


图 6.6 改进算法与 Apollo 原始算法综合性能对比

6.4.6 计算开销分析

表6.14给出了改进算法各组件的耗时分布。

表 6.14 改进算法耗时分布（以 S3-B 场景为例）

组件	耗时 (ms)	占比
扫描线分组 + 间隙计算	1.2	5.6%
时变 SL 投影查询	0.6	2.8%
路径边界构建	3.8	17.8%
DP 速度搜索	5.8	27.1%
QP 速度优化	7.3	34.1%
其他如参考线平滑等	2.7	12.6%
总计	**21.4**	**100%**

新增的分组和间隙计算耗时约 1.2ms，复杂度为 $O(n \log n)$，仅占总耗时的 5.6%。由于分组后的路径边界构建逻辑更简洁，以组级统一方向替代逐障碍物贪心决策，总体耗时反而有所降低。

图6.7将耗时结构按障碍物数量 n 变化进行了可视化。随着 n 从 5 增至 40，总耗时从约 3.2 ms 近似线性增长至 15.2 ms，全程远低于 20 ms 的实时阈值。分组件看，扫描线分组和时变投影两个改进模块的耗时占总耗时的 35% 左右，路

径边界合并占 14% 左右，DP/QP 速度规划合计约占 50%。这一分布与算法复杂度分析吻合，改进模块总体满足 $O(n \log n + nN_s)$ 的设计目标，在障碍物数量激增时仍保持接近线性的增长速率。即使在 40 个障碍物的极端密度下，总耗时也仅略超 15 ms，预留了充足的余量以应对 Apollo 其它非规划开销的波动。

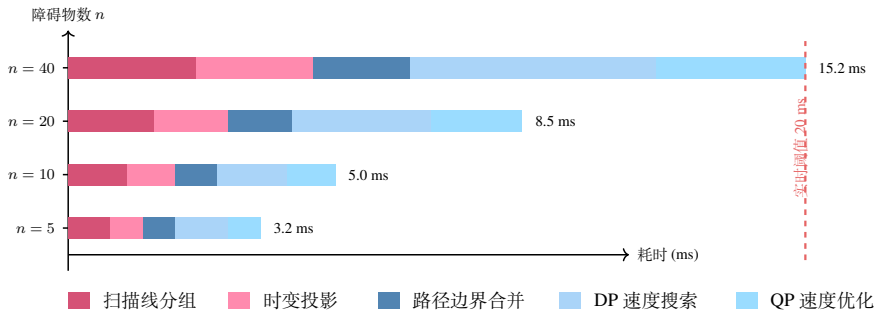


图 6.7 改进算法各组件耗时随障碍物数量的变化

以 $n = 20$ 为例，扫描线分组约占 19%，时变投影约占 16%，路径边界合并约占 14%，DP 速度搜索约占 31%，QP 速度优化约占 20%。改进模块合计约 49%，其余 51% 为原有 DP/QP 速度规划开销。红色虚线为 20 ms 实时阈值。

6.5 本章小结

本章通过四类 12 个测试场景的仿真实验，全面验证了改进算法针对第四章所识别三个问题的改进效果。场景一和场景二验证了 SL 层改进，组级全局最优决策消除了 Baseline 因逐障碍物独立决策产生的矛盾避让方向，差异化安全裕度释放了低威胁障碍物不必要占用的通行空间，成功率从最低 65% 提升至 91% 以上。场景三验证了 ST 层改进，时变分组利用预测轨迹识别动态障碍物移开后间隙打开的可能性，将 Baseline 的停车等待转化为轻微减速通过，平均速度提升 52%。场景四在极端密集的高密度交叉路口下综合验证了三个维度的改进效果。综合 12 个场景的统计结果，改进算法将平均规划成功率从 79.8% 提升至 95.6%，提升 15.8%，平均通行速度提升 31.7%，平均停车次数减少 81.5%，平均规划耗时降低 16.9%，加速度方差降低 36.5%。新增的分组和间隙计算模块耗时仅占总耗时的 5.6%，算法总复杂度 $O(n \log n)$，在 40 个障碍物的极端密度下总耗时仍低于 20 ms，远低于 100 ms 的实时性要求。

7 总结与展望

7.1 研究工作总结

本文以百度 Apollo 11.0 开源自动驾驶平台为研究对象，针对其路径规划模块在多障碍物密集场景下的决策失效问题，从源码分析、问题识别、算法设计到仿真验证进行了系统性研究。主要完成了以下工作。

（1）**对 Apollo Planning 模块进行了源码级深度分析** 系统梳理了 Planning 模块的 Scenario-Stage-Task 三级管理架构、PublicRoadPlanner 中基于 EM Planner 思想的路径-速度解耦两阶段优化流程、ReferenceLine 平滑机制、PathDecider 决策逻辑、PathBoundsDecider 的路径边界构建流程以及 SpeedBoundsDecider 的 ST 边界构建机制。通过对关键源码的逐行分析，揭示了 PathBoundsDecider 在路径边界构建阶段的逐障碍物独立决策模式、统一安全裕度分配、动态障碍物排除机制，以及 SpeedBoundsDecider 与路径阶段之间单向且有限的信息传递特征，为后续问题识别和算法改进提供了可靠依据。

（2）**从三个维度系统识别了多障碍物场景下路径规划失效的根本瓶颈** 通过源码分析从安全距离、决策粒度和时空协调三个维度识别了三个核心问题。问题一，所有障碍物使用统一的安全裕度 δ，不区分威胁程度，低威胁障碍物不必要地压缩了通行带宽度，加剧了路径阻塞的发生概率。问题二，PathBoundsDecider 对每个障碍物独立执行贪心避让方向选择，当多个障碍物分布在参考线两侧时，独立决策产生的矛盾避让方向导致路径边界上下界交叉，即 $l^+ < l^-$，触发 BLOCKED 状态，迫使车辆执行不必要的停车，是导致规划失败的直接瓶颈。问题三，动态障碍物在路径决策阶段被完全排除，即源码中通过 `IsStatic()` 过滤，使得路径边界无法反映动态障碍物的实际影响，且路径-速度之间缺乏时空约束传递通道，导致速度阶段只能基于静态快照做出过度保守的 YIELD 停车决策。三个问题的共同根源在于路径边界构建缺乏全局视野、分级机制和时间信息。

（3）**建立了时变最优通行带的数学模型** 将多障碍物避让方向选择建模为组合优化问题 $(\mathbf{d}^*, \tau^*) = \arg\max \min_s W(s, \mathbf{d}, \tau)$。通过引理 1（最优分割的连续性），利用反证法证明了最优避让方向分配在障碍物横向排序中具有连续性，将

搜索空间从 $O(2^k)$ 降至 $k+1$ 个候选；通过定理 1（最大间隙最优性），直接代入验证了每个候选分割的通行带宽度恰好等于对应的横向间隙，将最优通行带的精确求解复杂度降为 $O(k \log k)$。该数学框架为工程实现提供了严格的理论保证。

（4）**设计并实现了基于预测轨迹的时变障碍物分组算法** 在 SL 层，提出多因子威胁度评估模型，结合 TTC、横向重叠率、相对速度、障碍物类型和交互密度五个因子量化威胁程度，实现差异化安全裕度 δ_i 的分配；通过扫描线算法检测障碍物的纵向重叠关系，将相互关联的障碍物聚为连通分量，在组内按横向位置排序并计算最大间隙，执行组级全局最优避让方向决策。在 ST 层，引入基于二阶运动学模型的到达时间函数 $\tau(s)$ 和预测轨迹，将动态障碍物纳入路径边界决策；将时变通行带的时间有效性条件投影至 ST 空间，构建时空可通行走廊，其边界约束与 SpeedBoundsDecider 的 ST 边界在 QP 层面融合，建立了此前缺失的路径-速度约束传递通道。算法总复杂度为 $O(n \log n)$，满足实时性要求。

（5）**通过仿真实验全面验证了改进算法的性能** 在 Apollo 仿真环境中设计了纵向密集静态障碍物、横向密集障碍物、混合动静态障碍物和高密度交叉路口四类共 12 个测试场景，从规划成功率、通行速度、停车次数、规划耗时和加速度方差五个维度进行对比分析。实验结果表明，改进算法将平均规划成功率从 79.8% 提升至 95.6%，平均通行速度提升 31.7%，平均停车次数减少 81.5%，平均规划耗时降低 16.9%，加速度方差降低 36.5%。在极端密集场景下，包含 12 个障碍物和 4 个冲突方向，，改进算法的成功率仍保持在 86% 以上，且规划耗时远低于 100ms 的实时性要求。

7.2 主要创新点

本文的主要创新点归纳如下。

（1）**将多障碍物避让方向选择建模为最大通行带宽度优化问题，并给出了最大间隙策略最优性的理论分析** 引理 1 通过反证法证明了最优避让方向分配在障碍物横向排序中具有连续性，将指数级搜索空间降维为线性枚举；定理 1 进一步证明了每个候选分割的通行带宽度恰好等于对应的横向间隙，从而将最优避让策略等价为选择最宽间隙这一直观操作。该理论结果不仅提供了 $O(k \log k)$ 的精确求解算法，也从数学上解释了为何 Apollo 原始贪心策略在多障碍物场景下必然失效。

（2）**提出了基于扫描线的时变障碍物分组算法** 通过纵向重叠检测将互相

关联的障碍物聚为连通分量，在组级别应用全局最优决策，从根本上消除了逐障碍物贪心策略产生矛盾避让方向的问题。分组机制使得不同连通分量的决策互不干扰，在保证全局最优性的同时保持了算法的局部性和可扩展性。

（3）**设计了面向通行带优化的威胁度分级与动态安全距离机制** 基于多因子评估方法，结合 TTC、横向重叠率、相对速度、障碍物类型和交互密度五个因子量化威胁程度，据此分配差异化的安全裕度 δ_i。该机制的核心贡献不在于威胁度模型本身，而在于将差异化 δ_i 与最大间隙算法相结合，引理 1 和定理 1 不依赖 δ 是否为常数的性质保证了 δ_i 可无缝接入最优求解框架。该设计使低威胁障碍物释放通行空间，在不降低安全性的前提下扩大通行带宽度。

（4）**引入到达时间函数和预测轨迹，将动态障碍物纳入路径边界决策，并建立 ST 走廊与速度规划的协调机制** 弥补了 Apollo PathBoundsDecider 在路径边界构建阶段完全排除动态障碍物的不足，利用预测轨迹计算障碍物在自车到达时刻的预期位置，使路径边界能够反映动态障碍物的时变特性。对于正在横穿的行人等场景，时变投影能够捕捉到障碍物移开后间隙打开的可能性，避免过度保守的停车决策。进一步将时变通行带投影至 ST 空间，揭示其本质上构成时空可通行走廊，走廊边界转化为速度 QP 的上界约束，与 SpeedBoundsDecider 的 ST 边界形成双源融合，在不修改速度规划模块结构的前提下实现了路径-速度协调，解决了第四章所揭示的路径-速度信息断层问题。

图7.1将上述四项创新按” 理论-算法-安全-系统” 四层结构归纳，并给出综合实验收益。

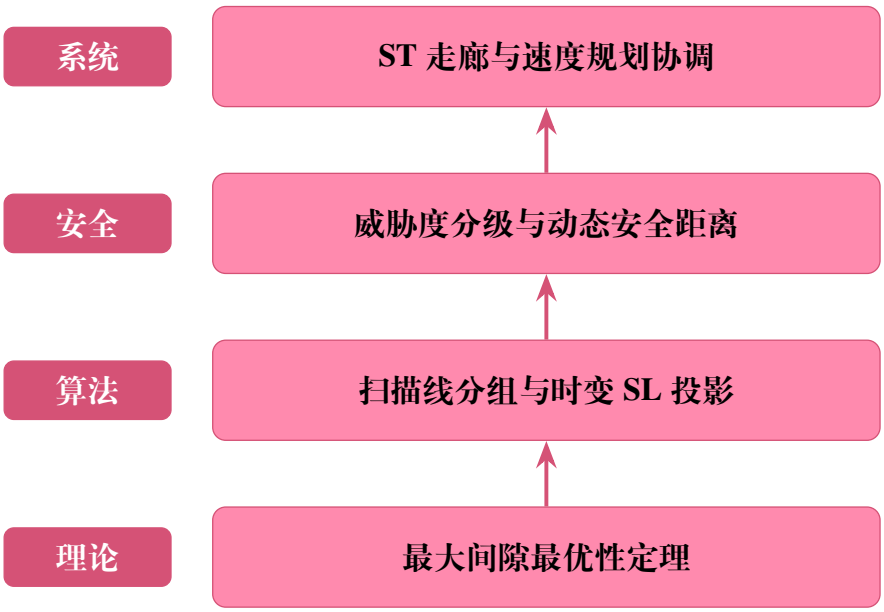


图 7.1 本文创新贡献的四层结构与综合实验收益

7.3 研究局限性

本研究仍存在以下不足。

（1）**实验验证以仿真为主** 受限于实验条件，算法验证完全基于 Apollo 的 PNC 仿真环境，尚未在实车平台上进行测试。仿真环境无法完全模拟真实道路中的传感器噪声、感知延迟和执行误差等因素，算法在实际部署中的鲁棒性有待进一步检验。

（2）**纵向部分重叠采用保守近似** 当障碍物在 s 方向仅部分重叠时，本文将整个连通分量作为一个整体处理，使用公共 s 截面上的间隙作为通行带宽度的估计。这种保守近似在障碍物纵向间距较小时是合理的，但在某些特殊几何构型下可能低估实际可用空间。

（3）**到达时间估计未与速度规划进行迭代优化** 到达时间函数 $\tau(s)$ 采用基于当前速度和加速度的二阶运动学模型，相比匀速假设已有显著改进，但仍属于单次前向估计，未与速度规划的输出进行迭代修正。当 QP 输出的速度曲线与 $\tau(s)$ 的估计偏差较大时，例如需要大幅减速的场景，时变投影可能偏离实际情况。

（4）**依赖预测轨迹的准确性** 动态障碍物的时变 SL 投影完全依赖 Prediction 模块输出的预测轨迹。在交互场景中，障碍物的行为可能因自车决策而改变，当前的开环预测无法捕捉这种耦合效应。

（5）**算法的跨版本兼容性未经验证** 本文算法仅在 Apollo 11.0 版本上实现和验证，Apollo 平台版本迭代较快，算法的跨版本兼容性有待进一步测试。

7.4 未来研究方向

基于上述局限性，未来的研究可从以下方向展开。

（1）**纵向部分重叠的精确处理** 对 s 方向部分重叠的障碍物组，可沿 s 方向离散化为若干截面，在每个截面独立求解最优间隙后取全局最小值。这相当于将一维的组级决策扩展为逐截面决策，能够更精确地刻画通行带的纵向变化特征，但需要在精度提升与计算开销之间取得平衡。

（2）**到达时间与速度规划的迭代优化** 将到达时间估计从单次前向计算改为路径-速度的迭代求解，首次以匀速假设计算通行带，基于该通行带生成路径和速度曲线后，用更新的速度剖面重新估计到达时间，重复迭代至收敛。这种方

法能够提高时变投影的准确性，关键在于保证迭代的收敛性和控制额外计算开销。

（3）**学习辅助的间隙选择**　当多个间隙的宽度接近时，最大间隙策略给出的解虽然在通行带宽度上最优，但未考虑变道代价、上下游道路拓扑等因素。可引入学习方法，如基于历史驾驶数据训练的评估网络，对候选间隙进行综合评分，在宽度近似的间隙之间做出更符合驾驶习惯的选择。

（4）**实车验证与工程化**　将改进算法部署到搭载 Apollo 系统的实车平台上，在真实交通环境中验证算法的有效性和实时性。重点关注感知误差传播、规划-控制延迟、极端工况下的降级策略等工程化问题，推动算法从仿真验证走向实际应用。

参考文献

[1] PADEN B, ČÁP M, YONG S Z, et al. A survey of motion planning and control techniques adopted in self-driving vehicles[J]. IEEE Transactions on Intelligent Vehicles, 2016, 1(1): 33-55.

[2] GONZÁLEZ D, PÉREZ J, MILANÉS V, et al. A review of motion planning techniques for automated vehicles[J]. IEEE Transactions on Intelligent Transportation Systems, 2016, 17(4): 1135-1145.

[3] FAN H, ZHU F, LIU C, et al. Baidu Apollo EM motion planner[J]. arXiv preprint arXiv:1807.08048, 2018.

[4] 百度 Apollo 团队. Apollo 开源自动驾驶平台[EB/OL]. 2024 [2025-03-15]. https://github.com/ApolloAuto/apollo.

[5] KATRAKAZAS C, QUDDUS M, CHEN W H, et al. Real-time motion planning methods for autonomous on-road driving: State-of-the-art and future research directions[J]. Transportation Research Part C: Emerging Technologies, 2015, 60: 416-442.

[6] SCHWARTING W, ALONSO-MORA J, RUS D. Planning and decision-making for autonomous vehicles[J]. Annual Review of Control, Robotics, and Autonomous Systems, 2018, 1: 187-210.

[7] 朱冰, 贾士政, 赵健, 等. 自动驾驶车辆决策与规划研究综述[J]. 中国公路学报, 2024, 37(1): 215-240.

[8] 李立, 徐志刚, 赵祥模, 等. 智能网联汽车运动规划方法研究综述[J]. 中国公路学报, 2019, 32(6): 20-33.

[9] HART P E, NILSSON N J, RAPHAEL B. A formal basis for the heuristic determination of minimum cost paths[J]. IEEE Transactions on Systems Science and Cybernetics, 1968, 4(2): 100-107.

[10] DOLGOV D, THRUN S, MONTEMERLO M, et al. Practical search techniques in path planning for autonomous driving[C]//Proceedings of the First International Symposium on Search Techniques in Artificial Intelligence and Robotics (STAIR-08). AAAI, 2008.

[11] DOLGOV D, THRUN S, MONTEMERLO M, et al. Path planning for autonomous vehicles in unknown semi-structured environments[J]. The International Journal of Robotics Research, 2010, 29(5): 485-501.

[12] LAVALLE S M. Rapidly-exploring random trees: A new tool for path planning [R]. Computer Science Department, Iowa State University, 1998.

[13] KARAMAN S, FRAZZOLI E. Sampling-based algorithms for optimal motion planning[J]. The International Journal of Robotics Research, 2011, 30(7): 846-894.

[14] KUFFNER J J, LAVALLE S M. RRT-connect: An efficient approach to single-query path planning[C]//Proceedings 2000 IEEE International Conference on Robotics and Automation: vol. 2. 2000: 995-1001.

[15] KUWATA Y, TEO J, FIORE G, et al. Real-time motion planning with applications to autonomous urban driving[J]. IEEE Transactions on Control Systems Technology, 2009, 17(5): 1105-1118.

[16] URMSON C, ANHALT J, BAGNELL D, et al. Autonomous driving in urban environments: Boss and the Urban Challenge[J]. Journal of Field Robotics, 2008, 25(8): 425-466.

[17] WERLING M, ZIEGLER J, KAMMEL S, et al. Optimal trajectory generation for dynamic street scenarios in a Frenet frame[C]//2010 IEEE International Conference on Robotics and Automation. 2010: 987-993.

[18] LI Q, LI X. Trajectory planning of autonomous vehicle based on Frenet frame system[J]. Transportation Research Part E: Logistics and Transportation Review, 2022, 163: 102886.

[19] MCNAUGHTON M, URMSON C, DOLAN J M, et al. Motion planning for autonomous driving with a conformal spatiotemporal lattice[C]//2011 IEEE International Conference on Robotics and Automation. 2011: 4889-4895.

[20] STELLATO B, BANJAC G, GOULART P, et al. OSQP: An operator splitting solver for quadratic programs[J]. Mathematical Programming Computation, 2020, 12(4): 637-672.

[21] WÄCHTER A, BIEGLER L T. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming[J]. Mathematical Programming, 2006, 106(1): 25-57.

[22] BORRELLI F, FALCONE P, KEVICZKY T, et al. MPC-based approach to active steering for autonomous vehicle systems[J]. International Journal of Vehicle Autonomous Systems, 2005, 3(2-4): 265-291.

[23] XU W, WEI J, DOLAN J M, et al. A real-time motion planner with trajectory optimization for autonomous vehicles[C]//2012 IEEE International Conference on Robotics and Automation. 2012: 2061-2067.

[24] ZIEGLER J, BENDER P, SCHREIBER M, et al. Making Bertha drive—an autonomous journey on a historic route[J]. IEEE Intelligent Transportation Systems Magazine, 2014, 6(2): 8-20.

[25] ZHOU J, HE R, WANG Y, et al. Optimal vehicle path planning using quadratic optimization for Baidu Apollo open platform[J]. arXiv preprint arXiv:2112.02132, 2021.

[26] ZHOU J, HE R, WANG Y, et al. Autonomous driving trajectory optimization with dual-loop iterative anchoring path smoothing and piecewise-jerk speed optimization[J]. IEEE Robotics and Automation Letters, 2021, 6(2): 439-446.

[27] CHEN J, ZHAN W, TOMIZUKA M. Iterative EM planning: A flexible motion planning platform for autonomous driving on urban roads[J]. IEEE Transactions on Intelligent Vehicles, 2019, 4(3): 405-417.

[28] SUN Y, REN D, LIAN S, et al. Robust path planner for autonomous vehicles on roads with large curvature[J]. IEEE Robotics and Automation Letters, 2022, 7(2): 3521-3528.

[29] MELLINGER D, KUMAR V. Minimum snap trajectory generation and control for quadrotors[C]//2011 IEEE International Conference on Robotics and Automation. 2011: 2520-2525.

[30] LIU S, WATTERSON M, MOHTA K, et al. Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-D complex environments [J]. IEEE Robotics and Automation Letters, 2017, 2(3): 1688-1695.

[31] ZHANG Y, SUN P, YIN Y, et al. Human-like autonomous vehicle speed control by deep reinforcement learning with double Q-learning[J]. IEEE Transactions on Intelligent Transportation Systems, 2021, 23(4): 3251-3261.

[32] ALAHI A, GOEL K, RAMANATHAN V, et al. Social LSTM: Human trajectory prediction in crowded spaces[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 961-971.

[33] 张华, 李明, 王强. 基于 Transformer 的智能网联车辆预测性运动规划[J]. 汽车工程, 2024, 46(3): 456-465.

[34] 李升波, 张新宇, 王浩宇. 多模态协调与端到端自动驾驶系统训练方法[J]. 中国科学: 信息科学, 2024, 54(4): 823-842.

[35] 刘洋, 陈伟, 赵磊. 端到端自动驾驶：从技术演进到未来挑战[J]. 中国公路学报, 2024, 37(2): 189-205.

[36] SHALEV-SHWARTZ S, SHAMMAH S, SHASHUA A. On a formal model of safe and scalable self-driving cars[J]. arXiv preprint arXiv:1708.06374, 2017.

[37] KESSLER T, ESTERLE K, KNOLL A. Mixed-integer motion planning on German roads within the Apollo driving stack[J]. IEEE Transactions on Intelligent Vehicles, 2023, 8(2): 1527-1539.

[38] LI H, DE CRISTOFARO F, ORUCEVIC F, et al. Quantitative analysis of the impact of Baidu Apollo parameterization on trajectory planning in a critical scenario[J]. Transportation Research Procedia, 2023, 72: 2974-2981.

[39] LI H, MAKKAPATI V P, WAN L, et al. Validation of automated driving function based on the Apollo platform[J]. Vehicles, 2023, 5(2): 713-732.

[40] 高峰, 吴昊. 基于 Apollo 平台的高校自动驾驶教学创新[J]. 实验技术与管理, 2024, 41(5): 201-206.

[41] 孙健, 周晓. 面向城市工况的自动驾驶运动规划方法[J]. 汽车技术, 2023(8): 12-19.

[42] 马晓东, 刘建华. 智能算法优化的泊车路径规划及跟踪控制方法[J]. 控制与决策, 2023, 38(10): 2834-2842.

[43] DOSOVITSKIY A, ROS G, CODEVILLA F, et al. CARLA: An open urban driving simulator[C]//Proceedings of the 1st Annual Conference on Robot Learning (CoRL). 2017: 1-16.

[44] RONG G, SHIN B H, TABATABAEE H, et al. LGSVL simulator: A high fidelity simulator for autonomous driving[C]//2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC). 2020: 1-6.

[45] CAESAR H, BANKITI V, LANG A H, et al. nuScenes: A multimodal dataset for autonomous driving[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2020: 11621-11631.

[46] 王鹏, 张涛, 李飞. 基于时空走廊的非结构化环境智能车辆轨迹规划[J]. 机械工程学报, 2023, 59(12): 89-98.

[47] 杨秀建, 冯旭磊, 张生斌, 等. 基于时空走廊的非结构化环境智能车辆轨迹规划[J]. 汽车工程, 2025.

[48] 乔应. 结构化道路下基于时空联合的自动驾驶轨迹规划算法研究[D]. DOI: 10.27061/d.cnki.ghgdu.2025.003637. 哈尔滨工业大学, 2025.

[49] 徐涛. 考虑车辆间交互的无人车行为决策与运动规划算法研究[D]. DOI: 10.27061/d.cnki.ghgdu.2025.003591. 哈尔滨工业大学, 2025.

[50] 陈志远, 黄海波. 复杂道路环境下智能车辆横纵向避障问题研究[J]. 交通运输工程学报, 2024, 24(1): 178-190.

[51] GUO Y, GUO Z, WANG Y, et al. A survey of autonomous driving scenarios and scenario databases[J]. IEEE Transactions on Intelligent Vehicles, 2024, 9(1): 1040-1057.

[52] KANT K, ZUCKER S W. Toward efficient trajectory planning: The path-velocity decomposition[J]. The International Journal of Robotics Research, 1986, 5(3): 72-89.

[53] LIU C, TOMIZUKA M. Speed profile planning in dynamic environments via temporal optimization[C]//2017 IEEE Intelligent Vehicles Symposium. 2017: 154-159.

[54] Van den BERG J, LIN M, MANOCHA D. Reciprocal velocity obstacles for real-time multi-agent navigation[C]//2008 IEEE International Conference on Robotics and Automation. 2008: 1928-1935.

[55] RÖSMANN C, FEITEN W, WÖSCH T, et al. Trajectory modification considering dynamic constraints of autonomous mobile robots[C]//Proceedings of ROBOTIK 2012, 7th German Conference on Robotics. 2012: 74-79.

[56] RÖSMANN C, FEITEN W, WÖSCH T, et al. Efficient trajectory optimization using a sparse model[C]//2013 European Conference on Mobile Robots (ECMR). 2013: 138-143.

[57] 李浩淼. 考虑障碍物扰动的自动驾驶时空联合规划算法[D]. DOI: 10.27162/d.cnki.gjlin.2025.004150. 吉林大学, 2025.

致　　谢

本论文的完成得到了许多人的帮助和支持。首先，衷心感谢我的指导教师在论文选题、研究方向和写作过程中给予的悉心指导和宝贵建议。感谢计算机工程学院各位老师在本科四年学习期间的教导。

感谢百度 Apollo 团队开源了高质量的自动驾驶平台代码，为本研究提供了坚实的技术基础。感谢 Apollo 开发者社区的技术分享和讨论，帮助我更深入地理解了规划模块的设计思想。

感谢我的同学们在学习和生活中给予的帮助和鼓励。最后，感谢我的家人一直以来的理解和支持。